

BrainRot Tracker

The Code, Explained

A hierarchical walkthrough of the entire codebase — the big picture first, then the UI/UX, then every file and every line of code, with the technology behind each part explained as it appears.

Part 1	The Big Picture
Part 2	UI / UX — the screens and the design system
Part 3	Foundations — manifest, build, configuration
Part 4	Entry point & navigation
Part 5	The data layer — Room, prefs, repository, screen time, cloud sync
Part 6	The services — reel counting, overlay, blocking
Part 7	Notifications, widget & theme
Part 8	UI components
Part 9	The screens & their ViewModels

Kotlin · Jetpack Compose · Room · AccessibilityService · Firebase · $\approx$ 7,900 lines of source, all included · June 2026

How to read this document

The document is **hierarchical**: structure → file → code. Part 1 gives you the mental model (what the app does and how the pieces talk to each other) — read it first; every later page assumes it. Part 2 shows what the user sees, so the UI code in Parts 8–9 has faces to names. Parts 3–9 are the code reference: every source file appears in full, in dependency order (low-level data first, screens last), chopped into logical chunks with an explanation after each chunk.

Code chunks look like this:

```
41 fun example(count: Int): Int {  
42     // numbers on the left are the real line numbers in the file  
43     return count + 1  
44 }
```

...and each is followed by a box like this one explaining what those lines do, why they're written that way, and which technology is involved. Long lines wrap with a █ marker. The PDF sidebar/outline (bookmarks) jumps to any part or file.

Three conventions used throughout:

Term	Meaning
composable	A Kotlin function annotated <code>@Composable</code> that <i>*describes*</i> a piece of UI. Compose re-runs it automatically when data it reads changes ('recomposition').
Flow / StateFlow	A Kotlin coroutines stream of values over time. Room emits a new list whenever a table changes; StateFlow is a Flow that always holds a current value for the UI.
suspend function	A function that can pause without blocking a thread — Kotlin's <code>async/await</code> . All database and network calls here are suspend functions running off the main thread.

PART 1

The Big Picture

BrainRot Tracker is an Android app that counts how many short-form videos (Instagram Reels, YouTube Shorts, TikTok videos, Snapchat Spotlight) you scroll each day, visualises the damage as a brain mascot that degrades, and helps you stop: daily limits, a three-mode blocking overlay, streaks, milestone notifications, a floating live counter, a home-screen widget, and an optional Google-sign-in cloud backup of daily totals.

1.1 · The five stages of brain health



Brainrot



Critical



Overstimulated



Tired



Healthy

0-9 · brain_rot 10-24 · brain_critical 25-49 · brain_overstimulated 50-74 · brain_tired 75-100 · brain_healthy

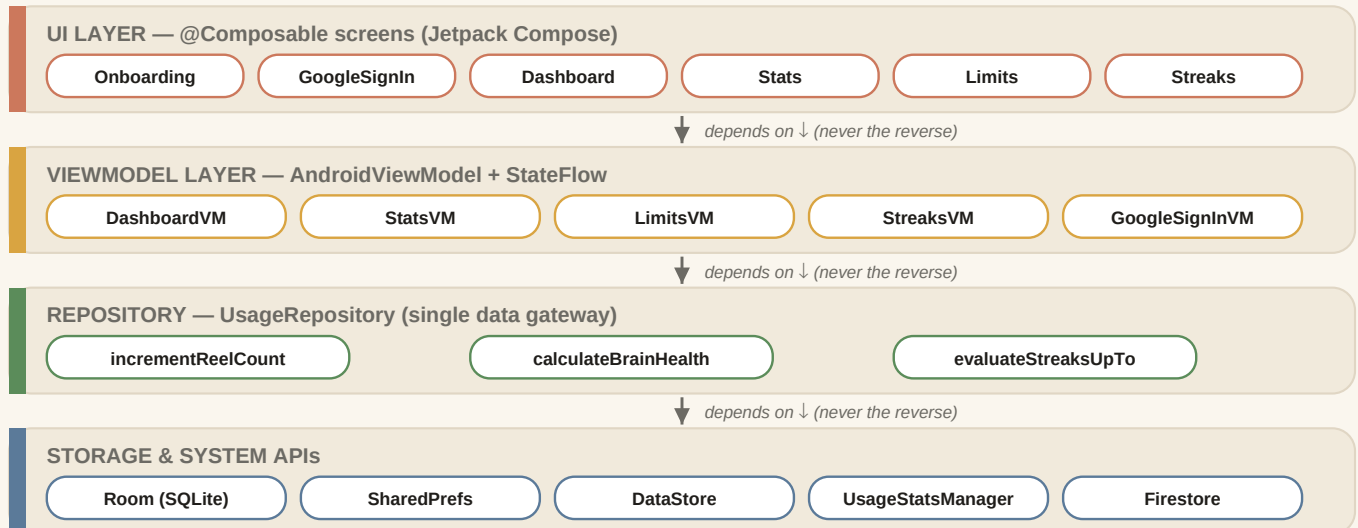
The score (0–100) is recomputed continuously from today's counts and screen-time versus your limits (the exact formula is in Part 5, UsageRepository). Each tier maps to one piece of mascot artwork — shown by the app, the floating pill, and the widget alike.

1.2 · The technology stack

Technology	What it is and what this app uses it for
Kotlin	The language — Google's recommended language for Android. The app leans on coroutines (lightweight threads), Flows (reactive streams), data classes, and sealed classes.
Jetpack Compose	Google's declarative UI toolkit: screens are functions that describe UI for the current state; when state changes, Compose redraws exactly the affected parts. All six screens use it. (The overlay pill cannot — see Part 6.)
Room	A type-safe layer over SQLite (the database engine inside every Android device). Tables are Kotlin data classes; queries are checked at compile time; reads can be live Flows.
AccessibilityService API	The OS feature built for screen readers. Grants the app a machine-readable view of other apps' screens — the core trick that makes reel counting possible at all. Whitelisted to 4 apps.
UsageStatsManager	The system's app-usage log (what Digital Wellbeing reads). Used for exact per-app screen-time minutes.
SYSTEM_ALERT_WINDOW	The 'draw over other apps' permission — powers the floating counter pill and the blocking scrim.
Navigation 3	Compose-native navigation (alpha): the back stack is a plain list of serializable key objects.
Glance	Compose-style syntax for home-screen widgets (compiles to RemoteViews).
DataStore + SharedPreferences	Two key-value stores: DataStore (modern, reactive) for sign-in state; SharedPreferences for settings services must read synchronously.
Firebase Auth + Firestore	Optional: Google sign-in and a per-user cloud document store for daily aggregate backup. The app builds and runs without any Firebase config.
Gradle + R8 + KSP	The build pipeline: Gradle assembles the APK, KSP runs Room's code generator, R8 shrinks/obfuscates release builds, desugaring backports java.time to Android 7.

1.3 · Architecture — MVVM in four layers

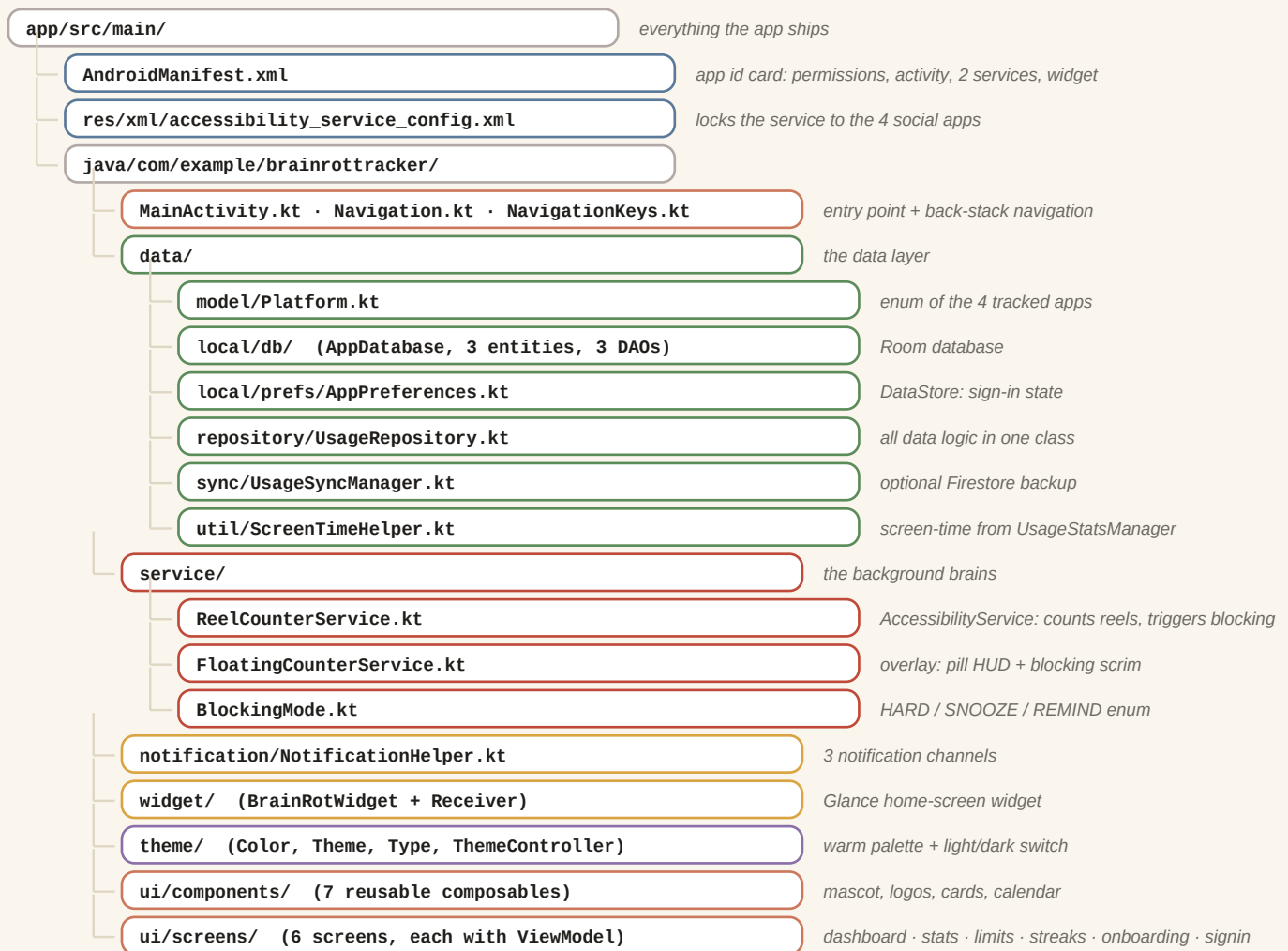
The codebase follows **MVVM** (Model–View–ViewModel). Screens are deliberately dumb: they render state and forward taps. ViewModels own the state as StateFlows and survive rotation. One repository is the only gateway to storage. Dependencies point strictly downward — UI code never touches a DAO, and the data layer knows nothing about screens. The two services live *outside* this stack (they have no UI) but respect the same rule: all data access goes through UsageRepository.



The four layers with the real class names. Each screen's ViewModel exposes StateFlows; screens collect them with `collectAsState()`.

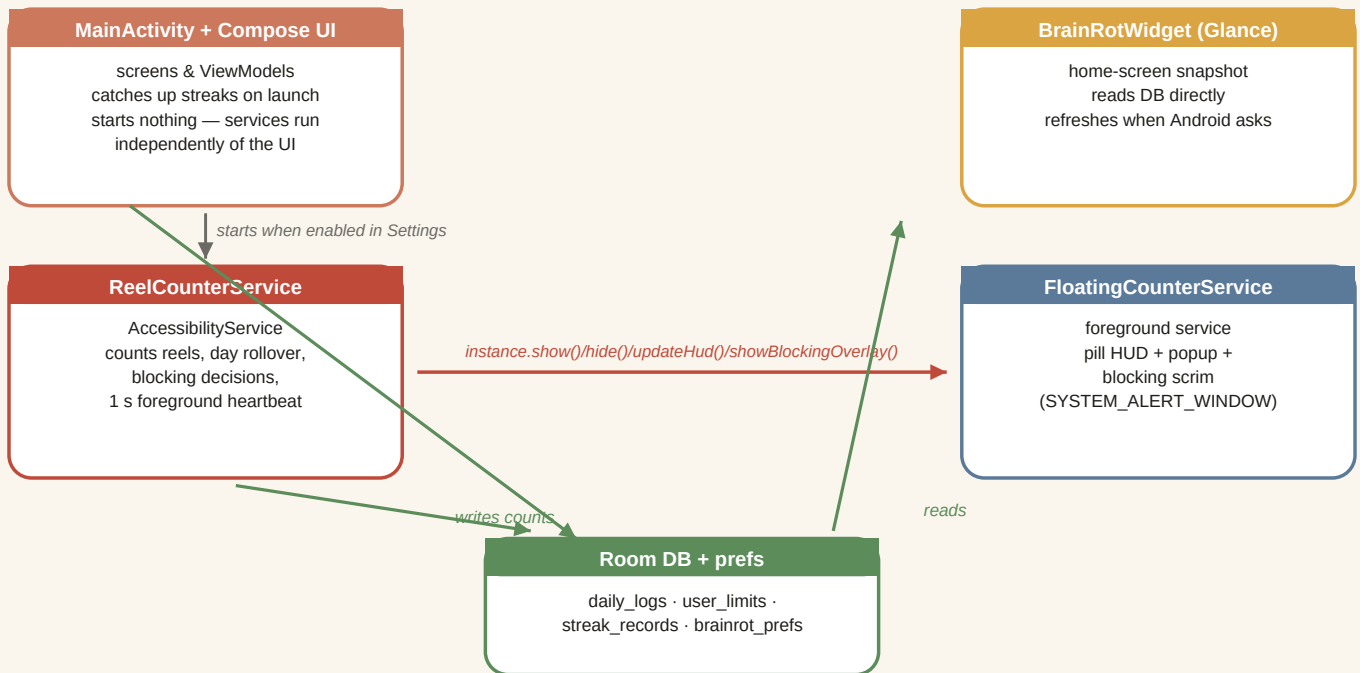
1.4 · The source tree

Single Gradle module. Everything lives under `app/src/main` — packages by responsibility:



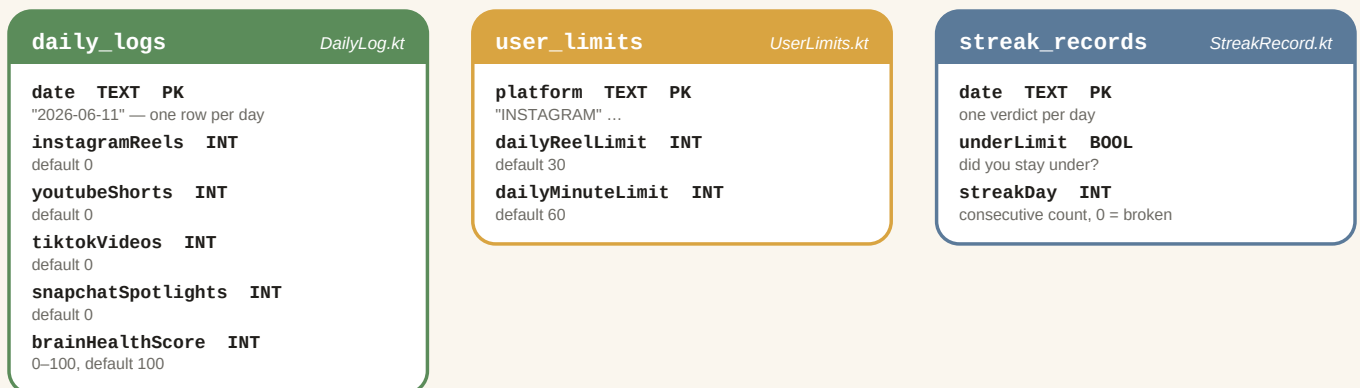
1.5 · The runtime picture

At runtime the app is really **three independent actors sharing one database**: the UI (only alive while you're in the app), the accessibility service (alive whenever enabled — this is the actual product), and the overlay service it commands. The widget is a fourth, passive reader. This is why the database, not any screen, is the source of truth — the UI can be dead while counting continues.



Cross-component calls go through static instance handles (ReelCounterService.instance, FloatingCounterService.instance) — pragmatic glue in lieu of a DI framework.

1.6 · The data model



Three tables, all keyed by ISO date strings (they sort correctly as text). Screen-time minutes are deliberately NOT stored — they're recomputed live from the system's usage log (Part 5 explains why).

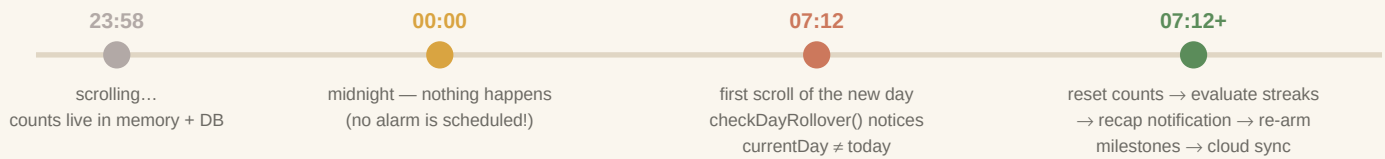
1.7 · How a swipe becomes a number — the core pipeline



The full implementation (with all its defensive details) is Part 6's ReelCounterService.

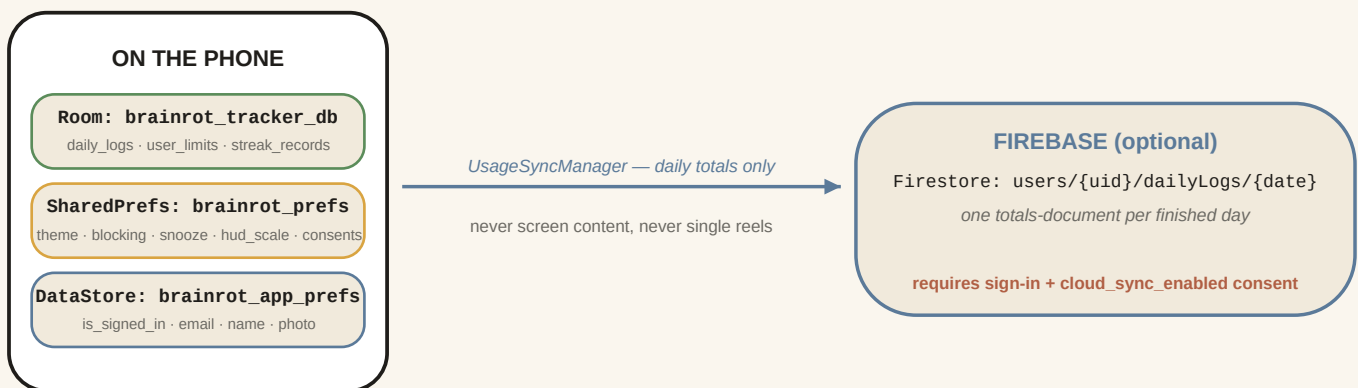
1.8 · The day-rollover trick — no alarms, ever

Android aggressively kills scheduled background work, so the app schedules **nothing**. Instead, every interesting moment (each accessibility event, each app launch, each service connect) cheaply checks 'is it still the same day / are there unevaluated days?' and catches up if not. Missed midnights are simply processed at the next opportunity, and every catch-up routine is idempotent (safe to run twice).



checkDayRollover() in ReelCounterService — triggered by the first event after midnight, whenever that happens to be.

1.9 · Where data lives — and what (optionally) leaves the phone



PART 2

UI / UX — Screens & Design System

Before reading UI code, meet the UI. The app has six screens (four tabs + two full-screen flows) plus three overlay surfaces drawn over other apps. The design language is a warm cream-and-terracotta palette with a single accent color, serif display titles, and the brain mascot as the emotional anchor.

2.1 · The six screens



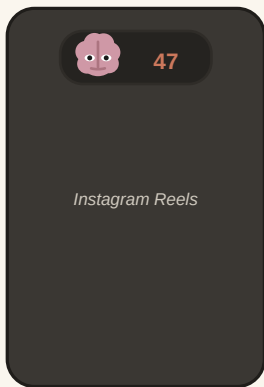
Onboarding and Sign-in are full-screen flows that hide the tab bar; the four tabs share one back stack that resets on every tab switch.

2.2 · Screen-by-screen UX intent

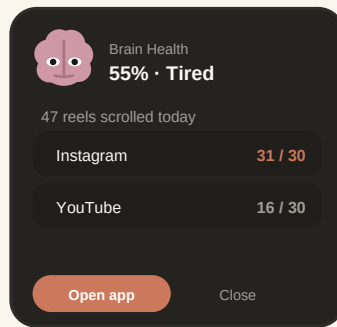
Screen	Job	Key UX decisions
Onboarding	Get 4 special permissions granted	Live GRANTED/PENDING badges flip on return from Settings; Play-policy disclosure dialog before accessibility; everything skippable.
Google Sign-in	Optional account link	Offered exactly once after onboarding; separate explicit consent dialog for cloud backup; app fully usable anonymously.
Dashboard	Today, at a glance	Mascot + speech bubble personify the damage; giant reel count; score ring; tap-to-cycle insights; usage-permission banner only when needed.
Stats	The week in review	Semicircle productivity gauge; stacked per-platform bars; per-day mini-mascots under the chart; best/worst day cards; ranked top apps.
Goals (Limits)	Settings hub	Sliders commit on release (not per-pixel); blocking mode picker with plain-language explainer; theme switch recolors instantly; account + HUD size.
Streaks	Motivation loop	Big flame number + record; month calendar of green/red day circles; five achievements with locked/done states.

2.3 · The overlay surfaces

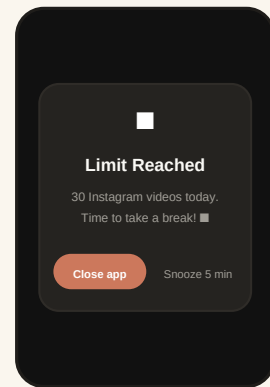
Three surfaces are drawn *over other apps* by FloatingCounterService — they are classic Android Views, not Compose, because overlay windows predate Compose:



The pill (drag = move, tap = popup)



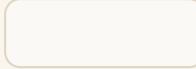
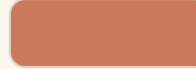
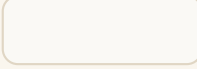
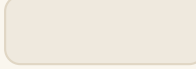
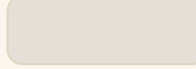
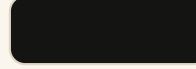
Tap-to-open popup (per-platform breakdown)



Mode-aware blocking scrim (0xDD black)

2.4 · The design system

One palette, two modes. Dark mode uses warm near-blacks (not pure black), light mode warm creams (not pure white). Terracotta #CC785C is the only accent and is identical in both modes — CTAs, selection, the tracking badge, the pill's count. Status colors: green #5DB872 (granted/streak/healthy), red #C64545 (over limit/danger), gold for the mid tier.

 WarmBackground #181715 · dark bg	 WarmSurface #252320 · dark card	 WarmBorder #2F2C28 · dark border	 WarmText #FAF9F5 · dark text	 WarmTextSecondary #A09D96 · dark sub	 WarmAccent #CC785C · terracotta CTA
 WarmGrantedGreen #5DB872 · success	 WarmError #C64545 · danger	 WarmLightBackground #FAF9F5 · light bg	 WarmLightSurface #EFE9DE · light card	 WarmLightBorder #E6DFD8 · light border	 WarmLightText #141413 · light text

Design rule	Where you see it
One accent color	Terracotta everywhere a choice or action lives; everything else is neutral — the eye finds the action instantly.
Cards on canvas	Every screen is a column of rounded cards (12–16 dp radius) on the background; borders only in light mode, tone contrast in dark.
Type hierarchy	36 sp serif-feel titles, 16 sp medium card titles, 10–11 sp ALL-CAPS letter-spaced labels for metadata.
The mascot is the message	Numbers say what happened; the brain's face says what it means. Same artwork at every size (hero, chart row, pill, widget).
Character art as empathy	Nine char_* illustrations (proud, excited, overwhelmed, stop, warrior, trophy...) react to the user's data on every screen.
Instant feedback	Theme switch recolors the same frame; permission badges flip on return; pill pop-in animation; insight dots advance on tap.

PART 3

Foundations — Manifest, Build & Configuration

Three files no code runs without: the manifest (what the app is), the accessibility config (what the service may see), and the Gradle build recipe (how source becomes an APK).

AndroidManifest.xml — the app's identity card

app/src/main/AndroidManifest.xml · 68 lines · Configuration · XML

Every Android app must declare *what it is* and *what it needs* in this file before the system will run a single line of its code. Android reads it at install time to know which permissions to enforce, which screen to launch, and which background components exist.

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
```

The XML header. The **manifest** tag opens the file; the two **xmlns** attributes declare the *android:* and *tools:* attribute namespaces so the rest of the file can use them. *tools:* attributes only affect the build tools, never the device.

```
4
5     <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW" />
6     <uses-permission android:name="android.permission.PACKAGE_USAGE_STATS"
7         tools:ignore="ProtectedPermissions" />
8     <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
9     <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
10    <uses-permission android:name="android.permission.FOREGROUND_SERVICE_SPECIAL_USE" />
11    <uses-permission android:name="android.permission.VIBRATE" />
12    <uses-permission android:name="android.permission.INTERNET" />
```

The seven permissions the app requests. **SYSTEM_ALERT_WINDOW** lets it draw the floating pill and blocking scrim over other apps. **PACKAGE_USAGE_STATS** is a special permission (granted in system settings, not a popup) for reading screen-time data — the *tools:ignore* silences the lint warning that it's normally reserved for system apps. **POST_NOTIFICATIONS** is the Android 13+ runtime notification permission. The two **FOREGROUND_SERVICE*** entries allow FloatingCounterService to run persistently with the 'specialUse' type. **VIBRATE** backs the milestone channel's vibration and **INTERNET** is needed only for the optional Firebase sync.

```
13
14    <application
15        android:allowBackup="true"
16        android:icon="@mipmap/ic_launcher"
17        android:label="@string/app_name"
18        android:roundIcon="@mipmap/ic_launcher_round"
19        android:supportsRtl="true"
20        android:windowSoftInputMode="adjustResize"
21        android:theme="@style/Theme.BrainRotTracker">
```

The **<application>** element configures the whole app: backup allowed, launcher icons, the display name (from strings.xml), right-to-left layout support, keyboard resize behaviour, and the Material theme applied before Compose takes over.

```
22
23    <activity
24        android:name=".MainActivity"
25        android:exported="true">
26        <intent-filter>
27            <action android:name="android.intent.action.MAIN" />
28            <category android:name="android.intent.category.LAUNCHER" />
29        </intent-filter>
30    </activity>
```

MainActivity is the single Activity (screen container). The intent-filter with ACTION_MAIN + CATEGORY_LAUNCHER is what makes the icon appear in the launcher; *exported=true* is mandatory for launcher activities so the home screen can start it.

```

31
32     <!-- Accessibility Service for reel detection -->
33     <service
34         android:name=".service.ReelCounterService"
35         android:permission="android.permission.BIND_ACCESSIBILITY_SERVICE"
36         android:exported="false">
37         <intent-filter>
38             <action android:name="android.accessibilityservice.AccessibilityService" />
39         </intent-filter>
40         <meta-data
41             android:name="android.accessibilityservice"
42             android:resource="@xml/accessibility_service_config" />
43     </service>

```

The accessibility service declaration. **android:permission=BIND_ACCESSIBILITY_SERVICE** means only the operating system itself can bind to (start) this service — another app can't hijack it. The **meta-data** line points to `accessibility_service_config.xml` (next file), which tells Android which apps and event types to forward.

```

44
45     <!-- Floating counter overlay service (only started from inside the app) -->
46     <service
47         android:name=".service.FloatingCounterService"
48         android:exported="false"
49         android:foregroundServiceType="specialUse" />

```

`FloatingCounterService` is declared as a not-exported foreground service of type **specialUse** — the catch-all type Android 14+ requires you to justify (see the `<property>` below). Not exported = no other app can start it.

```

50
51     <!-- Home screen widget -->
52     <receiver
53         android:name=".widget.BrainRotWidgetReceiver"
54         android:exported="true">
55         <intent-filter>
56             <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
57         </intent-filter>
58         <meta-data
59             android:name="android.appwidget.provider"
60             android:resource="@xml/brainrot_widget_info" />
61     </receiver>

```

The home-screen widget. Widgets are delivered through a **BroadcastReceiver**: Android sends `APPWIDGET_UPDATE` broadcasts when the widget should refresh, and the meta-data points to `brainrot_widget_info.xml` describing its size and preview.

```

62
63         <property
64             android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
65             android:value="Tracks screen time on social media apps to help users manage their digital wellness"
66         />
67     </application>
68 </manifest>

```

The **PROPERTY_SPECIAL_USE_FGS_SUBTYPE** property is the human-readable justification string for the `specialUse` foreground service — Google Play reviewers read this text when approving the app.

accessibility_service_config.xml — the service's leash

`app/src/main/res/xml/accessibility_service_config.xml` · 10 lines · Configuration · XML

This tiny file is one of the most important privacy controls in the app: it tells Android exactly what the accessibility service may see.

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <accessibility-service xmlns:android="http://schemas.android.com/apk/res/android"
3     android:description="@string/accessibility_service_description"
4     android:accessibilityEventTypes="typeAllMask"
5     android:accessibilityFeedbackType="feedbackGeneric"

```

```

6     android:canRetrieveWindowContent="true"
7     android:notificationTimeout="100"
8     android:accessibilityFlags="flagReportViewIds"
9     android:settingsActivity="com.example.brainrottracker.MainActivity"
10    android:packageNames="com.instagram.android,com.google.android.youtube,com.zhiliaoapp.musically,com.snapchat
    ■ .android" />

```

Line by line: **description** is the text shown in Settings → Accessibility when the user enables the service. **accessibilityEventTypes=typeAllMask** subscribes to all event types (scrolls, window changes, content changes — the service filters further in code). **canRetrieveWindowContent=true** is the key capability: it allows reading the view tree of the foreground app. **notificationTimeout=100** batches events so the service is pinged at most every 100 ms. **flagReportViewIds** makes Android include view IDs like *com.google.android.youtube:id/reel_recycler* — YouTube detection depends on it. Finally, **packageNames** is the hard whitelist: Android will *only* deliver events from Instagram, YouTube, TikTok and Snapchat. The service is structurally unable to observe your bank app, browser, or anything else.

build.gradle.kts (app module) — the build recipe

app/build.gradle.kts · 146 lines · Build configuration · Gradle Kotlin DSL

Gradle is Android's build system: it compiles the Kotlin, processes resources, pulls in libraries and packages everything into an APK. This file is the recipe for the *app* module. It is written in the Kotlin DSL (hence *.kts*).

```

1  import java.util.Properties
2
3  plugins {
4      alias(libs.plugins.android.application)
5      alias(libs.plugins.compose.compiler)
6      alias(libs.plugins.kotlin.serialization)
7      alias(libs.plugins.ksp)
8  }

```

Four plugins teach Gradle new tricks: **android-application** (build an APK), **compose-compiler** (the Compose compiler plugin that turns `@Composable` functions into efficient UI code), **kotlin-serialization** (generates serializers — needed for the `@Serializable` navigation keys), and **ksp** (Kotlin Symbol Processing — runs Room's annotation processor that generates the database code). The *libs.plugins.** aliases come from *gradle/libs.versions.toml*, the central version catalog.

```

9
10 // google-services hard-fails without google-services.json, so only apply it once the
11 // Firebase project is set up (download the json from the Firebase console into app/).
12 if (file("google-services.json").exists()) {
13     apply(plugin = libs.plugins.google.services.get().pluginId)
14 }

```

Conditional Firebase: the *google-services* plugin crashes the build if *google-services.json* is missing, so it's only applied when that file exists. This is why the project compiles and runs even before a Firebase project is created.

```

15
16 // Release signing credentials live in an untracked keystore.properties (see
17 // keystore.properties.example). Builds without it fall back to unsigned release.
18 val keystorePropsFile = rootProject.file("keystore.properties")
19 val keystoreProps = Properties().apply {
20     if (keystorePropsFile.exists()) keystorePropsFile.inputStream().use { load(it) }
21 }

```

Release signing credentials are read from an untracked *keystore.properties* file — passwords never enter git. If the file is absent, the *Properties* object stays empty and signing is simply skipped.

```

22
23 android {
24     namespace = "com.example.brainrottracker"
25     compileSdk = 36
26     defaultConfig {
27         // TODO: confirm final id before first Play upload — it is immutable once published
28         applicationId = "io.github.aasarmehdi.brainrot"
29         minSdk = 24
30         targetSdk = 36
31         versionCode = 1
32         versionName = "1.0"
33     }

```

Core identity: **namespace** is the Kotlin package used for generated code (kept as `com.example...`), while **applicationId** is the public Play Store identity — they're deliberately different. **minSdk 24** = runs on Android 7.0+; **targetSdk/compileSdk 36** = built against Android 16 behaviour.

```

34
35     signingConfigs {
36         if (keystorePropsFile.exists()) {
37             create("release") {
38                 storeFile = rootProject.file(keystoreProps.getProperty("storeFile"))
39                 storePassword = keystoreProps.getProperty("storePassword")
40                 keyAlias = keystoreProps.getProperty("keyAlias")
41                 keyPassword = keystoreProps.getProperty("keyPassword")
42             }
43         }
44     }
45
46     buildTypes {
47         release {
48             isMinifyEnabled = true
49             isShrinkResources = true
50             proguardFiles(getDefaultProguardFile("proguard-android-optimize.txt"), "proguard-rules.pro")
51             if (keystorePropsFile.exists()) {
52                 signingConfig = signingConfigs.getByName("release")
53             }
54         }
55     }

```

Signing + release build type. Release builds turn on **R8** (`isMinifyEnabled` — removes unused code and obfuscates names) and resource shrinking, guided by `proguard-rules.pro`. Smaller APK, harder to reverse-engineer.

```

56     compileOptions {
57         sourceCompatibility = JavaVersion.VERSION_17
58         targetCompatibility = JavaVersion.VERSION_17
59         // java.time is used throughout but minSdk is 24 (java.time needs API 26)
60         isCoreLibraryDesugaringEnabled = true
61     }
62     buildFeatures {
63         compose = true
64         aidl = false
65         buildConfig = false
66         shaders = false
67     }
68
69     packaging {
70         resources {
71             excludes += "/META-INF/{AL2.0,LGPL2.1}"
72         }
73     }

```

Core library desugaring (line 60) is the clever bit: `java.time` (`LocalDate` etc.) only exists on Android 8+, but `minSdk` is 7.0 — desugaring rewrites those calls at build time into backported equivalents. `buildFeatures` turns Compose on and disables unused generators (AIDL, BuildConfig, shaders) for faster builds. `packaging` excludes duplicate license files that would otherwise collide.

```

74 }
75
76 kotlin {
77     jvmToolchain(17)
78 }

```

All Kotlin compilation uses the Java 17 toolchain.

```

79
80     dependencies {
81         coreLibraryDesugaring(libs.desugar.jdk.libs)
82
83         val composeBom = platform(libs.androidx.compose.bom)
84         implementation(composeBom)
85         androidTestImplementation(composeBom)
86
87         // Core Android dependencies
88         implementation(libs.androidx.core.ktx)
89         implementation(libs.androidx.lifecycle.runtime.ktx)
90         implementation(libs.androidx.activity.compose)
91
92         // Arch Components
93         implementation(libs.androidx.lifecycle.runtime.compose)
94         implementation(libs.androidx.lifecycle.viewmodel.compose)
95     }

```

```

96 // Compose
97 implementation(libs.androidx.compose.ui)
98 implementation(libs.androidx.compose.ui.tooling.preview)
99 implementation(libs.androidx.compose.material3)
100 // Tooling
101 debugImplementation(libs.androidx.compose.ui.tooling)
102 // Instrumented tests
103 androidTestImplementation(libs.androidx.compose.ui.test.junit4)
104 debugImplementation(libs.androidx.compose.ui.test.manifest)
105
106 // Local tests: junit, coroutines, Android runner
107 testImplementation(libs.junit)
108 testImplementation(libs.kotlinx.coroutines.test)
109
110 // Instrumented tests: junit rules and runners
111 androidTestImplementation(libs.androidx.test.core)
112 androidTestImplementation(libs.androidx.test.ext.junit)
113 androidTestImplementation(libs.androidx.test.runner)
114 androidTestImplementation(libs.androidx.test.espresso.core)

```

Dependency block, part 1. The **Compose BOM** (Bill of Materials, line 83) pins all Compose library versions to a tested set so you never mix incompatible versions. Then: core KTX extensions, lifecycle-aware coroutine scopes, Activity↔Compose bridge, ViewModel support, the Compose UI + Material 3 toolkit, preview tooling, and the three test scopes (unit tests, instrumented tests, Espresso).

```

115
116 // Navigation
117 implementation(libs.androidx.navigation3.ui)
118 implementation(libs.androidx.navigation3.runtime)
119 implementation(libs.androidx.lifecycle.viewmodel.navigation3)
120
121 // Room
122 implementation(libs.androidx.room.runtime)
123 implementation(libs.androidx.room.ktx)
124 ksp(libs.androidx.room.compiler)
125
126 // Glance widget
127 implementation(libs.androidx.glance.appwidget)
128 implementation(libs.androidx.glance.material3)
129
130 // DataStore
131 implementation(libs.androidx.datastore.preferences)
132
133 // Google Sign-In (Credential Manager)
134 implementation(libs.androidx.credentials)
135 implementation(libs.androidx.credentials.play.services)
136 implementation(libs.googleid)
137
138 // Firebase (Auth + Firestore for optional cloud backup of daily aggregates)
139 implementation(platform(libs.firebase.bom))
140 implementation(libs.firebase.auth)
141 implementation(libs.firebase.firestore)
142 implementation(libs.kotlinx.coroutines.play.services)
143
144 // Material Icons Extended
145 implementation(libs.androidx.compose.material.icons.extended)
146 }

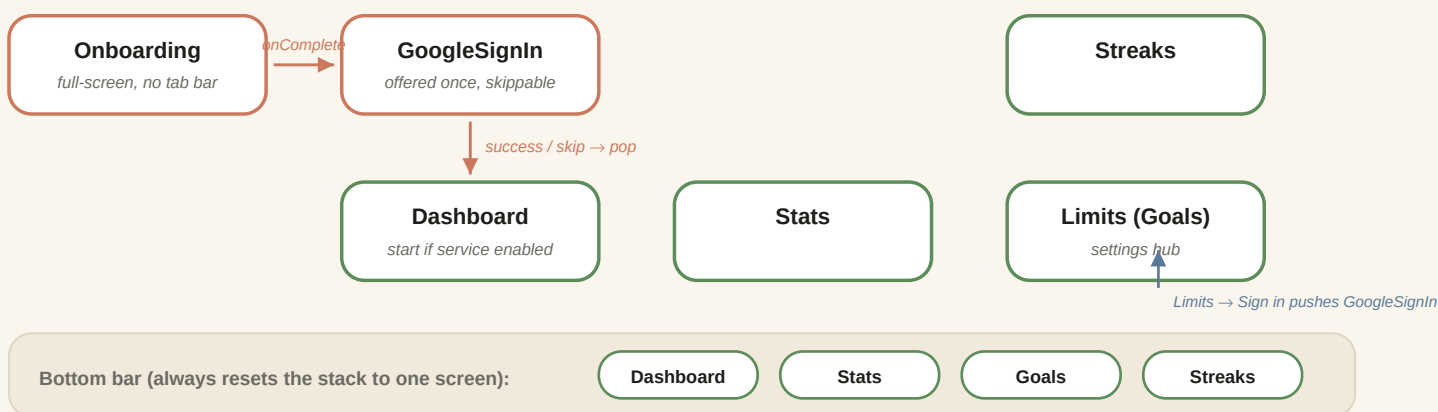
```

Dependency block, part 2 — the feature libraries this guide keeps referring to: **Navigation 3** (Compose-native back-stack), **Room** (runtime + Kotlin extensions + KSP compiler), **Glance** (widgets in Compose syntax), **DataStore** (modern key-value storage), **Credential Manager + googleid** (the Google sign-in sheet), **Firebase Auth + Firestore** via the Firebase BOM, a coroutines adapter for Google Play Services Tasks (.await()), and the extended Material icon set.

PART 4

Entry Point & Navigation

How the app starts and how screens are wired together. Android creates MainActivity; MainActivity hands over to Compose; one back-stack list drives everything else.



The navigation graph. Keys are pushed/popped on a single rememberNavController list; the tab bar resets the stack so Back never replays tab history.

MainActivity.kt — the front door

app/src/main/java/com/example/brainrottracker/MainActivity.kt · 58 lines · UI entry point · 58 lines

An Activity is Android's 'window' object — the OS creates one when the user taps the app icon. This app has exactly one Activity; every visible screen lives inside it as Compose content. MainActivity therefore does only startup chores and hands over to MainNavigation().

```

1 package com.example.brainrottracker
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.compose.foundation.layout.fillMaxSize
8 import androidx.compose.material3.MaterialTheme
9 import androidx.compose.material3.Surface
10 import androidx.compose.ui.Modifier
11 import androidx.lifecycle.LifecycleScope
12 import com.example.brainrottracker.data.local.db.AppDatabase
13 import com.example.brainrottracker.data.repository.UsageRepository
14 import com.example.brainrottracker.data.sync.UsageSyncManager
15 import com.example.brainrottracker.notification.NotificationHelper
16 import com.example.brainrottracker.theme.BrainRotTrackerTheme
17 import com.example.brainrottracker.theme.ThemeController
18 import kotlinx.coroutines.Dispatchers
19 import kotlinx.coroutines.launch
20 import java.time.LocalDate
21
  
```

Imports. Note the mix: Android framework (Bundle, ComponentActivity), Compose (setContent, MaterialTheme), the app's own data classes, and kotlinx.coroutines. **lifecycleScope** is a coroutine scope that is automatically cancelled when the Activity is destroyed — fire-and-forget work can't leak.

```

22 class MainActivity : ComponentActivity() {
23     override fun onCreate(savedInstanceState: Bundle?) {
24         super.onCreate(savedInstanceState)
25
26         // Initialize database singleton
27         val db = AppDatabase.getInstance(applicationContext)
  
```

onCreate is the Activity's birth callback. The first job is warming up the Room database singleton with **applicationContext** (the process-wide context) rather than the Activity itself — using the Activity here would leak it, because the DB singleton outlives every screen.

```

28
29     // Catch up streak evaluation in case the tracking service wasn't running at midnight,
30     // then push any unsynced days to the cloud (no-op unless signed in with backup on)
31     lifecycleScope.launch(Dispatchers.IO) {
32         val repository = UsageRepository(db)
33         repository.evaluateStreaksUpTo(LocalDate.now().minusDays(1))
34         try {
35             UsageSyncManager(applicationContext, repository).syncIfEnabled()
36         } catch (_: Exception) {
37         }
38     }

```

Startup catch-up, off the main thread (**Dispatchers.IO**): (1) *evaluateStreaksUpTo(yesterday)* writes streak verdicts for any days that ended while the tracking service wasn't running — this is one of the three triggers of the lazy streak system. (2) *syncIfEnabled()* pushes unsynced days to Firestore; it's a no-op unless the user signed in AND consented, and the empty catch means a missing Firebase config can never crash startup.

```

39
40     // Load saved theme preference
41     ThemeController.init(applicationContext)
42
43     // Create notification channels
44     NotificationHelper(applicationContext).createChannels()

```

Two synchronous initialisations that must happen before any UI: ThemeController loads the saved Light/Dark/System choice into Compose state, and NotificationHelper registers the three notification channels (channels must exist before any notification is posted).

```

45
46     enableEdgeToEdge()
47     setContent {
48         BrainRotTrackerTheme {
49             Surface(
50                 modifier = Modifier.fillMaxSize(),
51                 color = MaterialTheme.colorScheme.background
52             ) {
53                 MainNavigation()
54             }
55         }
56     }
57 }

```

enableEdgeToEdge() lets content draw behind the status/navigation bars (modern Android look). **setContent { }** is the bridge from the classic Activity world into Compose: everything inside is declarative UI. The theme wrapper applies Material 3 colors/typography, Surface paints the window background, and **MainNavigation()** (next file) takes over for the rest of the app's life.

```

58 }

```

NavigationKeys.kt — one tiny key per screen

app/src/main/java/com/example/brainrottracker/NavigationKeys.kt · 11 lines · Navigation · 11 lines

Navigation 3 represents 'where you can go' as plain values. Each screen gets a **data object** — a singleton with no fields — implementing the NavKey marker interface.

```

1 package com.example.brainrottracker
2
3 import androidx.navigation3.runtime.NavKey
4 import kotlinx.serialization.Serializable
5
6 @Serializable data object GoogleSignIn : NavKey
7 @Serializable data object Onboarding : NavKey
8 @Serializable data object Dashboard : NavKey
9 @Serializable data object Stats : NavKey
10 @Serializable data object Limits : NavKey
11 @Serializable data object Streaks : NavKey

```


@Serializable lets Android save the back stack to disk when the process is killed (e.g. user switches apps for an hour) and restore it on return — this is why the kotlin-serialization plugin is in the build, and why the ProGuard rules must keep these serializers in release builds. A **data object** (Kotlin 1.9+) is like an object but with a sensible toString/equals. The whole navigation 'graph' of the app is just these six values; pushing *Stats* onto a list shows the *Stats* screen.

Navigation.kt — back stack, bottom bar, screen wiring

app/src/main/java/com/example/brainrottracker/Navigation.kt · 219 lines · Navigation shell · 219 lines

This file owns the single back stack that drives the whole app, decides which screen the app starts on, and draws the custom bottom tab bar. It uses **Navigation 3** (alpha), where navigation state is literally just a list you mutate: add a key = go forward, remove = go back.

```
1 package com.example.brainrottracker
2
3 import android.provider.Settings
4 import androidx.compose.animation.AnimatedVisibility
5 import androidx.compose.animation.fadeIn
6 import androidx.compose.animation.fadeOut
7 import androidx.compose.animation.slideInVertically
8 import androidx.compose.animation.slideOutVertically
9 import androidx.compose.foundation.background
10 import androidx.compose.foundation.clickable
11 import androidx.compose.foundation.interaction.MutableInteractionSource
12 import androidx.compose.foundation.layout.Arrangement
13 import androidx.compose.foundation.layout.Box
14 import androidx.compose.foundation.layout.Column
15 import androidx.compose.foundation.layout.Row
16 import androidx.compose.foundation.layout.Spacer
17 import androidx.compose.foundation.layout.fillMaxSize
18 import androidx.compose.foundation.layout.fillMaxWidth
19 import androidx.compose.foundation.layout.height
20 import androidx.compose.foundation.layout.navigationBarsPadding
21 import androidx.compose.foundation.layout.padding
22 import androidx.compose.foundation.layout.size
23 import androidx.compose.material.icons.Icons
24 import androidx.compose.material.icons.filled.BarChart
25 import androidx.compose.material.icons.filled.Dashboard
26 import androidx.compose.material.icons.filled.LocalFireDepartment
27 import androidx.compose.material.icons.filled.TrackChanges
28 import androidx.compose.material3.HorizontalDivider
29 import androidx.compose.material3.Icon
30 import androidx.compose.material3.Scaffold
31 import androidx.compose.material3.Text
32 import androidx.compose.runtime.Composable
33 import androidx.compose.runtime.getValue
34 import androidx.compose.runtime.mutableStateOf
35 import androidx.compose.runtime.produceState
36 import androidx.compose.runtime.remember
37 import androidx.compose.runtime.setValue
38 import androidx.compose.ui.Alignment
39 import androidx.compose.ui.Modifier
40 import androidx.compose.ui.graphics.vector.ImageVector
41 import androidx.compose.ui.platform.LocalContext
42 import androidx.compose.ui.text.font.FontWeight
43 import androidx.compose.ui.unit.dp
44 import androidx.compose.ui.unit.sp
45 import androidx.navigation3.runtime.NavKey
46 import androidx.navigation3.runtime.entryProvider
47 import androidx.navigation3.runtime.rememberNavBackStack
48 import androidx.navigation3.ui.NavDisplay
49 import com.example.brainrottracker.theme.WarmAccent
50 import com.example.brainrottracker.theme.WarmBackground
51 import com.example.brainrottracker.theme.WarmBorder
52 import com.example.brainrottracker.theme.WarmLightBackground
53 import com.example.brainrottracker.theme.WarmLightBorder
54 import com.example.brainrottracker.theme.WarmLightTextSecondary
55 import com.example.brainrottracker.theme.WarmTextSecondary
56 import com.example.brainrottracker.data.local.prefs.AppPreferences
57 import com.example.brainrottracker.theme.rememberIsDark
58 import com.example.brainrottracker.ui.screens.dashboard.DashboardScreen
59 import com.example.brainrottracker.ui.screens.limits.LimitsScreen
60 import com.example.brainrottracker.ui.screens.onboarding.OnboardingScreen
61 import com.example.brainrottracker.ui.screens.signin.GoogleSignInScreen
62 import com.example.brainrottracker.ui.screens.stats.StatsScreen
63 import com.example.brainrottracker.ui.screens.streaks.StreaksScreen
```

```
64 import kotlinx.coroutines.flow.first
```

Imports plus two grouped concepts worth noting: the **androidx.navigation3** imports (NavKey, entryProvider, rememberNavBackStack, NavDisplay) are the whole navigation API surface, and the Warm* color imports show that the bar is hand-painted rather than using Material's NavigationBar component — this keeps it visually identical in light/dark.

```
65
66 data class BottomNavItem(
67     val key: NavKey,
68     val label: String,
69     val icon: ImageVector,
70 )
71
72 private data class InitState(val accessibilityEnabled: Boolean, val signIn: Boolean)
```

Two small data holders: **BottomNavItem** bundles a nav key with its label and icon for the tab bar; private **InitState** carries the two facts needed to choose the start screen (is the accessibility service on? is the user signed in?).

```
73
74 @Composable
75 fun MainNavigation() {
76     val context = LocalContext.current
77     val dark = rememberIsDark()
78     val navBg = if (dark) WarmBackground else WarmLightBackground
79     val navBorder = if (dark) WarmBorder else WarmLightBorder
80     val inactiveColor = if (dark) WarmTextSecondary else WarmLightTextSecondary
81
82     // Read startup state asynchronously before building the nav back stack.
83     // We only need the first emission – once we know where to start, we stop collecting.
84     val initState by produceState<InitState?>(initialValue = null) {
85         val accessibilityEnabled = try {
86             val enabledServices = Settings.Secure.getString(
87                 context.contentResolver,
88                 Settings.Secure.ENABLED_ACCESSIBILITY_SERVICES
89             ) ?: ""
90             enabledServices.contains(context.packageName)
91         } catch (_: Exception) { false }
92         val signIn = AppPreferences.isSignedInFlow(context).first()
93         value = InitState(accessibilityEnabled, signIn)
94     }
```

produceState runs a coroutine that computes a Compose state value asynchronously. It reads `ENABLED_ACCESSIBILITY_SERVICES` from `Settings.Secure` (a system string listing all enabled services) and checks whether our package appears in it, then suspends on `DataStore`'s first emission for the sign-in flag. Until both answers arrive, *initState* stays null.

```
95
96     // Show a blank matching-color splash while DataStore emits the first value
97     if (initState == null) {
98         Box(
99             modifier = Modifier
100                 .fillMaxSize()
101                 .background(navBg)
102         )
103         return
104     }
105
106     val startDestination: NavKey = if (initState!!.accessibilityEnabled) Dashboard else Onboarding
107
108     val backStack = rememberNavBackStack(startDestination)
109     var currentDestination by remember { mutableStateOf(startDestination) }
```

While *initState* is null, the function returns early with a `Box` painted in the theme background — a one-frame 'splash' that avoids flashing the wrong start screen. Once loaded, the start destination is chosen (**skip onboarding if the service is already enabled**), **rememberNavBackStack** creates the persistent back-stack list, and *currentDestination* mirrors the selected tab for highlighting.

```
110
111     val bottomNavItems = listOf(
112         BottomNavItem(Dashboard, "Dashboard", Icons.Filled.Dashboard),
113         BottomNavItem(Stats, "Stats", Icons.Filled.BarChart),
114         BottomNavItem(Limits, "Goals", Icons.Filled.TrackChanges),
115         BottomNavItem(Streaks, "Streaks", Icons.Filled.LocalFireDepartment),
116     )
117
118     // Hide the bar on full-screen flows (onboarding, sign-in)
```



```

191             // Offer the optional sign-in once after onboarding
192             if (initState?.signedIn == false) backStack += GoogleSignIn
193         },
194         modifier = Modifier.padding(16.dp)
195     )
196 }
197 entry<GoogleSignIn> {
198     GoogleSignInScreen(
199         onSignInSuccess = { backStack.removeLastOrNull() },
200         onSkip = { backStack.removeLastOrNull() }
201     )
202 }
203 entry<Dashboard> {
204     DashboardScreen()
205 }
206 entry<Stats> {
207     StatsScreen()
208 }
209 entry<Limits> {
210     LimitsScreen(onNavigateToSignIn = { backStack += GoogleSignIn })
211 }
212 entry<Streaks> {
213     StreaksScreen()
214 }
215 }
216 )
217 }
218 }

```

NavDisplay renders whatever key is on top of the stack; **onBack** pops it. **entryProvider** maps each key type to its composable. Two wirings matter: Onboarding's *onComplete* resets the stack to Dashboard and then pushes GoogleSignIn exactly once if the user has never signed in; the Limits screen receives *onNavigateToSignIn* so its Account card can push the sign-in screen later.

```

219 }

```

PART 5

The Data Layer

Bottom-up through the model: the Platform enum, the three Room entities and their DAOs, the database singleton, the two preference stores, the repository (with the health-score and streak algorithms), the screen-time helper, and the optional cloud sync.

Platform.kt — the four tracked apps

app/src/main/java/com/example/brainrottracker/data/model/Platform.kt · 18 lines · Domain model · 18 lines

The single source of truth for which apps are tracked. Nearly every other file iterates *Platform.entries*, so adding a platform here is step 1 of the extension recipe.

```
1 package com.example.brainrottracker.data.model
2
3 enum class Platform(
4     val packageName: String,
5     val displayName: String,
6     val emoji: String
7 ) {
8     INSTAGRAM("com.instagram.android", "Instagram", "📷"),
9     YOUTUBE("com.google.android.youtube", "YouTube", "📺"),
10    TIKTOK("com.zhiliaoapp.musically", "TikTok", "📺"),
11    SNAPCHAT("com.snapchat.android", "Snapchat", "📷");
12
13    companion object {
14        fun fromPackageName(packageName: String): Platform? {
15            return entries.find { it.packageName == packageName }
16        }
17    }
18 }
```

A Kotlin **enum with properties**: each constant carries the app's Android *packageName* (how the OS identifies an app — accessibility events arrive tagged with it), a display name, and an emoji fallback. TikTok's odd package (com.zhiliaoapp.musically) is its real historic ID from the Musical.ly merger. **fromPackageName()** is the reverse lookup the accessibility service calls on every event; returning null for unknown packages is the second line of defence after the config whitelist.

DailyLog.kt — one row per day

app/src/main/java/com/example/brainrottracker/data/local/db/entity/DailyLog.kt · 26 lines · Room entity · 26 lines

A Room **entity** is a Kotlin data class that maps 1:1 to a database table. Room's KSP compiler reads the annotations and generates the SQL plumbing.

```
1 package com.example.brainrottracker.data.local.db.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5 import com.example.brainrottracker.data.model.Platform
6
7 @Entity(tableName = "daily_logs")
8 data class DailyLog(
9     @PrimaryKey
10    val date: String,
11    val instagramReels: Int = 0,
12    val youtubeShorts: Int = 0,
13    val tiktokVideos: Int = 0,
14    val snapchatSpotlights: Int = 0,
15    val brainHealthScore: Int = 100
16 )
```

`@Entity(tableName = "daily_logs")` declares the table; `@PrimaryKey date` (an ISO string like "2026-06-11") guarantees exactly one row per calendar day — string dates sort correctly because ISO format is lexicographic. Each platform's count is its own column (simple, fast UPDATE ... +1 increments), defaulting to 0; the health score defaults to 100 (a fresh day starts with a healthy brain). The trade-off of one-column-per-platform: adding a platform requires a schema change (see CLAUDE.md's extension recipe).

```
16 } {
17     fun getTotalReels(): Int =
18         instagramReels + youtubeShorts + tiktokVideos + snapchatSpotlights
19
20     fun getReelsForPlatform(platform: Platform): Int = when (platform) {
21         Platform.INSTAGRAM -> instagramReels
22         Platform.YOUTUBE -> youtubeShorts
23         Platform.TIKTOK -> tiktokVideos
24         Platform.SNAPCHAT -> snapchatSpotlights
25     }
```

Two convenience helpers so callers never touch columns directly: `getTotalReels()` sums everything, and `getReelsForPlatform()` maps the `Platform` enum to the right column with an exhaustive `when` — if a fifth platform is added to the enum, this file fails to compile until handled, which is exactly the safety net you want.

```
26 }
```

UserLimits.kt — per-platform daily caps

app/src/main/java/com/example/brainrottracker/data/local/db/entity/UserLimits.kt · 12 lines · Room entity · 12 lines

```
1 package com.example.brainrottracker.data.local.db.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "user_limits")
7 data class UserLimits(
8     @PrimaryKey
9     val platform: String,
10     val dailyReelLimit: Int = 30,
11     val dailyMinuteLimit: Int = 60
12 )
```

One row per platform, keyed by the enum's *name* string ("INSTAGRAM"...). Defaults of 30 reels / 60 minutes are duplicated in code wherever no row exists yet — the entity default only applies when a row is created. The Limits screen currently writes the same values to all four rows ('global limits'), but the schema supports per-platform values.

StreakRecord.kt — one verdict per finished day

app/src/main/java/com/example/brainrottracker/data/local/db/entity/StreakRecord.kt · 12 lines · Room entity · 12 lines

```
1 package com.example.brainrottracker.data.local.db.entity
2
3 import androidx.room.Entity
4 import androidx.room.PrimaryKey
5
6 @Entity(tableName = "streak_records")
7 data class StreakRecord(
8     @PrimaryKey
9     val date: String,
10     val underLimit: Boolean,
11     val streakDay: Int
12 )
```

Written once per day by `evaluateStreaksUpTo()`: `underLimit` is the pass/fail verdict, and `streakDay` stores the running count (1, 2, 3..., reset to 0 on a fail). Storing the running count, not just the boolean, lets the next day's evaluation continue the chain with a single previous-row lookup instead of re-walking history.

DailyLogDao.kt — SQL for the daily_logs table

app/src/main/java/com/example/brainrottracker/data/local/db/dao/DailyLogDao.kt · 46 lines · Room DAO · 46 lines

A DAO (Data Access Object) is an interface where each method is a database operation. You write the SQL in annotations; Room generates the implementation and checks the SQL against the schema *at compile time* — a typo in a column name is a build error, not a crash.

```

1  package com.example.brainrottracker.data.local.db.dao
2
3  import androidx.room.Dao
4  import androidx.room.Insert
5  import androidx.room.OnConflictStrategy
6  import androidx.room.Query
7  import com.example.brainrottracker.data.local.db.entity.DailyLog
8  import kotlinx.coroutines.flow.Flow
9
10 @Dao
11 interface DailyLogDao {
12
13     @Insert(onConflict = OnConflictStrategy.REPLACE)
14     suspend fun insertOrUpdate(log: DailyLog)

```

Imports + the upsert. **OnConflictStrategy.REPLACE** means inserting a row whose date already exists overwrites it. **suspend** marks it as a coroutine function — Room runs it on a background thread automatically, so the UI thread can never be blocked by disk I/O.

```

15
16     @Query("SELECT * FROM daily_logs WHERE date = :date")
17     fun getByDate(date: String): Flow<DailyLog?>
18
19     @Query("SELECT * FROM daily_logs WHERE date = :date LIMIT 1")
20     suspend fun getByDateOnce(date: String): DailyLog?
21
22     @Query("SELECT * FROM daily_logs WHERE date BETWEEN :startDate AND :endDate ORDER BY date ASC")
23     fun getDateRange(startDate: String, endDate: String): Flow<List<DailyLog>>
24
25     @Query("SELECT * FROM daily_logs ORDER BY date ASC")
26     fun getAllLogs(): Flow<List<DailyLog>>

```

Read queries in two flavours, a pattern repeated all over this codebase: **Flow<...>** versions are *live* — Room re-runs the query and pushes a new value every time the underlying table changes, which is how screens update instantly when the service counts a reel. **...Once** suspend versions are one-shot snapshots for background logic that just needs the current value. `getDateRange` powers the weekly and monthly charts; **BETWEEN** works on the ISO strings because they sort lexicographically.

```

27
28     // Per-platform reel/video increment queries
29     @Query("UPDATE daily_logs SET instagramReels = instagramReels + 1 WHERE date = :date")
30     suspend fun incrementInstagramReels(date: String)
31
32     @Query("UPDATE daily_logs SET youtubeShorts = youtubeShorts + 1 WHERE date = :date")
33     suspend fun incrementYoutubeShorts(date: String)
34
35     @Query("UPDATE daily_logs SET tiktokVideos = tiktokVideos + 1 WHERE date = :date")
36     suspend fun incrementTiktokVideos(date: String)
37
38     @Query("UPDATE daily_logs SET snapchatSpotlights = snapchatSpotlights + 1 WHERE date = :date")
39     suspend fun incrementSnapchatSpotlights(date: String)

```

Four atomic increment queries — **UPDATE ... SET col = col + 1** executes inside SQLite, so two rapid counts can't race each other and lose an increment (a read-modify-write in Kotlin could). One method per column because SQL can't parameterise a column name.

```

40
41     @Query("SELECT MIN(date) FROM daily_logs")
42     suspend fun getEarliestDate(): String?
43
44     @Query("UPDATE daily_logs SET brainHealthScore = :score WHERE date = :date")
45     suspend fun updateBrainHealthScore(date: String, score: Int)
46 }

```

`getEarliestDate()` gives the streak evaluator its starting point on first run; `updateBrainHealthScore` persists the score so widget and history see it.

UserLimitsDao.kt

app/src/main/java/com/example/brainrottracker/data/local/db/dao/UserLimitsDao.kt · 24 lines · Room DAO · 24 lines

```

1 package com.example.brainrottracker.data.local.db.dao
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import com.example.brainrottracker.data.local.db.entity.UserLimits
8 import kotlinx.coroutines.flow.Flow
9
10 @Dao
11 interface UserLimitsDao {
12
13     @Query("SELECT * FROM user_limits")
14     fun getAll(): Flow<List<UserLimits>>
15
16     @Query("SELECT * FROM user_limits")
17     suspend fun getAllOnce(): List<UserLimits>
18
19     @Query("SELECT * FROM user_limits WHERE platform = :platform")
20     fun getForPlatform(platform: String): Flow<UserLimits?>
21
22     @Insert(onConflict = OnConflictStrategy.REPLACE)
23     suspend fun upsert(limits: UserLimits)
24 }

```

Same Flow/Once dual pattern: `getAll()` feeds screens live, `getAllOnce()` feeds one-shot logic (health calculation, sync), `getForPlatform()` filters by the enum-name key, and `upsert()` backs the Limits screen's sliders. REPLACE conflict handling makes 'insert or update' a single call.

StreakDao.kt

app/src/main/java/com/example/brainrottracker/data/local/db/dao/StreakDao.kt · 30 lines · Room DAO · 30 lines

```

1 package com.example.brainrottracker.data.local.db.dao
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import com.example.brainrottracker.data.local.db.entity.StreakRecord
8 import kotlinx.coroutines.flow.Flow
9
10 @Dao
11 interface StreakDao {
12
13     @Insert(onConflict = OnConflictStrategy.REPLACE)
14     suspend fun insert(record: StreakRecord)
15
16     @Query("SELECT * FROM streak_records WHERE date = :date")
17     fun getByDate(date: String): Flow<StreakRecord?>
18
19     @Query("SELECT * FROM streak_records WHERE date = :date LIMIT 1")
20     suspend fun getByDateOnce(date: String): StreakRecord?
21
22     @Query("SELECT MAX(date) FROM streak_records")
23     suspend fun getLatestDate(): String?
24
25     @Query("SELECT * FROM streak_records ORDER BY date ASC")
26     fun getAll(): Flow<List<StreakRecord>>
27
28     @Query("SELECT * FROM streak_records WHERE date BETWEEN :startDate AND :endDate ORDER BY date ASC")
29     fun getDateRange(startDate: String, endDate: String): Flow<List<StreakRecord>>
30 }

```

The notable query is `getLatestDate()` (`MAX(date)`) — the streak evaluator uses it to find where it left off, so catch-up after a week offline only processes the missing days. `getAll()` feeds the Streaks screen's calendar and the Dashboard's current-streak counter.

AppDatabase.kt — the Room database singleton

app/src/main/java/com/example/brainrottracker/data/local/db/AppDatabase.kt · 42 lines · Room database · 42 lines

Room sits on top of SQLite, the embedded SQL engine inside every Android device. This class ties the three entities and three DAOs together and guarantees the whole app shares one database connection.

```

1 package com.example.brainrottracker.data.local.db
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7 import com.example.brainrottracker.data.local.db.dao.DailyLogDao
8 import com.example.brainrottracker.data.local.db.dao.StreakDao
9 import com.example.brainrottracker.data.local.db.dao.UserLimitsDao
10 import com.example.brainrottracker.data.local.db.entity.DailyLog
11 import com.example.brainrottracker.data.local.db.entity.StreakRecord
12 import com.example.brainrottracker.data.local.db.entity.UserLimits
13
14 @Database(
15     entities = [DailyLog::class, UserLimits::class, StreakRecord::class],
16     version = 2,
17     exportSchema = false
18 )
19 abstract class AppDatabase : RoomDatabase() {
20     abstract fun dailyLogDao(): DailyLogDao
21     abstract fun userLimitsDao(): UserLimitsDao
22     abstract fun streakDao(): StreakDao

```

@Database lists the entities (tables) and the schema **version = 2** — Room compares this number against what's on disk to detect schema changes. The abstract DAO getters are implemented by generated code at build time.

```

23
24     companion object {
25         @Volatile
26         private var INSTANCE: AppDatabase? = null
27
28         fun getInstance(context: Context): AppDatabase {
29             return INSTANCE ?: synchronized(this) {
30                 INSTANCE ?: Room.databaseBuilder(
31                     context.applicationContext,
32                     AppDatabase::class.java,
33                     "brainrot_tracker_db"
34                 )
35                 // Pre-launch: schema changes recreate the DB instead of migrating
36                 .fallbackToDestructiveMigration(true)
37                 .build()
38                 .also { INSTANCE = it }
39             }
40         }
41     }

```

A textbook thread-safe singleton: **@Volatile** makes the **INSTANCE** field immediately visible across threads, and **double-checked locking** (check → synchronized → check again) ensures exactly one database is ever built even if the UI and the accessibility service race to call `getInstance()` at the same moment. **fallbackToDestructiveMigration(true)** says: on a version bump, wipe and recreate rather than migrate — acceptable pre-launch, but must be replaced with real Migration objects once users have data worth keeping.

```

42 }

```

AppPreferences.kt — sign-in state in DataStore

app/src/main/java/com/example/brainrottracker/data/local/prefs/AppPreferences.kt · 57 lines · Preferences · 57 lines

The app deliberately uses two preference systems. **DataStore** (this file) is the modern, coroutine-friendly store — used for sign-in state that the UI observes reactively. Plain **SharedPreferences** (brainrot_prefs) is used by the services for settings they must read synchronously on the main thread (theme, blocking mode, snooze deadlines).

```

1 package com.example.brainrottracker.data.local.prefs
2
3 import android.content.Context
4 import androidx.datastore.core.DataStore
5 import androidx.datastore.preferences.core.Preferences
6 import androidx.datastore.preferences.core.booleanPreferencesKey
7 import androidx.datastore.preferences.core.edit
8 import androidx.datastore.preferences.core.emptyPreferences
9 import androidx.datastore.preferences.core.stringPreferencesKey
10 import androidx.datastore.preferences.preferencesDataStore

```

```

11 import kotlinx.coroutines.flow.Flow
12 import kotlinx.coroutines.flow.catch
13 import kotlinx.coroutines.flow.map
14
15 private val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name = "brainrot_app_prefs")

```

Line 15 is a Kotlin idiom worth understanding: a **property delegate on Context** creates one process-wide DataStore named `brainrot_app_prefs`, available as `context.dataStore` from anywhere. Declaring it at file level (not inside a class) is what makes it a true singleton.

```

16
17 object AppPreferences {
18
19     private val IS_SIGNED_IN = booleanPreferencesKey("is_signed_in")
20     private val USER_EMAIL = stringPreferencesKey("user_email")
21     private val USER_NAME = stringPreferencesKey("user_name")
22     private val USER_PHOTO_URL = stringPreferencesKey("user_photo_url")
23
24     data class SignedInUser(val email: String, val name: String)
25
26     fun isSignedInFlow(context: Context): Flow<Boolean> =
27         context.dataStore.data
28             .catch { emit(emptyPreferences()) }
29             .map { it[IS_SIGNED_IN] ?: false }
30
31     fun userFlow(context: Context): Flow<SignedInUser?> =
32         context.dataStore.data
33             .catch { emit(emptyPreferences()) }
34             .map { prefs ->
35                 if (prefs[IS_SIGNED_IN] == true) {
36                     SignedInUser(prefs[USER_EMAIL] ?: "", prefs[USER_NAME] ?: "")
37                 } else null
38             }

```

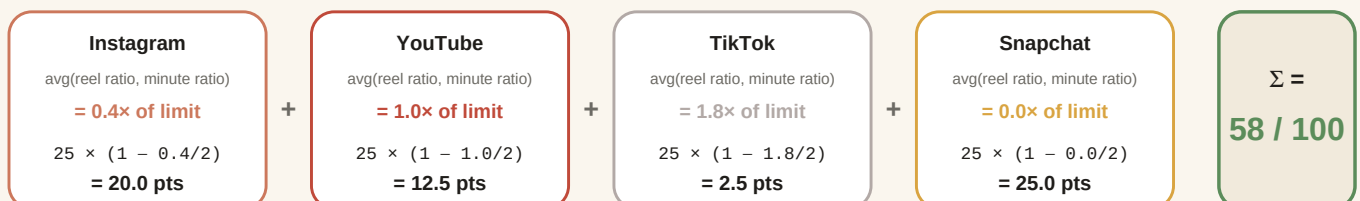
Typed keys (DataStore keys carry their value type) and the two read flows. The `.catch { emit(emptyPreferences()) }` guards against a corrupted preferences file — the app degrades to 'signed out' instead of crashing. `userFlow()` maps the raw prefs into a nullable `SignedInUser`, so the Limits screen can simply check for null.

```

39
40 suspend fun setSignedIn(
41     context: Context,
42     email: String,
43     name: String,
44     photoUrl: String?
45 ) {
46     context.dataStore.edit { prefs ->
47         prefs[IS_SIGNED_IN] = true
48         prefs[USER_EMAIL] = email
49         prefs[USER_NAME] = name
50         prefs[USER_PHOTO_URL] = photoUrl ?: ""
51     }
52 }
53
54 suspend fun signOut(context: Context) {
55     context.dataStore.edit { it.clear() }
56 }
57 }

```

Writes: `edit { }` applies changes atomically (all-or-nothing) and suspends instead of blocking. `signOut()` clears every key — note `photoUrl` is stored but currently unused by the UI.



Each platform contributes up to 25 points. Usage at 2× a limit zeroes that platform. Reel counts come from the DB; minutes come live from UsageStatsManager.

calculateBrainHealth() with example numbers: Instagram at 0.4× of its limits, YouTube at 1.0×, TikTok at 1.8×, Snapchat unused → score 58.

Streak accessors + the doc-comment that explains the lazy design: instead of a scheduled midnight job (which Android may kill or delay), evaluation happens opportunistically and is idempotent.

```

136 suspend fun evaluateStreaksUpTo(yesterday: LocalDate) {
137     val firstUnevaluated = streakDao.getLatestDate()?.let { LocalDate.parse(it).plusDays(1) }
138     ?: dailyLogDao.getEarliestDate()?.let { LocalDate.parse(it) }
139     ?: yesterday
140     if (firstUnevaluated.isAfter(yesterday)) return
141
142     val limits = getLimitsSnapshot().associateBy { it.platform }
143     fun limitFor(platform: Platform): Int =
144         limits[platform.name]?.dailyReelLimit ?: 30
145
146     var date = firstUnevaluated
147     var previous = streakDao.getByDateOnce(date.minusDays(1).toString())
148     while (!date.isAfter(yesterday)) {
149         val log = dailyLogDao.getByDateOnce(date.toString())
150         val underLimit = log == null ||
151             Platform.entries.all { log.getReelsForPlatform(it) <= limitFor(it) }
152         val streakDay = if (underLimit) {
153             (previous?.takeIf { it.underLimit }?.streakDay ?: 0) + 1
154         } else {
155             0
156         }
157         val record = StreakRecord(date.toString(), underLimit, streakDay)
158         streakDao.insert(record)
159         previous = record
160         date = date.plusDays(1)
161     }
162 }

```

evaluateStreaksUpTo(yesterday), the heart of streaks. Find the first day with no verdict yet: the day after the latest StreakRecord, else the earliest DailyLog (first run), else bail to yesterday. Then walk forward one day at a time: a day passes if it has *no log at all* (phone untouched = win) or every platform's count is within its limit (judged against *current* limits — limits aren't versioned, an accepted simplification). streakDay = previous day's count + 1 if the chain holds, else 0 on a fail, restarting at 1 on the next pass. Each verdict becomes the 'previous' for the next iteration, so one pass catches up an arbitrary number of missed days.

```

163
164 /**
165  * Calculates the current consecutive streak ending with the latest record.
166  */
167 fun calculateCurrentStreak(records: List<StreakRecord>): Int {
168     if (records.isEmpty()) return 0
169
170     val sorted = records.sortedByDescending { it.date }
171     var streak = 0
172
173     for (record in sorted) {
174         if (record.underLimit) {
175             streak++
176         } else {
177             break
178         }
179     }
180
181     return streak
182 }
183
184 /**
185  * Finds the longest consecutive streak across all recorded history.
186  */
187 fun calculateLongestStreak(records: List<StreakRecord>): Int {

```

```

188         if (records.isEmpty()) return 0
189
190         val sorted = records.sortedBy { it.date }
191         var longest = 0
192         var current = 0
193
194         for (record in sorted) {
195             if (record.underLimit) {
196                 current++
197                 if (current > longest) {
198                     longest = current
199                 }
200             } else {
201                 current = 0
202             }
203         }
204
205         return longest
206     }
207 }

```

Two pure helpers over the records list: `calculateCurrentStreak()` walks from the most recent record backwards until the first fail; `calculateLongestStreak()` scans the whole history tracking the best run. Both are trivial to unit-test because they take plain lists.

ScreenTimeHelper.kt — minutes from the system's usage log

app/src/main/java/com/example/brainrottracker/data/util/ScreenTimeHelper.kt · 101 lines · System API wrapper · 101 lines

Screen-time minutes are deliberately **not stored** in the database. Android already keeps a complete system-wide usage log (what Digital Wellbeing shows); this object queries it live and replays the raw events like a stopwatch. The key design decision: it uses *queryEvents* (raw RESUMED/PAUSED timestamps) instead of the convenient *queryAndAggregateUsageStats*, because the aggregate API attributes whole sessions to buckets that bleed across midnight — counting from raw events respects exact window boundaries.

```

1  package com.example.brainrottracker.data.util
2
3  import android.app.AppOpsManager
4  import android.app.usage.UsageEvents
5  import android.app.usage.UsageStatsManager
6  import android.content.Context
7  import android.os.Process
8  import com.example.brainrottracker.data.model.Platform
9  import java.util.Calendar
10
11  object ScreenTimeHelper {
12
13      /** True if the user has granted Usage Access via Settings → Apps → Special app access. */
14      fun hasPermission(context: Context): Boolean {
15          val appOps = context.getSystemService(Context.APP_OPS_SERVICE) as AppOpsManager
16          val mode = appOps.checkOpNoThrow(
17              AppOpsManager.OPSTR_GET_USAGE_STATS,
18              Process.myUid(),
19              context.packageName
20          )
21          return mode == AppOpsManager.MODE_ALLOWED
22      }

```

hasPermission(): `PACKAGE_USAGE_STATS` isn't a normal permission — it's an AppOps grant the user flips in Settings → Special app access. `checkOpNoThrow` asks the AppOps service whether this uid/package holds the `GET_USAGE_STATS` op.

```

23
24      /** Total foreground minutes per platform since midnight today. */
25      fun getTodayMinutesByPlatform(context: Context): Map<Platform, Int> {
26          val midnight = Calendar.getInstance().apply {
27              set(Calendar.HOUR_OF_DAY, 0)
28              set(Calendar.MINUTE, 0)
29              set(Calendar.SECOND, 0)
30              set(Calendar.MILLISECOND, 0)
31          }.timeInMillis
32          return queryMinutesFromEvents(context, midnight, System.currentTimeMillis())
33      }
34
35      /** Total foreground minutes per platform for yesterday (midnight-to-midnight). */

```

```

36 fun getYesterdayMinutesByPlatform(context: Context): Map<Platform, Int> {
37     val todayMidnight = Calendar.getInstance().apply {
38         set(Calendar.HOUR_OF_DAY, 0)
39         set(Calendar.MINUTE, 0)
40         set(Calendar.SECOND, 0)
41         set(Calendar.MILLISECOND, 0)
42     }.timeInMillis
43     val yesterdayMidnight = todayMidnight - 24L * 60 * 60 * 1000
44     return queryMinutesFromEvents(context, yesterdayMidnight, todayMidnight)
45 }
46
47 /** Total foreground minutes per platform over the last 7 days. */
48 fun getWeekMinutesByPlatform(context: Context): Map<Platform, Int> {
49     val startMs = System.currentTimeMillis() - 7L * 24 * 60 * 60 * 1000
50     return queryMinutesFromEvents(context, startMs, System.currentTimeMillis())
51 }

```

Three public windows built on the same engine: today (midnight→now), yesterday (midnight→midnight), and the rolling last-7-days. The Calendar dance zeroes hour/minute/second/millisecond to get local midnight.

```

52
53 /**
54  * Sums foreground time precisely by processing raw activity events.
55  * Unlike queryAndAggregateUsageStats, this respects exact window boundaries
56  * and won't bleed sessions from outside the window.
57  */
58 private fun queryMinutesFromEvents(
59     context: Context,
60     startMs: Long,
61     endMs: Long
62 ): Map<Platform, Int> {
63     if (!hasPermission(context)) return Platform.entries.associateWith { 0 }
64     val platformPackages = Platform.entries.map { it.packageName }.toSet()
65     return try {
66         val usm = context.getSystemService(Context.USAGE_STATS_SERVICE) as UsageStatsManager
67         val events = usm.queryEvents(startMs, endMs)
68         val event = UsageEvents.Event()
69         val foregroundStart = mutableMapOf<String, Long>()
70         val totalMs = mutableMapOf<String, Long>()
71
72         while (events.hasNextEvent()) {
73             events.getNextEvent(event)
74             val pkg = event.packageName
75             if (pkg !in platformPackages) continue
76             when (event.eventType) {
77                 UsageEvents.Event.ACTIVITY_RESUMED -> {
78                     if (pkg !in foregroundStart) foregroundStart[pkg] = event.timeStamp
79                 }
80                 UsageEvents.Event.ACTIVITY_PAUSED,
81                 UsageEvents.Event.ACTIVITY_STOPPED -> {
82                     foregroundStart.remove(pkg)?.let { start ->
83                         totalMs[pkg] = (totalMs[pkg] ?: 0L) + (event.timeStamp - start)
84                     }
85                 }
86             }
87         }
88     }
89 }

```

The stopwatch replay. `queryEvents` streams every activity transition in the window into a reusable `Event` object (an Android-style allocation optimisation). For each tracked package: on **ACTIVITY_RESUMED** remember the timestamp (only if not already in the foreground — duplicate **RESUMED** events happen); on **PAUSED/STOPPED** pop the start time and add the elapsed span to that package's total. Untracked packages are skipped at line 75.

```

88
89 // Any app still in foreground at the end of the window
90 for ((pkg, start) in foregroundStart) {
91     totalMs[pkg] = (totalMs[pkg] ?: 0L) + (endMs - start)
92 }
93
94 Platform.entries.associateWith { platform ->
95     ((totalMs[platform.packageName] ?: 0L) / 60_000L).toInt()
96 }
97 } catch (_: Exception) {
98     Platform.entries.associateWith { 0 }
99 }
100 }
101 }

```

Loose ends: an app still open at the window's end has an unfinished span — close it at `endMs` (this is what makes 'today so far' correct while you're inside Instagram). Finally, convert ms → whole minutes per platform; any exception (e.g. permission revoked mid-query) degrades to all-zeros instead of crashing.

UsageSyncManager.kt — optional Firestore backup

app/src/main/java/com/example/brainrottracker/data/sync/UsageSyncManager.kt · 74 lines · Cloud sync · 74 lines

Uploads one small document per *finished* day to the user's private Firestore space. Because the counts originate from the Accessibility API, Google Play treats any upload as sensitive — hence the triple gate: Firebase must be configured, the user must have signed in, and the explicit `cloud_sync_enabled` consent must be on.

```
1 package com.example.brainrottracker.data.sync
2
3 import android.content.Context
4 import com.example.brainrottracker.data.model.Platform
5 import com.example.brainrottracker.data.repository.UsageRepository
6 import com.example.brainrottracker.data.util.ScreenTimeHelper
7 import com.google.firebase.FirebaseApp
8 import com.google.firebase.auth.FirebaseAuth
9 import com.google.firebase.firestore.FirebaseFirestore
10 import java.time.LocalDate
11
12 /**
13  * Uploads one document per finished day to `users/{uid}/dailyLogs/{date}` for the
14  * signed-in user. Runs only when the user has signed in AND explicitly enabled cloud
15  * backup ("cloud_sync_enabled" pref) – required because the underlying counts come
16  * from the Accessibility API.
17  *
18  * Writes are fire-and-forget: Firestore persists them locally and retries when the
19  * device is online, so no explicit retry logic is needed.
20  */
21 class UsageSyncManager(
22     private val context: Context,
23     private val repository: UsageRepository
24 ) {
```

Imports + the class doc. 'Fire-and-forget' (line 18) is a real Firestore feature: `set()` persists the write to a local queue and the SDK retries automatically when online, so this code needs no retry logic and works on a plane.

```
25
26 suspend fun syncIfEnabled() {
27     if (FirebaseApp.getApps(context).isEmpty()) return
28     val prefs = context.getSharedPreferences(PREFS, Context.MODE_PRIVATE)
29     if (!prefs.getBoolean(KEY_ENABLED, false)) return
30     val user = FirebaseAuth.getInstance().currentUser ?: return
31
32     val yesterday = LocalDate.now().minusDays(1)
33     val lastSynced = prefs.getString(KEY_LAST_SYNCED, null)?.let(LocalDate::parse)
34     var date = lastSynced?.plusDays(1) ?: yesterday
35     if (date.isAfter(yesterday)) return
```

The three gates, then the catch-up window: `sync` covers from the day after `last_synced_date` (or just yesterday on first sync) up to yesterday. Today is never uploaded — it's still changing.

```
36
37 val firestore = FirebaseFirestore.getInstance()
38 val limits = repository.getLimitsSnapshot().associateBy { it.platform }
39 // UsageStats only retains recent activity, so minutes are only reliable for yesterday
40 val yesterdayMinutes = ScreenTimeHelper.getYesterdayMinutesByPlatform(context)
41
```

Limits are snapshotted once for the loop. Minutes come from UsageStats, which only retains recent event history — so minutes are attached only to yesterday's document; older catch-up days upload counts without minutes.

```
42 while (!date.isAfter(yesterday)) {
43     val dateStr = date.toString()
44     val log = repository.getLogSnapshot(dateStr)
45     val streak = repository.getStreakSnapshot(dateStr)
46     val doc = buildMap<String, Any> {
47         put("date", dateStr)
48         Platform.entries.forEach { p ->
49             put("reels_${p.name.lowercase()}", log?.getReelsForPlatform(p) ?: 0)
50             put("limit_${p.name.lowercase()}", limits[p.name]?.dailyReelLimit ?: 30)
51             if (date == yesterday) {
52                 put("minutes_${p.name.lowercase()}", yesterdayMinutes[p] ?: 0)
53             }
54         }
55         put("brainHealthScore", log?.brainHealthScore ?: 100)
56         streak?.let {
```



```
57         put("underLimit", it.underLimit)
58         put("streakDay", it.streakDay)
59     }
60 }
61 firestore.collection("users").document(user.uid)
62     .collection("dailyLogs").document(dateStr)
63     .set(doc)
64 prefs.edit().putString(KEY_LAST_SYNCED, dateStr).apply()
65 date = date.plusDays(1)
66 }
```

The upload loop. **buildMap** assembles a flat document: per-platform reel counts and limits (lower-cased keys like `reels_instagram`), conditional minutes, the health score, and the streak verdict if one exists. The path **users/{uid}/dailyLogs/{date}** plus the Firestore security rule (`request.auth.uid == uid`) is what makes the data private per account. `last_synced_date` advances only after each `set()` is queued, so an interrupted loop resumes where it left off; document writes are idempotent (same path → overwrite).

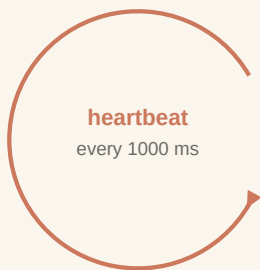
```
67     }
68
69     companion object {
70         const val PREFS = "brainrot_prefs"
71         const val KEY_ENABLED = "cloud_sync_enabled"
72         const val KEY_LAST_SYNCED = "last_synced_date"
73     }
74 }
```

The pref-key constants are shared publicly — the Limits screen uses `KEY_ENABLED` for its toggle and clears `KEY_LAST_SYNCED` on sign-out so a future sign-in re-uploads history.

PART 6

The Services — Counting, Overlay & Blocking

The heart of the product. ReelCounterService listens to accessibility events from the four social apps and turns swipes into counts; FloatingCounterService draws everything that floats over other apps. Between them they implement blocking, with three strictness modes.



- rootInActiveWindow → which app is in front?
- left to an untracked app → end session, hide pill, STOP loop
- switched tracked apps → onTrackedAppLeft(old) clears REMIND dismissal
- reel feed open (hasReelPager / TikTok) → show pill, else hide
- evaluateBlocking() → expired snooze re-blocks within ~1 s

Why poll at all? Accessibility events only arrive from the 4 whitelisted apps — Android sends nothing when you switch to Chrome, so the service must look for itself.

The 1-second heartbeat — the workaround for the accessibility API's blind spot.

BlockingMode.kt — HARD / SNOOZE / REMIND

.../service/BlockingMode.kt · 22 lines · Enum · 22 lines

```
1 package com.example.brainrottracker.service
2
3 /**
4  * How the blocking overlay behaves once a platform's daily limit is reached.
5  *
6  * HARD — the scrim re-appears every time the app comes to the foreground until
7  *         midnight; the only way out is the "Close app" button.
8  * SNOOZE — dismissing grants a 5-minute grace period, after which the scrim returns.
9  * REMIND — the scrim shows once per foreground session; dismissing lets the user
10 *            keep scrolling until they leave and reopen the app.
11 */
12 enum class BlockingMode {
13     HARD, SNOOZE, REMIND;
14
15     companion object {
16         const val PREF_KEY = "blocking_mode"
17         const val SNOOZE_MS = 5 * 60_000L
18
19         fun fromPref(value: String?): BlockingMode =
20             entries.find { it.name == value } ?: REMIND
21     }
22 }
```

The three strictness levels for the blocking overlay, documented right in the source. The companion holds the SharedPreferences key, the snooze length (5 min in ms), and **fromPref()** which parses the stored string and falls back to REMIND — the gentlest mode — if the pref is missing or garbage. The enforcement logic itself lives in FloatingCounterService.showBlockingOverlay(); this enum is just the shared vocabulary between the Limits screen (writes the pref) and the services (read it).

ReelCounterService.kt — the counting engine

.../service/ReelCounterService.kt · 754 lines · AccessibilityService · 754 lines · the core of the app

This is the technically densest file in the project. An **AccessibilityService** is a special service type built for assistive tech (screen readers): once the user enables it in system settings, Android streams it UI events from other apps along with a machine-readable tree of what's on screen (AccessibilityNodeInfo nodes). This service uses that to detect 'the user swiped to the next reel' inside Instagram, YouTube, TikTok and Snapchat. It also owns day-rollover detection, blocking decisions, and a 1-second heartbeat that compensates for the API's biggest blind spot: no events arrive from non-whitelisted apps.

```

1 package com.example.brainrottracker.service
2
3 import android.accessibilityservice.AccessibilityService
4 import android.content.Context
5 import android.content.Intent
6 import android.os.Handler
7 import android.os.Looper
8 import android.util.Log
9 import android.view.accessibility.AccessibilityEvent
10 import android.view.accessibility.AccessibilityNodeInfo
11 import com.example.brainrottracker.data.local.db.AppDatabase
12 import com.example.brainrottracker.data.local.db.entity.DailyLog
13 import com.example.brainrottracker.data.local.db.entity.UserLimits
14 import com.example.brainrottracker.data.model.Platform
15 import com.example.brainrottracker.data.repository.UsageRepository
16 import com.example.brainrottracker.data.sync.UsageSyncManager
17 import com.example.brainrottracker.data.util.ScreenTimeHelper
18 import com.example.brainrottracker.notification.NotificationHelper
19 import kotlinx.coroutines.CoroutineScope
20 import kotlinx.coroutines.Dispatchers
21 import kotlinx.coroutines.SupervisorJob
22 import kotlinx.coroutines.cancel
23 import kotlinx.coroutines.launch
24 import kotlinx.coroutines.withContext
25 import java.time.LocalDate

```

Imports. CoroutineScope/Dispatchers/SupervisorJob: the service does DB work on a background pool. Handler/Looper drive the heartbeat on the main thread (UI toolkits and the accessibility API are main-thread).

```

26
27 /**
28  * Reel/Shorts counter using VIEW HIERARCHY INSPECTION.
29  *
30  * Instead of guessing from event types (which are unreliable and noisy),
31  * we inspect the actual view tree to:
32  * 1. Find a full-screen vertical pager (the Reels/Shorts viewer)
33  * 2. Read the currently visible item INDEX from the pager
34  * 3. Only count when the index CHANGES (= user swiped to next reel)
35  *
36  * This approach:
37  *   ■ Never counts without an actual swipe (no false positives from content updates)
38  *   ■ Never counts on feed pages (feed RecyclerViews have many visible items, not 1)
39  *   ■ Handles YouTube Shorts, Instagram Reels, TikTok, Snapchat Spotlight
40  *   ■ Unified counter across all platforms
41  *
42  * The view hierarchy is only inspected when a CONTENT_CHANGE_TYPE_SUBTREE event
43  * fires AND at least 1 second has passed since the last inspection (rate limiting).
44  */

```

The strategy doc-comment: don't guess from event types — inspect the actual view tree, find the full-screen vertical pager, and count only when its current-item *index changes*. The checklist explains why this beats naive approaches: feeds show many items at once (never match), and content updates without a swipe don't move the index.

```

45 class ReelCounterService : AccessibilityService() {
46
47     private val serviceScope = CoroutineScope(Dispatchers.IO + SupervisorJob())
48     private lateinit var repository: UsageRepository
49     private lateinit var notificationHelper: NotificationHelper
50
51     private val mainHandler = Handler(Looper.getMainLooper())
52
53     // Heartbeat that runs while a tracked app is in the foreground. It keeps the HUD visible
54     // while a reel feed is open, re-evaluates blocking (so an expired snooze re-blocks within
55     // ~1 s), and detects when the user leaves to an untracked app — the accessibility service
56     // gets no events from other apps, so we poll the active window instead. Leaving a tracked
57     // app ends its "foreground session" (used by REMIND-mode blocking dismissals).
58     private val HEARTBEAT_MS = 1000L
59     private var heartbeatActive = false
60     private var lastForegroundTracked: String? = null

```

State, part 1. **serviceScope** = IO dispatcher + SupervisorJob (one failed job won't kill the others). The long comment explains the heartbeat's three jobs: keep the pill visible while a reel feed is open, re-evaluate blocking every second (so an expired snooze re-blocks promptly), and detect departure to untracked apps by polling, because Android sends this service nothing about apps outside its whitelist.

```

61     private val heartbeatRunnable = object : Runnable {
62         override fun run() {
63             val fg = rootInActiveWindow?.packageName?.toString()
64             val fgPlatform = fg?.let { Platform.fromPackageName(it) }
65
66             if (fg != null && fgPlatform == null) {
67                 // Left all tracked apps: end the foreground session and stop polling.
68                 lastForegroundTracked?.let { FloatingCounterService.instance?.onTrackedAppLeft(it) }
69                 lastForegroundTracked = null
70                 heartbeatActive = false
71                 FloatingCounterService.instance?.hide()
72                 return
73             }
74
75             if (fg != null && fgPlatform != null) {
76                 if (fg != lastForegroundTracked) {
77                     lastForegroundTracked?.let { FloatingCounterService.instance?.onTrackedAppLeft(it) }
78                     lastForegroundTracked = fg
79                 }
80                 if (hasReelPager[fg] == true || fgPlatform == Platform.TIKTOK) {
81                     FloatingCounterService.instance?.show()
82                 } else {
83                     FloatingCounterService.instance?.hide()
84                 }
85                 evaluateBlocking(fgPlatform, fg)
86             }
87             // fg == null means the window couldn't be read — keep polling.
88             mainHandler.postDelayed(this, HEARTBEAT_MS)
89         }
90     }

```

The heartbeat Runnable. Every second it reads **rootInActiveWindow** — the package that owns the focused window. Three cases: (a) a non-tracked app is in front → notify FloatingCounterService that the previous tracked app's 'foreground session' ended (clears REMIND dismissals), hide the pill, and *stop the loop* by not re-posting; (b) a tracked app is in front → handle app-to-app switches, show the pill only if a reel feed is confirmed open (hasReelPager, or TikTok where everything counts), and re-run evaluateBlocking(); (c) null (window unreadable, e.g. during transitions) → just keep polling. Note how it re-posts itself with postDelayed — a classic self-scheduling loop.

```

91
92     // Per-package: the last known item index in the reel pager
93     private val lastPagerIndex = mutableMapOf<String, Int>()
94
95     // Per-package: whether we've confirmed a reel pager exists on the current screen
96     private val hasReelPager = mutableMapOf<String, Boolean>()
97
98     // Per-package: the last known signature for signature-based tracking (YouTube)
99     private val lastSeenSignature = mutableMapOf<String, String>()
100
101     // List of recently counted signatures to prevent double counting on YouTube Shorts
102     private val recentYouTubeSignatures = mutableListOf<String>()
103     private val MAX_SIGNATURES_HISTORY = 15
104
105     // Rate limit: don't inspect the view tree more than once per second
106     private val lastInspectionMs = mutableMapOf<String, Long>()
107     private val INSPECTION_COOLDOWN_MS = 800L
108
109     // Debounce: minimum time between two counted reels on the same platform
110     private val DEBOUNCE_MS = 1200L
111     private val lastCountedMs = mutableMapOf<String, Long>()
112
113     // In-memory reel counts for today
114     private val reelCounts = mutableMapOf<String, Int>()
115
116     // The date the in-memory state belongs to; reset at midnight rollover
117     private var currentDay: String = LocalDate.now().toString()
118
119     // Cache of daily reel limits for each platform name
120     private val limitsCache = mutableMapOf<String, Int>()
121

```

State, part 2 — all keyed by package name: last pager index (Instagram/Snapchat), whether a reel pager is currently confirmed on screen, the last YouTube signature plus a 15-entry history (so re-watching a Short you just saw doesn't double-count), the 0.8 s tree-inspection cooldown, the 1.2 s per-platform count debounce, today's in-memory counts (mirror of the DB for fast limit checks), the date those counts belong to, and a live cache of reel limits.

```
122     companion object {
123         private const val TAG = "ReelCounter"
124         private const val MAX_TREE_DEPTH = 12
125
126         var isRunning = false
127         private set
128         var instance: ReelCounterService? = null
129         private set
130     }
```

Companion: `MAX_TREE_DEPTH` caps the BFS at 12 levels (deep trees are expensive and reels pagers live near the top). **isRunning** and **instance** are how the rest of the app reaches the service — Android instantiates services itself, so a static instance handle is the practical (if inelegant) bridge; 'private set' means only the service can write them.

onServiceConnected() fires when the user enables the service. It publishes the static handles, builds repository + notification helper, then two background jobs: load today's existing counts from the DB into memory (so a service restart doesn't reset the visible count), and run the streak catch-up. A third job *collects* the limits Flow forever, keeping limitsCache fresh whenever the user moves a slider. Finally it starts FloatingCounterService so the overlay machinery is ready.

```
168
169     override fun onInterrupt() {}
170
171     override fun onDestroy() {
172         super.onDestroy()
173         isRunning = false
174         instance = null
175         heartbeatActive = false
176         mainHandler.removeCallbacks(heartbeatRunnable)
177         serviceScope.cancel()
178         stopService(Intent(this, FloatingCounterService::class.java))
179     }

```


handleViewScrolled(). TikTok short-circuits: the whole app is short videos, so every forward scroll counts — no tree inspection needed. For other platforms the event is ignored unless a reel pager was previously confirmed (`hasReelPager`): then, if the event's own indices say at most one item is visible (`toIndex - fromIndex ≤ 1`) in a list of more than 2 items — i.e. a full-screen pager, not a feed — a forward scroll counts.

Figure 1. The effect of the number of trials on the number of correct responses. The number of correct responses was plotted against the number of trials for each condition. The number of correct responses increased with the number of trials for all conditions. The number of correct responses was highest for the condition with the highest number of trials (10 trials) and lowest for the condition with the lowest number of trials (2 trials).

```

320         root.recycle()
321     }
322 }

```

handleContentChanged() — the gateway to tree inspection. TikTok skips it. The 0.8 s cooldown bounds CPU cost: walking a view tree allocates hundreds of node objects. Getting the root is defensive three ways: `rootInActiveWindow` can throw, can belong to a different package (overlay windows — discard), or be null, in which case it climbs *up* from `event.source` to find the root. Only once a valid root exists is the cooldown clock reset (line 311), then YouTube gets its bespoke detector and everything else the generic pager detector; **finally { root.recycle() }** returns the node to Android's pool (pre-API-33 requirement; harmless after).

```

323
324     private fun handleYouTubeShorts(root: AccessibilityNodeInfo, platform: Platform, pkg: String) {
325         val info = inspectYouTubeShorts(root)
326         if (info.isActive) {
327             hasReelPager[pkg] = true
328             keepBubbleVisible()
329
330             val signature = info.signature
331             if (signature != null) {
332                 val lastKnownSig = lastSeenSignature[pkg]
333
334                 if (lastKnownSig == null) {
335                     // First detection — record baseline
336                     lastSeenSignature[pkg] = signature
337                     recentYouTubeSignatures.add(signature)
338                     Log.d(TAG, "YT SHORTS BASELINE: signature=$signature")
339                 } else if (signature != lastKnownSig) {
340                     lastSeenSignature[pkg] = signature
341                     Log.d(TAG, "YT SHORTS CHANGED: $lastKnownSig → $signature")
342
343                     if (!recentYouTubeSignatures.contains(signature)) {
344                         // New video, count it!
345                         recentYouTubeSignatures.add(signature)
346                         if (recentYouTubeSignatures.size > MAX_SIGNATURES_HISTORY) {
347                             recentYouTubeSignatures.removeAt(0)
348                         }
349                         tryCountReel(platform, pkg, "YT_SHORTS_SIG_CHANGE")
350                     } else {
351                         Log.d(TAG, "YT SHORTS RE-WATCH: signature already in history, not counting")
352                     }
353                 }
354             }
355         } else {
356             if (hasReelPager[pkg] == true) {
357                 Log.d(TAG, "YT SHORTS LOST: navigated away")
358             }
359             hasReelPager[pkg] = false
360             hideBubble()
361         }
362     }

```

handleYouTubeShorts() — signature-based counting, needed because YouTube's pager exposes no usable item index. `inspectYouTubeShorts()` (below) answers 'is the Shorts player open?' and 'what's on screen?'. First sighting records a baseline without counting (you didn't swipe yet). When the signature *changes*: count only if it isn't in the recent-history list — swiping back up to a Short you just watched changes the signature but shouldn't count. History is capped at 15 (FIFO). If the player is gone, mark the pager lost and hide the pill.

```

363
364     private fun handleGenericReels(root: AccessibilityNodeInfo, platform: Platform, pkg: String) {
365         val pagerInfo = findReelPager(root)
366
367         if (pagerInfo != null) {
368             val (currentIndex, totalItems) = pagerInfo
369             hasReelPager[pkg] = true
370             keepBubbleVisible()
371
372             val lastIndex = lastPagerIndex[pkg]
373             if (lastIndex == null) {
374                 // First detection — record baseline, don't count
375                 lastPagerIndex[pkg] = currentIndex
376                 Log.d(TAG, "PAGER FOUND: pkg=$pkg index=$currentIndex total=$totalItems (baseline)")
377             } else if (currentIndex != lastIndex) {
378                 // Index changed! User swiped to a different reel.
379                 val isForward = currentIndex > lastIndex
380                 lastPagerIndex[pkg] = currentIndex
381                 Log.d(TAG, "PAGER CHANGED: pkg=$pkg $lastIndex→$currentIndex (forward=$isForward)")
382
383                 if (isForward) {

```



```

455         }
456
457         if (commentDesc == null) {
458             if (desc.contains("comment", ignoreCase = true)) {
459                 commentDesc = desc
460             } else if (text.contains("comment", ignoreCase = true)) {
461                 commentDesc = text
462             }
463         }
464
465         if (likeDesc == null) {
466             if (desc.contains("like this video", ignoreCase = true) || desc.contains("like this shor
467 ■ t", ignoreCase = true)) {
468                 likeDesc = desc
469             }
470         }
471
472         if (titleText == null && text.isNotEmpty() && !text.startsWith("@") &&
473 ■ !text.contains("comment", ignoreCase = true) && !text.contains("like", ignoreCase = true
474 ■ ) &&
475             text.length > 5 && text.length < 150
476         ) {
477             val lowerText = text.lowercase()
478             if (lowerText != "home" && lowerText != "shorts" && lowerText != "subscriptions" && lowe
479 ■ rText != "you" && lowerText != "library") {
480                 val rect = android.graphics.Rect()
481                 node.getBoundsInScreen(rect)
482                 if (rect.top > 1200) {
483                     titleText = text
484                 }
485             }
486         }
487
488         for (i in 0 until node.childCount) {
489             val child = node.getChild(i)
490             if (child != null) {
491                 toRecycle.add(child)
492                 queue.add(child)
493             }
494         }
495     } catch (e: Exception) {
496         Log.w(TAG, "Error inspecting YouTube Shorts: ${e.message}")
497     } finally {
498         toRecycle.forEach {
499             try { it.recycle() } catch (_: Exception) {}
500         }
501     }

```

The BFS body — a tour of what an accessibility tree contains. For each *visible* node (isVisibleToUser filters the hidden Home/Subscriptions tabs that stay in the tree!): (1) seven known YouTube view-IDs (reel_recycler, reel_watch_fragment_root, play/next/prev buttons...) directly prove the Shorts player is open — this is why flagReportViewIds is set in the service config; (2) note any RecyclerView; (3) grab the first @handle-looking text (3–40 chars); (4) grab comment/like button content-descriptions; (5) title heuristic — a 5–150-char text that isn't a handle, isn't a nav-tab label, and sits below y=1200 px (Shorts overlay text is in the lower half on basically all current phones; a hard-coded but pragmatic threshold). Children are enqueued and the loop continues until the tree is exhausted.

```

501
502     // Fallback: if no view ID matched, we identify Shorts by presence of a recycler,
503     // creator handle (@), and either a like button description ("like this...") or comment button.
504     // This avoids matches on the Home feed where like buttons and comment lists are not directly visible on
505 ■ cards.
506     if (!isActive && hasRecycler && hasHandle && (likeDesc != null || commentDesc != null)) {
507         isActive = true
508     }
509
510     val signature = if (handle != null || commentDesc != null || titleText != null) {
511         "$handle|$commentDesc|$likeDesc|$titleText"
512     } else {
513         null
514     }
515
516     return YouTubeShortsInfo(isActive, signature)
517 }

```



```

656         val limits = repository.getLimitsSnapshot()
657         val health = if (log != null) repository.calculateBrainHealth(log, limits) else 100
658         withContext(Dispatchers.Main) {
659             showBubbleAndResetTimer(log, limits, health)
660             evaluateBlocking(platform, pkg)
661         }
662         log?.let { notificationHelper.checkMilestone(it.getTotalReels()) }
663     }
664 }

```

tryCountReel() — every detection path converges here. The 1.2 s per-platform debounce is the final over-count guard (multiple event types often fire for one swipe). Then: bump the in-memory count, and in a background coroutine write the DB increment, recompute health from fresh snapshots, hop to the main thread (`withContext(Dispatchers.Main)`) to update the pill and re-check blocking, and finally let `NotificationHelper` fire a milestone if the new total is exactly 25/50/75/100/150/200.

```

665
666     /**
667     * Shows the blocking scrim if blocking is on and the platform's count has reached its
668     * limit. Called on every counted reel AND every time the app reaches the foreground, so
669     * the block re-triggers when the user reopens the app. Suppression (snooze, per-session
670     * dismissal) is mode-dependent and handled inside [FloatingCounterService].
671     */
672     private fun evaluateBlocking(platform: Platform, pkg: String) {
673         val prefs = getSharedPreferences(FloatingCounterService.PREFS, Context.MODE_PRIVATE)
674         if (!prefs.getBoolean("blocking_enabled", false)) return
675         // Fall back to the entity default so blocking works before any limit was saved
676         val limit = limitsCache[platform.name] ?: 30
677         val count = reelCounts[pkg] ?: 0
678         if (count >= limit) {
679             val mode = BlockingMode.fromPref(prefs.getString(BlockingMode.PREF_KEY, null))
680             FloatingCounterService.instance?.showBlockingOverlay(platform, limit, mode)
681         }
682     }

```

evaluateBlocking() — the single gatekeeper, called from three places (counted reel, app reopened, heartbeat tick). Reads `blocking_enabled` straight from `SharedPreferences` (cheap, and always current), falls back to limit 30 when the user never saved limits, and when `count ≥ limit` hands off to `FloatingCounterService.showBlockingOverlay()` with the current mode. Mode-specific suppression (snooze timers, per-session dismissals) deliberately lives in the overlay service, next to the state it depends on.

```

683
684     /**
685     * Detects midnight rollover. In-memory counts are reset, the finished day's streak is
686     * evaluated, a recap notification is shown, and milestone alerts are re-armed. Runs
687     * lazily on the next accessibility event, so a phone that was off at midnight catches
688     * up on the next doomscroll.
689     */
690     private fun checkDayRollover() {
691         val today = LocalDate.now().toString()
692         if (today == currentDay) return
693         val previousDay = currentDay
694         currentDay = today
695         reelCounts.clear()
696         lastCountedMs.clear()
697
698         serviceScope.launch {
699             repository.evaluateStreaksUpTo(LocalDate.now().minusDays(1))
700             // Recap only makes sense right after the day ended, not days later
701             if (LocalDate.parse(previousDay).plusDays(1).toString() == today) {
702                 val log = repository.getLogSnapshot(previousDay)
703                 val minutes = ScreenTimeHelper.getYesterdayMinutesByPlatform(applicationContext)
704                     .values.sum()
705                 notificationHelper.showDailySummary(
706                     totalReels = log?.getTotalReels() ?: 0,
707                     totalMinutes = minutes,
708                     brainHealth = log?.brainHealthScore ?: 100
709                 )
710             }
711             notificationHelper.resetDailyMilestones()
712             // Push the finished day to the cloud (no-op unless signed in with backup on)
713             try {
714                 UsageSyncManager(applicationContext, repository).syncIfEnabled()
715             } catch (_: Exception) {
716             }
717         }
718     }

```

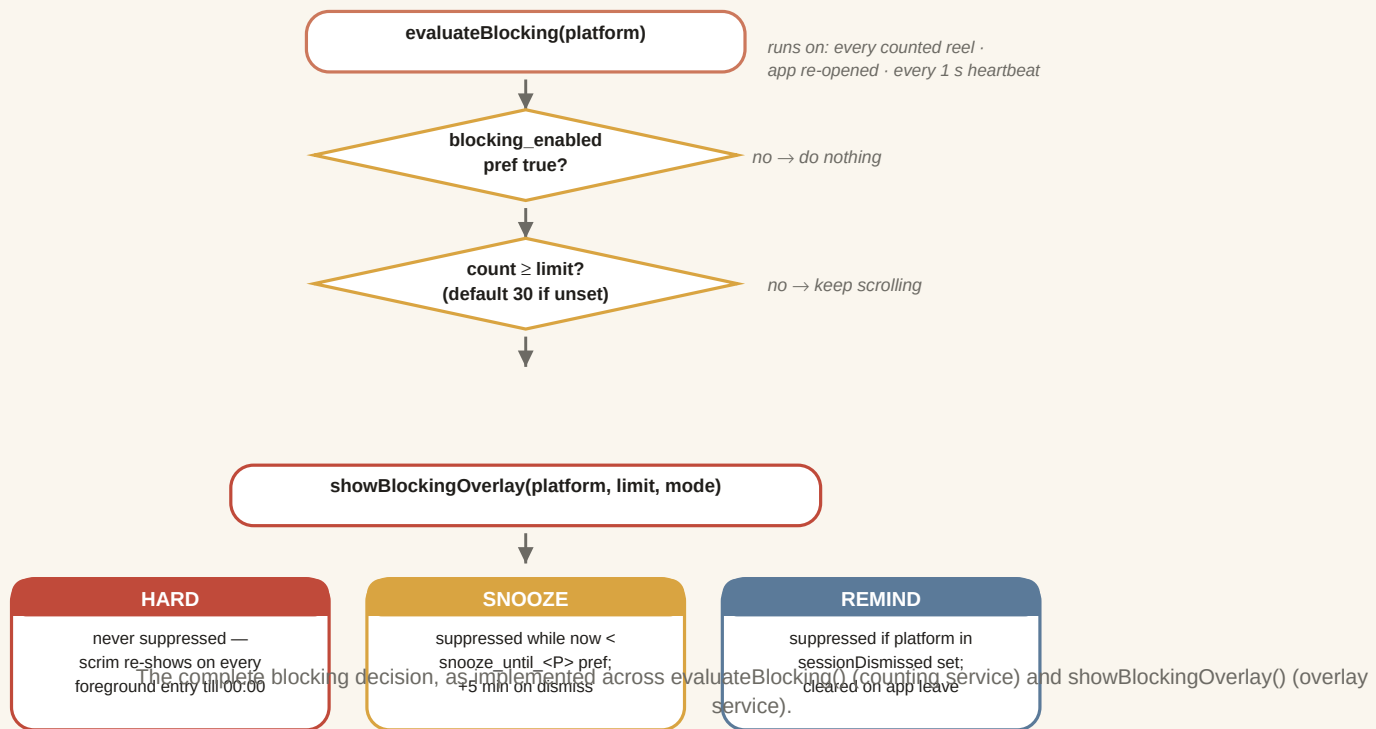
checkDayRollover() — the no-alarm midnight. Compare today's date string with `currentDay` on every event; on the first event after midnight: reset in-memory counts and debounce timers (this is also what lifts all blocks — counts are now 0), then in the background evaluate yesterday's streak, show the recap notification *only if the gap is exactly one day* (a recap for last Tuesday after a week off would be noise), re-arm milestone notifications, and push the finished day to the cloud.

```

719
720     private fun showBubbleAndResetTimer(log: DailyLog?, limits: List<UserLimits>, health: Int) {
721         val total = log?.getTotalReels() ?: 0
722         // Per-platform breakdown for the tap-to-open popup — only platforms used today.
723         val breakdown = Platform.entries.mapNotNull { p ->
724             val c = log?.getReelsForPlatform(p) ?: 0
725             if (c > 0) {
726                 val limit = limits.find { it.platform == p.name }?.dailyReelLimit
727                 ?: limitsCache[p.name] ?: 30
728                 FloatingCounterService.HudPlatform(p, c, limit)
729             } else null
730         }
731         FloatingCounterService.instance?.updateHud(total, health, breakdown)
732         keepBubbleVisible()
733     }
734
735     /** Show the pill and keep it visible (via the heartbeat) for as long as a reel feed is open. */
736     private fun keepBubbleVisible() {
737         FloatingCounterService.instance?.show()
738         startHeartbeat()
739     }
740
741     /** Start the foreground-polling heartbeat without forcing the pill to show. */
742     private fun startHeartbeat() {
743         if (!heartbeatActive) {
744             heartbeatActive = true
745             mainHandler.postDelayed(heartbeatRunnable, HEARTBEAT_MS)
746         }
747     }
748
749     private fun hideBubble() {
750         // The heartbeat keeps running: it manages pill visibility and foreground-session
751         // tracking itself, and stops when the user leaves all tracked apps.
752         FloatingCounterService.instance?.hide()
753     }
754 }

```

Bubble plumbing. `showBubbleAndResetTimer()` builds the popup's per-platform breakdown — only platforms with a non-zero count today, each with its limit — and pushes total/health/breakdown to the overlay. `keepBubbleVisible()` shows the pill and ensures the heartbeat runs; `startHeartbeat()` guards against double-scheduling with the `heartbeatActive` flag. `hideBubble()` hides the pill but leaves the heartbeat alive — the loop owns its own shutdown (when the user leaves all tracked apps).



FloatingCounterService.kt — pill HUD, popup & blocking scrim

.../service/FloatingCounterService.kt · 876 lines · Foreground service + overlay UI · 876 lines

Everything that floats over other apps lives here: the counter pill, its tap-to-open stats popup, and the full-screen blocking scrim. Because Compose can't render into raw overlay windows, this file builds UI the classic way — Views created in code, added to **WindowManager** with `TYPE_APPLICATION_OVERLAY` (the `SYSTEM_ALERT_WINDOW` permission). It runs as a *foreground service* so Android won't kill it while the persistent notification shows. It's also where the platform logos are drawn by hand on a Canvas — no trademarked image assets ship with the app.

```

1 package com.example.brainrottracker.service
2
3 import android.animation.ObjectAnimator
4 import android.animation.ValueAnimator
5 import android.app.Service
6 import android.content.Context
7 import android.content.Intent
8 import android.content.SharedPreferences
9 import android.content.res.Configuration
10 import android.graphics.Canvas
11 import android.graphics.Color
12 import android.graphics.LinearGradient
13 import android.graphics.Outline
14 import android.graphics.Paint
15 import android.graphics.Path
16 import android.graphics.PixelFormat
17 import android.graphics.RectF
18 import android.graphics.Shader
19 import android.graphics.Typeface
20 import android.graphics.drawable.GradientDrawable
21 import android.os.Build
22 import android.os.Handler
23 import android.os.IBinder
24 import android.os.Looper
25 import android.util.Log
26 import android.util.TypedValue
27 import android.view.Gravity
28 import android.view.MotionEvent
29 import android.view.View
30 import android.view.ViewGroup
31 import android.view.ViewOutlineProvider
32 import android.view.WindowManager
33 import android.view.animation.OvershootInterpolator
34 import android.widget.FrameLayout
35 import android.widget.ImageView
36 import android.widget.LinearLayout
37 import android.widget.TextView

```

```

38 import com.example.brainrottracker.R
39 import com.example.brainrottracker.data.model.Platform
40 import com.example.brainrottracker.ui.components.brainMoodDrawable
41 import com.example.brainrottracker.notification.NotificationHelper
42
43 /**
44  * Floating HUD overlay: a rounded pill showing a brain mascot that degrades with brain health
45  * plus today's total reel count. Tapping it opens an info popup with the per-platform breakdown.
46  * Dragging moves the pill.
47  *
48  * Appearance follows the app's Warm palette and the user's light/dark theme; size follows the
49  * `hud_scale` preference. Both are re-read from prefs every time the pill is shown.
50  */

```

Imports (all classic `android.view/android.graphics` — note: zero Compose) and the class doc. `brainMoodDrawable` is shared with the in-app mascot and the widget so all three always show the same face for the same health score.

```

51 class FloatingCounterService : Service() {
52
53     private var windowManager: WindowManager? = null
54     private var rootView: FrameLayout? = null
55     private var pill: LinearLayout? = null
56     private var brainView: ImageView? = null
57     private var countTextView: TextView? = null
58
59     private var layoutParams: WindowManager.LayoutParams? = null
60     private var isViewAdded = false
61     private var visible = false
62     private val mainHandler = Handler(Looper.getMainLooper())
63     private var criticalAnimator: ObjectAnimator? = null
64
65     private lateinit var prefs: SharedPreferences
66
67     // Latest stats, kept so the popup can render the current snapshot.
68     private var currentTotal = 0
69     private var currentHealth = 100
70     private var currentBreakdown: List<HudPlatform> = emptyList()
71
72     // Resolved appearance (recomputed from prefs on each show()).
73     private var isDark = true
74     private var scale = 1.2f
75
76     // Popup overlay state.
77     private var popupView: View? = null
78     private var isPopupAdded = false
79
80     // Blocking overlay state. REMIND-mode dismissals last for one foreground session of the
81     // blocked app (cleared by onTrackedAppLeft); SNOOZE expiry lives in prefs so it survives
82     // service restarts; HARD is never suppressed.
83     private var blockingView: View? = null
84     private var isBlockingAdded = false
85     private val sessionDismissed = mutableSetOf<String>()
86
87     /** One row of the tap-to-open breakdown. */
88     data class HudPlatform(val platform: Platform, val count: Int, val limit: Int)
89
90     companion object {
91         private const val TAG = "FloatingCounter"
92         const val PREFS = "brainrot_prefs"
93         const val KEY_SCALE = "hud_scale"
94         const val KEY_THEME = "theme_mode"
95         const val DEFAULT_SCALE = 1.2f
96
97         var instance: FloatingCounterService? = null
98         private set
99     }

```

The state fields tell the architecture story: `WindowManager` + the view handles, added/visible flags (a view can be attached but GONE), a main-thread `Handler` (window operations must happen there), the pulsing animator for critical health, the latest stats snapshot (so the popup can render without asking anyone), the resolved theme/scale, popup state, and blocking state — note **`sessionDismissed`**, the in-memory set behind REMIND mode, vs. SNOOZE deadlines which live in prefs precisely so they survive service restarts (no cheating by toggling the service). The companion exposes the prefs name/keys and the static instance handle.

```

100
101 override fun onCreate() {
102     super.onCreate()
103     instance = this
104     prefs = getSharedPreferences(PREFS, Context.MODE_PRIVATE)
105     windowManager = getSystemService(WINDOW_SERVICE) as WindowManager
106     resolveAppearance()

```



```

107     createFloatingView()
108     Log.d(TAG, "FloatingCounterService created")
109 }
110
111 override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
112     val notificationHelper = NotificationHelper(applicationContext)
113     notificationHelper.createChannels()
114     startForeground(NotificationHelper.SERVICE_NOTIFICATION_ID, notificationHelper.getServiceNotification())
115
116     if (intent != null && intent.action == "com.example.brainrottracker.UPDATE_HUD") {
117         val count = intent.getIntExtra("count", 0)
118         val limit = intent.getIntExtra("limit", 30)
119         val name = intent.getStringExtra("platform") ?: "Instagram"
120         val ratio = if (limit > 0) count.toFloat() / limit else 0f
121         val health = ((1f - ratio).coerceIn(0f, 1f) * 100).toInt()
122         val p = Platform.entries.find { it.displayName == name }
123         updateHud(count, health, if (p != null) listOf(HudPlatform(p, count, limit)) else emptyList())
124         show()
125     }
126     return START_STICKY
127 }
128
129 override fun onBind(intent: Intent?): IBinder? = null
130
131 private fun dp(value: Float): Int =
132     TypedValue.applyDimension(TypedValue.COMPLEX_UNIT_DIP, value, resources.displayMetrics).toInt()

```

Lifecycle. onCreate caches prefs + WindowManager, resolves appearance, and builds the pill view once (it's reused forever — show/hide just toggle visibility). onStartCommand promotes the service to **foreground** with the persistent notification (mandatory for overlay longevity), and supports a manual UPDATE_HUD intent used as a test hook. **START_STICKY** asks Android to restart the service if it's killed under memory pressure. onBind returns null — nobody binds. dp() converts design units to physical pixels per the device's density.

Appearance resolution: `theme_mode` pref (LIGHT/DARK/SYSTEM, written by the in-app ThemeController) falls back to the OS night mode; `hud_scale` is clamped to 0.8–1.8×. The color properties mirror `theme/Color.kt`'s warm palette as ARGB ints — including the terracotta accent that's identical in both themes. Re-read on every `show()`, so a theme change in the app applies the next time the pill appears.

```

155
156     private fun createFloatingView() {
157         val context = this
158         rootView = FrameLayout(context)
159
160         brainView = ImageView(context).apply {
161             setImageResource(brainMoodDrawable(currentHealth))
162         }
163         countTextView = TextView(context).apply {
164             text = "0"
165             setTypeface(Typeface.DEFAULT_BOLD)
166             gravity = Gravity.CENTER_VERTICAL
167         }
168
169         pill = LinearLayout(context).apply {
170             orientation = LinearLayout.HORIZONTAL
171             gravity = Gravity.CENTER_VERTICAL
172             // No elevation: the shadow is what produced the translucent rectangular "film".
173             // A slight rounded-rect outline keeps content corners crisp.

```

```

174         val corner = dp(14f).toFloat()
175         outlineProvider = object : ViewOutlineProvider() {
176             override fun getOutline(view: View, outline: Outline) {
177                 outline.setRoundRect(0, 0, view.width, view.height, corner)
178             }
179         }
180         clipToOutline = true
181         addView(brainView)
182         addView(countTextView)
183         layoutParams = FrameLayout.LayoutParams(
184             FrameLayout.LayoutParams.WRAP_CONTENT,
185             FrameLayout.LayoutParams.WRAP_CONTENT
186         )
187     }
188     rootView?.addView(pill)
189     applyPillAppearance()
190
191     val type = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
192         WindowManager.LayoutParams.TYPE_APPLICATION_OVERLAY
193     } else {
194         @Suppress("DEPRECATION")
195         WindowManager.LayoutParams.TYPE_PHONE
196     }
197
198     layoutParams = WindowManager.LayoutParams(
199         WindowManager.LayoutParams.WRAP_CONTENT,
200         WindowManager.LayoutParams.WRAP_CONTENT,
201         tvp.e,
202         WindowManager.LayoutParams.FLAG_NOT_FOCUSABLE or
203             WindowManager.LayoutParams.FLAG_NOT_TOUCH_MODAL,
204         PixelFormat.TRANSLUCENT
205     ).apply {
206         gravity = Gravity.TOP or Gravity.CENTER_HORIZONTAL
207         y = dp(80f)
208     }
209
210     attachTouchListener()
211     Log.d(TAG, "Floating view created")
212 }

```

Building the pill: an `ImageView` (brain face) + bold `TextView` (count) inside a horizontal `LinearLayout`. The `outlineProvider` + `clipToOutline` pair rounds the corners without elevation — the comment records a real bug: elevation's shadow rendered as an ugly translucent rectangle over other apps' content. The window params are the heart of overlay magic: **`TYPE_APPLICATION_OVERLAY`** (the post-Android-8 overlay type; `TYPE_PHONE` for older devices), **`FLAG_NOT_FOCUSABLE`** so keyboard/back keep going to the app underneath, and `WRAP_CONTENT` sizing anchored top-center, 80 dp down.

```

213
214     /** Push the resolved theme + scale onto the pill's views. */
215     private fun applyPillAppearance() {
216         brainView?.layoutParams = LinearLayout.LayoutParams(dp(34f * scale), dp(34f * scale))
217         countTextView?.apply {
218             textSize = 22f * scale
219             setTextColor(accentColor)
220             layoutParams = LinearLayout.LayoutParams(
221                 LinearLayout.LayoutParams.WRAP_CONTENT,
222                 LinearLayout.LayoutParams.WRAP_CONTENT
223             ).apply { leftMargin = dp(10f * scale) }
224         }
225         pill?.apply {
226             setPadding(dp(16f * scale), dp(10f * scale), dp(20f * scale), dp(10f * scale))
227             background = GradientDrawable().apply {
228                 cornerRadius = dp(14f).toFloat()
229                 setColor(surfaceColor)
230                 setStroke(dp(1f), borderColor)
231             }
232             requestLayout()
233         }
234     }

```

`applyPillAppearance()` pushes theme + scale into the views: brain size, text size, padding all multiply by the scale factor; the background is a code-built `GradientDrawable` (rounded rect, surface fill, hairline border).

```

235
236     /** Distinguishes a tap (→ open popup) from a drag (→ move the pill). */
237     private fun attachTouchListener() {
238         rootView?.setOnTouchListener(object : View.OnTouchListener {
239             private var initialX = 0
240             private var initialY = 0
241             private var touchX = 0f
242             private var touchY = 0f

```

```

243         private var downTime = 0L
244         private val slop = dp(8f)
245
246         override fun onTouch(v: View, event: MotionEvent): Boolean {
247             when (event.action) {
248                 MotionEvent.ACTION_DOWN -> {
249                     initialX = layoutParams?.x ?: 0
250                     initialY = layoutParams?.y ?: 0
251                     touchX = event.rawX
252                     touchY = event.rawY
253                     downTime = System.currentTimeMillis()
254                     pill?.animate()?.scaleX(0.94f)?.scaleY(0.94f)?.setDuration(80)?.start()
255                     return true
256                 }
257                 MotionEvent.ACTION_MOVE -> {
258                     layoutParams?.x = initialX + (event.rawX - touchX).toInt()
259                     layoutParams?.y = initialY + (event.rawY - touchY).toInt()
260                     if (isViewAdded) windowManager?.updateViewLayout(rootView, layoutParams)
261                     return true
262                 }
263                 MotionEvent.ACTION_UP, MotionEvent.ACTION_CANCEL -> {
264                     pill?.animate()?.scaleX(1f)?.scaleY(1f)?.setDuration(80)?.start()
265                     val moved = kotlin.math.hypot(event.rawX - touchX, event.rawY - touchY)
266                     val elapsed = System.currentTimeMillis() - downTime
267                     if (event.action == MotionEvent.ACTION_UP && moved < slop && elapsed < 250) togglePopup()
268                     return true
269                 }
270             }
271             return false
272         }
273     })
274 }

```

The touch handler — tap-vs-drag disambiguation on the raw event stream: DOWN records the window position and finger origin and squishes the pill to 94% as feedback; MOVE offsets the window params by the finger delta and calls `updateViewLayout` (this is how dragging an overlay works — you move the *window*, not the view); UP restores scale and, if the finger travelled under 8 dp in under 250 ms, treats it as a tap → `togglePopup()`.

```

275
276 /**
277  * Show the floating pill – safe to call from any thread. Re-applies theme/size and plays a
278  * pop-in animation only on the hidden→visible transition, so the 1-second heartbeat that calls
279  * this repeatedly doesn't re-animate or flicker the pill.
280  */
281 fun show() {
282     mainHandler.post {
283         try {
284             if (visible && isViewAdded) return@post
285             resolveAppearance()
286             applyPillAppearance()
287             if (!isViewAdded && rootView != null && layoutParams != null) {
288                 windowManager?.addView(rootView, layoutParams)
289                 isViewAdded = true
290             }
291             rootView?.visibility = View.VISIBLE
292             visible = true
293             playPopIn()
294         } catch (e: Exception) {
295             Log.e(TAG, "show() failed: ${e.message}", e)
296         }
297     }
298 }
299
300 private fun playPopIn() {
301     pill?.apply {
302         scaleX = 0.7f
303         scaleY = 0.7f
304         alpha = 0f
305         animate()
306             .scaleX(1f).scaleY(1f).alpha(1f)
307             .setDuration(240)
308             .setInterpolator(OvershootInterpolator(1.6f))
309             .start()
310     }
311 }
312
313 /** Hide the floating pill (and any popup) – safe to call from any thread. */
314 fun hide() {
315     mainHandler.post {
316         visible = false
317         rootView?.visibility = View.GONE
318         removePopup()

```

```

319     }
320 }
321
322 /** Legacy entry point retained for the manual UPDATE_HUD test path. */
323 fun updateCount(totalCount: Int) {
324     val health = ((1f - totalCount / 120f).coerceIn(0f, 1f) * 100).toInt()
325     updateHud(totalCount, health, emptyList())
326 }

```

`show()/hide()` are thread-safe via `mainHandler.post`. `show()` short-circuits if already visible — important because the heartbeat calls it every second; otherwise it re-resolves appearance, lazily attaches the window, and plays a 240 ms overshoot pop-in. `hide()` also tears down the popup. `updateCount()` is a legacy path for the manual test intent.

```

327 /**
328  * Update the pill with today's [total] reel count, overall brain [health] (0..100), and the
329  * per-platform [breakdown] used to populate the tap-to-open popup.
330  */
331 fun updateHud(total: Int, health: Int, breakdown: List<HudPlatform>) {
332     mainHandler.post {
333         currentTotal = total
334         currentHealth = health.coerceIn(0, 100)
335         currentBreakdown = breakdown
336
337         countTextView?.text = "$total"
338         brainView?.setImageResource(brainMoodDrawable(currentHealth))
339
340         // Gentle pulse when the brain is in a bad way.
341         if (currentHealth < 25) {
342             if (criticalAnimator == null) {
343                 criticalAnimator = ObjectAnimator.ofFloat(pill, "alpha", 0.65f, 1f).apply {
344                     duration = 800
345                     repeatMode = ValueAnimator.REVERSE
346                     repeatCount = ValueAnimator.INFINITE
347                     start()
348                 }
349             }
350         } else {
351             criticalAnimator?.cancel()
352             criticalAnimator = null
353             pill?.alpha = 1f
354         }
355
356         if (isPopupAdded) refreshPopupContent()
357         Log.d(TAG, "HUD updated: total=$total health=$currentHealth")
358     }
359 }
360 }

```

updateHud() — called by the counting service after every reel: store the snapshot, update text + brain face, and manage the **critical pulse**: below 25 health an infinite alpha animation (0.65↔1.0) makes the pill breathe; at 25+ it's cancelled and alpha restored. If the popup is open it re-renders live.

[illegible]

```

391     popupView = scrim
392     try {
393         windowManager?.addView(scrim, params)
394         isPopupAdded = true
395     } catch (e: Exception) {
396         Log.e(TAG, "showPopup failed: ${e.message}", e)
397     }
398 }
399
400 private fun centeredCardParams(): FrameLayout.LayoutParams {
401     val maxw = resources.displayMetrics.widthPixels - dp(40f)
402     val cardW = minOf(maxw, dp(340f))
403     return FrameLayout.LayoutParams(cardW, FrameLayout.LayoutParams.WRAP_CONTENT)
404         .apply { gravity = Gravity.CENTER }
405 }
406
407 private fun refreshPopupContent() {
408     val scrim = popupView as? FrameLayout ?: return
409     scrim.removeAllViews()
410     scrim.addView(buildPopupCard(), centeredCardParams())
411 }

```

The popup: a full-screen FLAG_NOT_FOCUSABLE window whose scrim (66% black) closes on tap. The card is centered, min(screen-40dp, 340dp) wide. refreshPopupContent() just rebuilds the card — crude but correct for a card this small.

```

412
413 private fun buildPopupCard(): View {
414     val ctx = this
415     val healthLabel = when {
416         currentHealth >= 90 -> "Elite"
417         currentHealth >= 75 -> "Healthy"
418         currentHealth >= 50 -> "Tired"
419         currentHealth >= 25 -> "Overstimulated"
420         else -> "Brainrot"
421     }
422
423     val card = LinearLayout(ctx).apply {
424         orientation = LinearLayout.VERTICAL
425         setPadding(dp(20f), dp(20f), dp(20f), dp(16f))
426         background = GradientDrawable().apply {
427             cornerRadius = dp(20f).toFloat()
428             setColor(surfaceColor)
429             setStroke(dp(1f), borderColor)
430         }
431         setOnClickListener { } // consume taps so they don't reach the scrim
432     }
433
434     // Header: brain face + health summary.
435     val header = LinearLayout(ctx).apply {
436         orientation = LinearLayout.HORIZONTAL
437         gravity = Gravity.CENTER_VERTICAL
438     }
439     header.addView(ImageView(ctx).apply {
440         layoutParams = LinearLayout.LayoutParams(dp(56f), dp(56f))
441         setImageResource(brainMoodDrawable(currentHealth))
442     })
443     header.addView(LinearLayout(ctx).apply {
444         orientation = LinearLayout.VERTICAL
445         layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
446         .apply { leftMargin = dp(14f) }
447         addView(TextView(ctx).apply {
448             text = "Brain Health"
449             setTextColor(textSecondaryColor)
450             textSize = 12f
451         })
452         addView(TextView(ctx).apply {
453             text = "$currentHealth% · $healthLabel"
454             setTextColor(textColor)
455             textSize = 20f
456             setTypeface(Typeface.DEFAULT_BOLD)
457         })
458     })
459     card.addView(header)
460
461     card.addView(TextView(ctx).apply {
462         text = "$currentTotal reels scrolled today"
463         setTextColor(textSecondaryColor)
464         textSize = 13f
465         setPadding(0, dp(14f), 0, dp(10f))
466     })
467
468     if (currentBreakdown.isEmpty()) {

```

```

469         card.addView(TextView(ctx).apply {
470             text = "No reels yet today – nice. ■"
471             setTextColor(textSecondaryColor)
472             textSize = 13f
473             setPadding(0, dp(4f), 0, dp(8f))
474         })
475     } else {
476         currentBreakdown.forEach { row ->
477             card.addView(LinearLayout(ctx).apply {
478                 orientation = LinearLayout.HORIZONTAL
479                 gravity = Gravity.CENTER_VERTICAL
480                 setPadding(dp(12f), dp(10f), dp(12f), dp(10f))
481                 background = GradientDrawable().apply {
482                     cornerRadius = dp(10f).toFloat()
483                     setColor(innerColor)
484                 }
485                 layoutParams = LinearLayout.LayoutParams(
486                     LinearLayout.LayoutParams.MATCH_PARENT,
487                     LinearLayout.LayoutParams.WRAP_CONTENT
488                 ).apply { bottomMargin = dp(8f) }
489
490                 addView(PlatformLogoView(ctx, row.platform).apply {
491                     layoutParams = LinearLayout.LayoutParams(dp(28f), dp(28f))
492                     .apply { rightMargin = dp(12f) }
493                 })
494                 addView(TextView(ctx).apply {
495                     text = row.platform.displayName
496                     setTextColor(textColor)
497                     textSize = 14f
498                     layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
499                 })
500                 addView(TextView(ctx).apply {
501                     text = "${row.count} / ${row.limit}"
502                     setTextColor(if (row.count >= row.limit) accentColor else textSecondaryColor)
503                     textSize = 14f
504                     set Typeface(TvDeface.DEFAULT_BOLD)
505                 })
506             })
507         }
508     }
509
510     // Actions.
511     val actions = LinearLayout(ctx).apply {
512         orientation = LinearLayout.HORIZONTAL
513         gravity = Gravity.CENTER_VERTICAL
514         setPadding(0, dp(8f), 0, 0)
515     }
516     actions.addView(TextView(ctx).apply {
517         text = "Open app"
518         setTextColor(Color.WHITE)
519         textSize = 14f
520         setTypeface(Typeface.DEFAULT_BOLD)
521         gravity = Gravity.CENTER
522         setPadding(dp(16f), dp(12f), dp(16f), dp(12f))
523         background = GradientDrawable().apply {
524             cornerRadius = dp(100f).toFloat()
525             setColor(accentColor)
526         }
527         layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
528         setOnClickListener { openApp() }
529     })
530     actions.addView(TextView(ctx).apply {
531         text = "Close"
532         setTextColor(textSecondaryColor)
533         textSize = 14f
534         gravity = Gravity.CENTER
535         setPadding(dp(16f), dp(12f), dp(16f), dp(12f))
536         layoutParams = LinearLayout.LayoutParams(0, ViewGroup.LayoutParams.WRAP_CONTENT, 1f)
537         .apply { leftMargin = dp(8f) }
538         setOnClickListener { removePopup() }
539     })
540     card.addView(actions)
541
542     return card
543 }

```

`buildPopupCard()` — pure programmatic UI. Header: 56 dp brain + 'Brain Health' label + '62% · Tired' line (the five tier names match the Dashboard's labels). Then the count line, then either a zen empty-state or one row per used platform: hand-drawn logo (`PlatformLogoView`), name, and 'count / limit' — tinted terracotta when at/over the limit. The card's empty click listener (line 431) consumes taps so they don't fall through to the scrim and close the popup. Action row: 'Open app' (terracotta pill button) and 'Close'.

```

545     private fun openApp() {
546         removePopup()
547         val launch = packageManager.getLaunchIntentForPackage(packageName)?.apply {
548             addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
549         }
550         if (launch != null) startActivity(launch)
551     }
552
553     private fun removePopup() {
554         if (isPopupAdded && popupView != null) {
555             try {
556                 windowManager?.removeView(popupView)
557             } catch (_: Exception) {}
558         }
559         isPopupAdded = false
560         popupView = null
561     }

```

`openApp()` launches `MainActivity` via the package's launch intent — `FLAG_ACTIVITY_NEW_TASK` is mandatory when starting an Activity from a Service context. `removePopup()` detaches defensively (`removeView` throws if the view is already gone).

[illegible]

The blocking overlay. **onTrackedAppLeft()** is the REMIND-mode reset hook called by the heartbeat. `showBlockingOverlay()` first applies mode-specific suppression — HARD: never; SNOOZE: suppressed while `now < snooze_until_<PLATFORM>`; REMIND: suppressed if this platform was already dismissed this session — then builds a darker (87% black) full-screen scrim with the blocking card. Only one scrim at a time (`isBlockingAdded` guard).


```

614     private fun buildBlockingCard(platform: Platform, limit: Int, mode: BlockingMode): View {
615         val card = LinearLayout(this).apply {
616             orientation = LinearLayout.VERTICAL
617             gravity = Gravity.CENTER_HORIZONTAL
618             setPadding(dp(24f), dp(28f), dp(24f), dp(20f))
619             background = GradientDrawable().apply {
620                 cornerRadius = dp(20f).toFloat()
621                 setColor(surfaceColor)
622                 setStroke(dp(1f), borderColor)
623             }
624             setOnClickListener { }
625         }
626
627         card.addView(ImageView(this).apply {
628             setImageResource(R.drawable.char_stop)
629             adjustViewBounds = true
630             layoutParams = LinearLayout.LayoutParams(
631                 LinearLayout.LayoutParams.WRAP_CONTENT,
632                 dp(120f)
633             ).apply { bottomMargin = dp(12f) }
634         })
635
636         card.addView(TextView(this).apply {
637             text = "Limit Reached"
638             setTextColor(textColor)
639             textSize = 22f
640             set Typeface(Typeface.DEFAULT_BOLD)
641             gravity = Gravity.CENTER
642             layoutParams = LinearLayout.LayoutParams(
643                 LinearLayout.LayoutParams.MATCH_PARENT,
644                 LinearLayout.LayoutParams.WRAP_CONTENT
645             ).apply { bottomMargin = dp(8f) }
646         })
647
648         val subtitle = when (mode) {
649             BlockingMode.HARD -> "You've watched $limit ${platform.displayName} videos today.\nBlocked until mid
■ night. ■"
650             BlockingMode.SNOOZE -> "You've watched $limit ${platform.displayName} videos today.\nTime to take a
■ break! ■"
651             BlockingMode.REMIND -> "You've watched $limit ${platform.displayName} videos today.\nTime to take a
■ break! ■"
652         }
653         card.addView(TextView(this).apply {
654             text = subtitle
655             setTextColor(textSecondaryColor)
656             textSize = 14f
657             gravity = Gravity.CENTER
658             layoutParams = LinearLayout.LayoutParams(
659                 LinearLayout.LayoutParams.MATCH_PARENT,
660                 LinearLayout.LayoutParams.WRAP_CONTENT
661             ).apply { bottomMargin = dp(24f) }

```

buildBlockingCard(): the stop-character illustration, the 'Limit Reached' title, and a mode-specific subtitle (HARD says 'Blocked until midnight', the others 'Time to take a break'). All colors come from the same resolved warm palette as the pill.

```

662     }
663
664     val actions = LinearLayout(this).apply {
665         orientation = LinearLayout.HORIZONTAL
666         gravity = Gravity.CENTER_VERTICAL
667     }
668     // Primary action in every mode: leave the blocked app.
669     actions.addView(TextView(this).apply {
670         text = "Close app"
671         setTextColor(Color.WHITE)
672         textSize = 14f
673         set Typeface(Typeface.DEFAULT_BOLD)
674         gravity = Gravity.CENTER
675         setPadding(dp(16f), dp(12f), dp(12f), dp(12f))
676         background = GradientDrawable().apply {
677             cornerRadius = dp(100f).toFloat()
678             setColor(accentColor)
679         }
680         layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f)
681         setOnClickListener {
682             removeBlockingOverlay()
683             ReelCounterService.instance?.performGlobalAction(
684                 android.accessibilityservice.AccessibilityService.GLOBAL_ACTION_HOME
685             )
686         }
687     })
688     // Secondary action depends on the mode; HARD offers no way to keep scrolling.

```



```

688     when (mode) {
689         BlockingMode.HARD -> Unit
690         BlockingMode.SNOOZE -> actions.addView(TextView(this)).apply {
691             text = "Snooze 5 min"
692             setTextColor(textSecondaryColor)
693             textSize = 14f
694             gravity = Gravity.CENTER
695             setPadding(dp(16f), dp(12f), dp(16f), dp(12f))
696             layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f)
697                 .apply { leftMargin = dp(8f) }
698             setOnClickListener {
699                 prefs.edit()
700                     .putLong(
701                         "snooze_until_${platform.name}",
702                         System.currentTimeMillis() + BlockingMode.SNOOZE_MS
703                     )
704                     .apply()
705                 removeBlockingOverlay()
706             }
707         })
708     }
709     BlockingMode.REMIND -> actions.addView(TextView(this)).apply {
710         text = "Got it"
711         setTextColor(textSecondaryColor)
712         textSize = 14f
713         gravity = Gravity.CENTER
714         setPadding(dp(16f), dp(12f), dp(16f), dp(12f))
715         layoutParams = LinearLayout.LayoutParams(0, LinearLayout.LayoutParams.WRAP_CONTENT, 1f)
716             .apply { leftMargin = dp(8f) }
717         setOnClickListener {
718             sessionDismissed.add(platform.name)
719             removeBlockingOverlay()
720         }
721     })
722 }
723 card.addView(actions)
724
725 return card
726 }

```

The action buttons encode the modes' contracts. **Close app** (always present, always primary) removes the scrim and calls **performGlobalAction(GLOBAL_ACTION_HOME)** on the accessibility service — a superpower only accessibility services have: programmatically pressing Home, which actually gets the user out of the blocked app. **HARD** adds nothing else — no escape. **SNOOZE** adds 'Snooze 5 min', which writes the deadline into prefs (restart-proof) and dismisses; the heartbeat will re-block within a second of expiry. **REMIND** adds 'Got it', which records the platform in `sessionDismissed` — cleared when the user leaves the app, so reopening re-shows the reminder once.

```

727
728     private fun removeBlockingOverlay() {
729         if (isBlockingAdded && blockingView != null) {
730             try {
731                 windowManager?.removeView(blockingView)
732             } catch (_: Exception) {}
733         }
734         isBlockingAdded = false
735         blockingView = null
736     }
737
738     override fun onDestroy() {
739         super.onDestroy()
740         criticalAnimator?.cancel()
741         criticalAnimator = null
742         instance = null
743         removePopup()
744         removeBlockingOverlay()
745         if (isViewAdded && rootView != null) {
746             try {
747                 windowManager?.removeView(rootView)
748             } catch (_: Exception) {}
749             isViewAdded = false
750         }
751         Log.d(TAG, "FloatingCounterService destroyed")
752     }

```

Teardown: remove the scrim, the popup, the pill, stop the animator, clear the instance — each guarded, because removing an unattached view throws.

```
753
754 // █ Custom views █
755
756 /**
757  * Original Canvas-drawn renditions of each platform logo. Mirrors the Compose `PlatformLogo`
```

```

758     * so the popup uses the real logos instead of emoji.
759     */
760     class PlatformLogoView(context: Context, private val platform: Platform) : View(context) {
761         private val paint = Paint(Paint.ANTI_ALIAS_FLAG)
762         private val rect = RectF()
763
764         override fun onDraw(canvas: Canvas) {
765             super.onDraw(canvas)
766             val s = minOf(width, height).toFloat()
767             when (platform) {
768                 Platform.INSTAGRAM -> drawInstagram(canvas, s)
769                 Platform.YOUTUBE -> drawYouTube(canvas, s)
770                 Platform.TIKTOK -> drawTikTok(canvas, s)
771                 Platform.SNAPCHAT -> drawSnapchat(canvas, s)
772             }
773         }
774
775         private fun drawInstagram(canvas: Canvas, s: Float) {
776             val corner = s * 0.28f
777             paint.style = Paint.Style.FILL
778             paint.shader = LinearGradient(
779                 0f, s, s, 0f,
780                 intArrayOf(0xFFFFEDA77.toInt(), 0xFFFF58529.toInt(), 0xFFDD2A7B.toInt(), 0xFF8134AF.toInt()),
781                 null, Shader.TileMode.CLAMP
782             )
783             rect.set(0f, 0f, s, s)
784             canvas.drawRoundRect(rect, corner, corner, paint)
785             paint.shader = null
786             paint.style = Paint.Style.STROKE
787             paint.strokeWidth = s * 0.085f
788             paint.color = Color.WHITE
789             val inset = s * 0.24f
790             rect.set(inset, inset, s - inset, s - inset)
791             canvas.drawRoundRect(rect, s * 0.14f, s * 0.14f, paint)
792             canvas.drawCircle(s / 2f, s / 2f, s * 0.155f, paint)
793             paint.style = Paint.Style.FILL
794             canvas.drawCircle(s * 0.685f, s * 0.315f, s * 0.045f, paint)
795         }
796
797         private fun drawYouTube(canvas: Canvas, s: Float) {
798             val rectH = s * 0.72f
799             val top = (s - rectH) / 2f
800             paint.style = Paint.Style.FILL
801             paint.color = 0xFFE62C2C.toInt()
802             rect.set(0f, top, s, top + rectH)
803             canvas.drawRoundRect(rect, rectH * 0.32f, rectH * 0.32f, paint)
804             paint.color = Color.WHITE
805             val cx = s / 2f
806             val cy = s / 2f
807             val tri = rectH * 0.26f
808             val play = Path().apply {
809                 moveTo(cx - tri * 0.7f, cy - tri)
810                .lineTo(cx - tri * 0.7f, cy + tri)
811                .lineTo(cx + tri * 0.95f, cy)
812                 close()
813             }
814             canvas.drawPath(play, paint)
815         }
816
817         private fun drawTikTok(canvas: Canvas, s: Float) {
818             val corner = s * 0.26f
819             paint.style = Paint.Style.FILL
820             paint.color = 0xFF010101.toInt()
821             rect.set(0f, 0f, s, s)
822             canvas.drawRoundRect(rect, corner, corner, paint)
823
824             fun note(color: Int, dx: Float, dy: Float) {
825                 paint.color = color
826                 paint.style = Paint.Style.STROKE
827                 paint.strokeCap = Paint.Cap.ROUND
828                 paint.strokeWidth = s * 0.085f
829                 val stem = Path().apply {
830                     moveTo(s * 0.56f + dx, s * 0.26f + dy)
831                    .lineTo(s * 0.56f + dx, s * 0.62f + dy)
832                 }
833                 canvas.drawPath(stem, paint)
834                 val flag = Path().apply {
835                     moveTo(s * 0.56f + dx, s * 0.26f + dy)
836                     quadTo(s * 0.74f + dx, s * 0.28f + dy, s * 0.72f + dx, s * 0.40f + dy)
837                 }
838                 canvas.drawPath(flag, paint)
839                 paint.style = Paint.Style.FILL
840                 canvas.drawCircle(s * 0.45f + dx, s * 0.64f + dy, s * 0.085f, paint)
841             }
842             note(0xFF25F4EE.toInt(), -s * 0.025f, s * 0.02f)

```

```

843         note(0xFFFE2C55.toInt(), s * 0.025f, -s * 0.02f)
844         note(Color.WHITE, 0f, 0f)
845     }
846
847     private fun drawSnapchat(canvas: Canvas, s: Float) {
848         val corner = s * 0.26f
849         paint.style = Paint.Style.FILL
850         paint.color = 0xFFFFFC00.toInt()
851         rect.set(0f, 0f, s, s)
852         canvas.drawRoundRect(rect, corner, corner, paint)
853
854         val left = s * 0.28f
855         val right = s * 0.72f
856         val topY = s * 0.22f
857         val bottomY = s * 0.74f
858         val ghost = Path().apply {
859             moveTo(left, bottomY)
860             lineTo(left, s * 0.42f)
861             quadTo(left, topY, s / 2f, topY)
862             quadTo(right, topY, right, s * 0.42f)
863             lineTo(right, bottomY)
864             quadTo(s * 0.64f, bottomY + s * 0.07f, s * 0.575f, bottomY)
865             quadTo(s * 0.5f, bottomY + s * 0.07f, s * 0.425f, bottomY)
866             quadTo(s * 0.36f, bottomY + s * 0.07f, left, bottomY)
867             close()
868         }
869         paint.color = Color.WHITE
870         canvas.drawPath(ghost, paint)
871         paint.color = 0xFFFFFC00.toInt()
872         canvas.drawCircle(s * 0.43f, s * 0.42f, s * 0.03f, paint)
873         canvas.drawCircle(s * 0.57f, s * 0.42f, s * 0.03f, paint)
874     }
875 }
876 }

```

PlatformLogoView — the platform logos as pure Canvas geometry, mirroring the Compose version pixel-for-pixel (the popup is classic Views, so the Compose one can't be reused). Instagram: the brand gradient as a LinearGradient shader on a rounded square, then a white stroked camera body, lens circle and flash dot. YouTube: red rounded rect + white play triangle (a closed Path). TikTok: black squircle, then the same musical note drawn three times — cyan offset one way, pink the other, white on top — recreating the brand's chromatic-aberration look. Snapchat: yellow squircle + white ghost built from quadratic Béziers (dome top, scalloped bottom) + two yellow dot eyes. Drawing logos in code keeps trademarked bitmaps out of the APK (nominative use) and scales perfectly at any size.

PART 7

Notifications, Widget & Theme

The supporting cast: three notification channels, the Glance home-screen widget, and the theme system that keeps the Compose app, the raw-View overlay, and the widget visually in sync through one shared preference.

NotificationHelper.kt — three channels

.../notification/NotificationHelper.kt · 154 lines · Notifications · 154 lines

Since Android 8, every notification must belong to a user-visible **channel** the user can mute individually. The app has three: high-importance milestones, default-importance daily summary, and the low-importance permanent service notice.

```
1 package com.example.brainrottracker.notification
2
3 import android.app.Notification
4 import android.app.NotificationChannel
5 import android.app.NotificationManager
6 import android.app.PendingIntent
7 import android.content.Context
8 import android.content.Intent
9 import android.os.Build
10 import androidx.core.app.NotificationCompat
11 import com.example.brainrottracker.MainActivity
12
13 class NotificationHelper(private val context: Context) {
14
15     companion object {
16         const val CHANNEL_MILESTONES = "reel_milestones"
17         const val CHANNEL_DAILY_SUMMARY = "daily_summary"
18         const val CHANNEL_SERVICE = "service_running"
19         const val SERVICE_NOTIFICATION_ID = 1001
20         const val MILESTONE_NOTIFICATION_ID = 2000
21         const val DAILY_SUMMARY_ID = 3001
22
23         private val MILESTONES = listOf(25, 50, 75, 100, 150, 200)
24     }
25
26     private val shownMilestones = mutableSetOf<Int>()
```

Constants: channel IDs, notification IDs (the service notification reuses ID 1001 so Android replaces rather than stacks it), the milestone thresholds, and **shownMilestones** — per-instance memory of which milestones already fired today.

```
27
28 fun createChannels() {
29     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
30         val manager = context.getSystemService(NotificationManager::class.java)
31
32         val milestoneChannel = NotificationChannel(
33             CHANNEL_MILESTONES,
34             "Reel Milestones",
35             NotificationManager.IMPORTANCE_HIGH
36         ).apply {
37             description = "Alerts when you hit reel watching milestones"
38             enableVibration(true)
39         }
40
41         val summaryChannel = NotificationChannel(
42             CHANNEL_DAILY_SUMMARY,
43             "Daily Summary",
44             NotificationManager.IMPORTANCE_DEFAULT
45         ).apply {
46             description = "End-of-day usage summary"
47         }
48
49         val serviceChannel = NotificationChannel(
50             CHANNEL_SERVICE,
51             "Tracking Service",
52             NotificationManager.IMPORTANCE_LOW
53         ).apply {
```

```

54         description = "Shows when BrainRot Tracker is actively monitoring"
55         setShowBadge(false)
56     }
57
58     manager.createNotificationChannel(milestoneChannel)
59     manager.createNotificationChannel(summaryChannel)
60     manager.createNotificationChannel(serviceChannel)
61 }
62 }

```

createChannels() — wrapped in an SDK check because channels don't exist before API 26. Channel creation is idempotent, so calling it from MainActivity, both services, etc. is safe. Milestones get HIGH importance + vibration (heads-up banner); the service channel gets LOW importance and no badge so the permanent notice stays unobtrusive.

```

63
64 fun checkMilestone(totalReels: Int) {
65     val milestone = MILESTONES.firstOrNull { it == totalReels && it !in shownMilestones }
66     ?: return
67
68     shownMilestones.add(milestone)
69
70     val (emoji, message) = when (milestone) {
71         25 -> "■" to "You've watched 25 reels – doing okay!"
72         50 -> "■■" to "50 reels! Time for a break?"
73         75 -> "■■" to "75 reels – your brain needs rest!"
74         100 -> "■■" to "100 reels! Consider closing the app"
75         150 -> "■■" to "150 reels – serious doomscrolling alert!"
76         200 -> "■■" to "200 reels! Your brain is melting!"
77         else -> "■■" to "You've watched $milestone reels today"
78     }
79
80     val intent = Intent(context, MainActivity::class.java)
81     val pendingIntent = PendingIntent.getActivity(
82         context, 0, intent,
83         PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
84     )
85
86     val notification = NotificationCompat.Builder(context, CHANNEL_MILESTONES)
87         .setSmallIcon(android.R.drawable.ic_dialog_info)
88         .setContentTitle("$emoji BrainRot Alert")
89         .setContentText(message)
90         .setPriority(NotificationCompat.PRIORITY_HIGH)
91         .setContentIntent(pendingIntent)
92         .setAutoCancel(true)
93         .build()
94
95     val manager = context.getSystemService(NotificationManager::class.java)
96     manager.notify(MILESTONE_NOTIFICATION_ID + milestone, notification)
97 }

```

checkMilestone(): fires only when the total *exactly equals* a threshold not yet shown — called after every count, so no value is skipped (counts increment by 1). Each milestone gets escalating copy (■ → ■■ → ■■■) and its own notification ID (2000+milestone) so a new milestone doesn't replace the previous one. The PendingIntent opens MainActivity on tap; FLAG_IMMUTABLE is required on modern Android.

```

98
99 fun showDailySummary(totalReels: Int, totalMinutes: Int, brainHealth: Int) {
100     val intent = Intent(context, MainActivity::class.java)
101     val pendingIntent = PendingIntent.getActivity(
102         context, 0, intent,
103         PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
104     )
105
106     val healthEmoji = when {
107         brainHealth >= 75 -> "■■■"
108         brainHealth >= 50 -> "■■"
109         brainHealth >= 25 -> "■"
110         else -> "■■"
111     }
112
113     val hours = totalMinutes / 60
114     val mins = totalMinutes % 60
115     val timeStr = if (hours > 0) "${hours}h ${mins}m" else "${mins}m"
116
117     val notification = NotificationCompat.Builder(context, CHANNEL_DAILY_SUMMARY)
118         .setSmallIcon(android.R.drawable.ic_dialog_info)
119         .setContentTitle("■ Today's Recap")
120         .setContentText("$totalReels reels • $timeStr screen time • $healthEmoji $brainHealth% brain health"
121         ■ )
122     .setStyle(

```

```

122         NotificationCompat.BigTextStyle()
123         .bigText("$totalReels reels watched\n$timeStr screen time\n$healthEmoji Brain Health: $brain
    ■ Health%\n\nKeep working on your digital habits!")
124     )
125     .setPriority(NotificationCompat.PRIORITY_DEFAULT)
126     .setContentIntent(pendingIntent)
127     .setAutoCancel(true)
128     .build()
129
130     val manager = context.getSystemService(NotificationManager::class.java)
131     manager.notify(DAILY_SUMMARY_ID, notification)
132 }

```

showDailySummary() — the recap posted at day rollover: reel total, screen time formatted as h/m, and a health emoji tier. BigTextStyle provides the expanded multi-line form.

```

133
134     fun getServiceNotification(): Notification {
135         val intent = Intent(context, MainActivity::class.java)
136         val pendingIntent = PendingIntent.getActivity(
137             context, 0, intent,
138             PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE
139         )
140
141         return NotificationCompat.Builder(context, CHANNEL_SERVICE)
142             .setSmallIcon(android.R.drawable.ic_dialog_info)
143             .setContentTitle("BrainRot Tracker Active")
144             .setContentText("Monitoring your screen time")
145             .setPriority(NotificationCompat.PRIORITY_LOW)
146             .setContentIntent(pendingIntent)
147             .setOngoing(true)
148             .build()
149     }
150
151     fun resetDailyMilestones() {
152         shownMilestones.clear()
153     }
154 }

```

getServiceNotification() builds the persistent notice FloatingCounterService passes to startForeground() — **setOngoing(true)** makes it non-dismissable, which is the deal Android demands for letting a service live indefinitely. resetDailyMilestones() clears the fired set at midnight. Honest limitation: shownMilestones is in-memory and per-instance, so a service restart mid-day could re-fire a milestone — at worst a duplicate nudge.

BrainRotWidget.kt — the Glance home-screen widget

.../widget/BrainRotWidget.kt · 147 lines · Glance widget · 147 lines

Glance is Google's library for writing home-screen widgets in Compose-style syntax. Under the hood it compiles to RemoteViews (the only thing launchers can display), so only a restricted composable set is available — note the GlanceModifier instead of Modifier.

```

1 package com.example.brainrottracker.widget
2
3 import android.content.Context
4 import android.content.res.Configuration
5 import androidx.compose.runtime.Composable
6 import androidx.compose.ui.graphics.Color
7 import androidx.compose.ui.unit.dp
8 import androidx.compose.ui.unit.sp
9 import androidx.glance.GlanceId
10 import androidx.glance.GlanceModifier
11 import androidx.glance.Image
12 import androidx.glance.ImageProvider
13 import androidx.glance.action.clickable
14 import androidx.glance.action.actionStartActivity
15 import androidx.glance.appwidget.GlanceAppWidget
16 import androidx.glance.appwidget.cornerRadius
17 import androidx.glance.appwidget.provideContent
18 import androidx.glance.background
19 import androidx.glance.layout.Alignment
20 import androidx.glance.layout.Column
21 import androidx.glance.layout.Row
22 import androidx.glance.layout.Spacer
23 import androidx.glance.layout.fillMaxSize
24 import androidx.glance.layout.height
25 import androidx.glance.layout.padding
26 import androidx.glance.layout.size

```

```

27 import androidx.glance.layout.width
28 import androidx.glance.text.FontWeight
29 import androidx.glance.text.Text
30 import androidx.glance.text.TextStyle
31 import androidx.glance.unit.ColorProvider
32 import com.example.brainrottracker.MainActivity
33 import com.example.brainrottracker.data.local.db.AppDatabase
34 import com.example.brainrottracker.data.local.db.entity.DailyLog
35 import com.example.brainrottracker.ui.components.brainMoodDrawable
36 import java.time.LocalDate
37
38 class BrainRotWidget : GlanceAppWidget() {
39
40     override suspend fun provideGlance(context: Context, id: GlanceId) {
41         val db = AppDatabase.getInstance(context)
42         val todayLog = db.dailyLogDao().getByDateOnce(LocalDate.now().toString())
43         val dark = resolveIsDark(context)
44
45         provideContent {
46             WidgetContent(todayLog = todayLog, dark = dark)
47         }
48     }

```

`provideGlance()` is called whenever Android wants fresh widget content. It reads today's log *directly* from Room — no `ViewModel`, no `Flow`: widgets are snapshots, redrawn only when the launcher asks (or the app triggers an update), not live.

```

49
50 /** Same theme resolution as the floating HUD: user pref, falling back to system. */
51 private fun resolveIsDark(context: Context): Boolean {
52     val prefs = context.getSharedPreferences("brainrot_prefs", Context.MODE_PRIVATE)
53     return when (prefs.getString("theme_mode", "SYSTEM")) {
54         "LIGHT" -> false
55         "DARK" -> true
56         else -> {
57             val night = context.resources.configuration.uiMode and
58                 Configuration.UI_MODE_NIGHT_MASK
59             night == Configuration.UI_MODE_NIGHT_YES
60         }
61     }
62 }

```

Theme resolution duplicated from the HUD (same pref, same fallback) so widget, pill and app always agree on light/dark.

```

63
64 @Composable
65 fun WidgetContent(todayLog: DailyLog?, dark: Boolean) {
66     val totalReels = todayLog?.getTotalReels() ?: 0
67     val healthScore = todayLog?.brainHealthScore ?: 100
68
69     // Warm palette, mirroring the app/HUD colors
70     val bgColor = ColorProvider(if (dark) Color(0xFF252320) else Color(0xFFEFE9DE))
71     val textColor = ColorProvider(if (dark) Color(0xFFFFAF9F) else Color(0xFF141413))
72     val subTextColor = ColorProvider(if (dark) Color(0xFFA09D96) else Color(0xFF6C6A64))
73     val accentColor = ColorProvider(Color(0xFFCC785C))
74     val healthColor = ColorProvider(
75         when {
76             healthScore >= 75 -> Color(0xFF5B8C5A)
77             healthScore >= 50 -> Color(0xFFD9A441)
78             else -> Color(0xFFC04A3A)
79         }
80     )
81
82     Column(
83         modifier = GlanceModifier
84             .fillMaxSize()
85             .background(bgColor)
86             .cornerRadius(20.dp)
87             .padding(12.dp)
88             .clickable(actionStartActivity<MainActivity>()),
89         horizontalAlignment = Alignment.CenterHorizontally,
90         verticalAlignment = Alignment.CenterVertically
91     ) {
92         Text(
93             text = "* BrainRot",
94             style = TextStyle(
95                 color = accentColor,
96                 fontSize = 13.sp,
97                 fontWeight = FontWeight.Bold
98             )
99     )

```

100

WidgetContent: extracts total + health (defaults 0/100 when no row exists yet), maps the warm palette into ColorProviders, and tiers the health color green/gold/red. The root Column is fully clickable via **actionStartActivity<MainActivity>** — tapping anywhere opens the app.

```

101         Spacer(GlanceModifier.height(8.dp))
102
103         Row(verticalAlignment = Alignment.CenterVertically) {
104             // The mascot degrades with today's brain health
105             Image(
106                 provider = ImageProvider(brainMoodDrawable(healthScore)),
107                 contentDescription = "Brain health $healthScore",
108                 modifier = GlanceModifier.size(52.dp)
109             )
110
111             Spacer(GlanceModifier.width(14.dp))
112
113             Column(horizontalAlignment = Alignment.CenterHorizontally) {
114                 Text(
115                     text = "$totalReels",
116                     style = TextStyle(
117                         color = textColor,
118                         fontSize = 26.sp,
119                         fontWeight = FontWeight.Bold
120                     )
121                 )
122                 Text(
123                     text = "reels",
124                     style = TextStyle(color = subtextColor, fontSize = 11.sp)
125                 )
126             }
127
128             Spacer(GlanceModifier.width(16.dp))
129
130             Column(horizontalAlignment = Alignment.CenterHorizontally) {
131                 Text(
132                     text = "$healthScore%",
133                     style = TextStyle(
134                         color = healthColor,
135                         fontSize = 26.sp,
136                         fontWeight = FontWeight.Bold
137                     )
138                 )
139                 Text(
140                     text = "health",
141                     style = TextStyle(color = subtextColor, fontSize = 11.sp)
142                 )
143             }
144         }
145     }
146 }
147 }
```

The layout: '* BrainRot' header, then a row of [mascot image | reels count | health %]. The mascot uses the shared brainMoodDrawable() — the same artwork the Dashboard and pill show, degrading as health drops.

BrainRotWidgetReceiver.kt

.../widget/BrainRotWidgetReceiver.kt · 8 lines · Widget receiver · 8 lines

```

1 package com.example.brainrottracker.widget
2
3 import androidx.glance.appwidget.GlanceAppWidget
4 import androidx.glance.appwidget.GlanceAppWidgetReceiver
5
6 class BrainRotWidgetReceiver : GlanceAppWidgetReceiver() {
7     override val glanceAppWidget: GlanceAppWidget = BrainRotWidget()
8 }
```

The bridge Android requires: widgets are delivered via broadcast, so this receiver (declared in the manifest) catches APPWIDGET_UPDATE and hands off to the Glance widget. Two lines of real code — the base class does everything.

theme/Color.kt — the palettes

.../theme/Color.kt · 76 lines · Design tokens · 76 lines

All colors in one place. The file is a small archaeology of the app's design history: the navy/cyan block at the top is the legacy first design, still referenced by a few components; everything actually on screen today uses the Warm* blocks at the bottom.

```

1  package com.example.brainrottracker.theme
2
3  import androidx.compose.ui.graphics.Color
4
5  // Background gradient
6  val DarkBackground = Color(0xFF070B22) // Deep Navy background
7  val DarkSurface = Color(0xFF0A0F2C)    // Matte Navy surface
8  val DarkSurfaceVariant = Color(0xFF111740)
9
10 // Glass card
11 val GlassWhite = Color(0x08FFFFFF)
12 val GlassBorder = Color(0x12FFFFFF)
13
14 // Primary accent (cyan-blue)
15 val PrimaryCyan = Color(0xFF25F3ED)    // Figma neon cyan
16 val PrimaryBlue = Color(0xFF2B7FFF)    // Figma vibrant blue
17
18 // Danger/Warning
19 val DangerRed = Color(0xFFE7000A)      // Figma primary warning red
20 val DangerOrange = Color(0xFFE12F6B)
21 val WarningYellow = Color(0xFFFF9B629) // Figma warning yellow
22
23 // Success/Streak
24 val SuccessGreen = Color(0xFF20C35F)   // Figma success green
25 val SuccessTeal = Color(0xFFA4F3CF)
26
27
28 // Brain Health States (Premium Matte Color System)
29 val BrainHealthy = Color(0xFF10B981)   // Deep emerald green
30 val BrainTired = Color(0xFFFFBBF24)    // Calming golden yellow
31 val BrainOverstimulated = Color(0xFFFF97316) // Energetic amber-orange
32 val BrainCritical = Color(0xFFEF4444)   // Striking coral red
33 val BrainRot = Color(0xFF881337)        // Dark decaying crimson rose
34
35 // Platform Brand Colors
36 val InstagramPink = Color(0xFFED67AD)
37 val InstagramGradientStart = Color(0xFFFF58529)
38 val InstagramGradientEnd = Color(0xFFDD2A7B)
39 val YouTubeRed = Color(0xFFE62C2C)
40 val TikTokCyan = Color(0xFF00B5BE)
41 val TikTokPink = Color(0xFFFFF0050)
42 val SnapchatYellow = Color(0xFFFFBC600)
43
44 // Text
45 val TextPrimary = Color(0xFFFFFFFF)
46 val TextSecondary = Color(0xB3FFFFFF)
47 val TextTertiary = Color(0x66FFFFFF)
48
49 // Misc
50 val CardBackground = Color(0xFF1E1E3A)
51 val DividerColor = Color(0x1AFFFFFF)

```

Legacy palette: deep-navy backgrounds, neon cyan/blue accents, glassmorphism whites, brain-state colors used by the old BrainAvatar, platform brand colors, and white-with-alpha text tiers (0xB3 = 70% opacity). Kept because GlassCard, PlatformCard, StreakCalendar and BrainAvatar still compile against them.

```

52
53 // Warm / dark palette (used across all screens in dark mode)
54 val WarmBackground = Color(0xFF181715)
55 val WarmSurface = Color(0xFF252320)
56 val WarmBorder = Color(0xFF2F2C28)
57 val WarmStepDim = Color(0xFF1F1E1B)
58 val WarmText = Color(0xFFFFAF9F5)
59 val WarmTextSecondary = Color(0xFFA09D96)
60 val WarmAccent = Color(0xFFCC785C)    // terracotta CTA
61 val WarmGrantedGreen = Color(0xFF5DB872) // granted badge
62
63 // Warm / light palette (used across all screens in light mode)
64 val WarmLightBackground = Color(0xFFFFAF9F5)
65 val WarmLightSurface = Color(0xFFE9E9DE)
66 val WarmLightBorder = Color(0xFFE6DFD8)
67 val WarmLightInner = Color(0xFFE6E6DF)
68 val WarmLightText = Color(0xFF141413)
69 val WarmLightTextSecondary = Color(0xFF6C6A64)
70
71 // Stats chart colors (consistent across light/dark)
72 val StatsChartTikTok = Color(0xFFCC785C)

```

```

73 val StatsChartYouTube = Color(0xFFC64545)
74 val StatsChartInstagram = Color(0xFFE8A55A)
75 val StatsChartSnapchat = Color(0xFF5DB8A6)
76 val WarmError = Color(0xFFC64545)

```

The live design system. Dark mode: near-black warm browns (#181715 background, #252320 surface) with off-white text. Light mode: cream (#FAF9F5) and warm greys. **WarmAccent #CC785C** — the terracotta — is the single brand accent, identical in both modes, used for CTAs, selection states and the tracking badge. The four StatsChart* colors are tuned to stay distinguishable on both backgrounds.

theme/Theme.kt — the Material 3 wrapper

.../theme/Theme.kt · 51 lines · Theme · 51 lines

```

1 package com.example.brainrottracker.theme
2
3 import androidx.compose.foundation.shape.RoundedCornerShape
4 import androidx.compose.material3.*
5 import androidx.compose.runtime.Composable
6 import androidx.compose.ui.unit.dp
7
8 private val BrainRotColorScheme = darkColorScheme(
9     primary = PrimaryCyan,
10    onPrimary = DarkBackground,
11    primaryContainer = PrimaryBlue,
12    onPrimaryContainer = TextPrimary,
13    secondary = SuccessTeal,
14    onSecondary = DarkBackground,
15    secondaryContainer = SuccessGreen,
16    onSecondaryContainer = DarkBackground,
17    tertiary = InstagramPink,
18    onTertiary = TextPrimary,
19    error = DangerRed,
20    onError = DarkBackground,
21    errorContainer = DangerOrange,
22    onErrorContainer = TextPrimary,
23    background = DarkBackground,
24    onBackground = TextPrimary,
25    surface = DarkSurface,
26    onSurface = TextPrimary,
27    surfaceVariant = DarkSurfaceVariant,
28    onSurfaceVariant = TextSecondary,
29    outline = GlassBorder,
30    outlineVariant = DividerColor,
31    surfaceContainerHighest = CardBackground,
32 )
33
34 private val BrainRotShapes = Shapes(
35     small = RoundedCornerShape(8.dp),
36     medium = RoundedCornerShape(16.dp),
37     large = RoundedCornerShape(24.dp),
38     extraLarge = RoundedCornerShape(32.dp),
39 )

```

A Material 3 darkColorScheme built from the *legacy* palette, plus the shared corner radii (8/16/24/32 dp). Honest note: most screens bypass MaterialTheme and pick Warm* colors directly via rememberIsDark() — this scheme mainly feeds Material components like AlertDialog and Slider defaults.

```

40
41 @Composable
42 fun BrainRotTrackerTheme(
43     content: @Composable () -> Unit,
44 ) {
45     MaterialTheme(
46         colorScheme = BrainRotColorScheme,
47         typography = Typography,
48         shapes = BrainRotShapes,
49         content = content,
50     )
51 }

```

BrainRotTrackerTheme is the single wrapper MainActivity applies: color scheme + the Typography scale from Type.kt + shapes, then renders the app inside.

theme/ThemeController.kt — instant theme switching

.../theme/ThemeController.kt · 43 lines · Theme state · 43 lines

```

1 package com.example.brainrottracker.theme
2
3 import android.content.Context
4 import android.content.SharedPreferences
5 import androidx.compose.foundation.isSystemInDarkTheme
6 import androidx.compose.runtime.Composable
7 import androidx.compose.runtime.getValue
8 import androidx.compose.runtime.mutableStateOf
9 import androidx.compose.runtime.setValue
10
11 /**
12  * App-wide theme controller. Holds the user's theme preference (System / Light / Dark)
13  * as Compose state backed by SharedPreferences so every screen recomposes when it changes.
14  */
15 object ThemeController {
16
17     enum class Mode { SYSTEM, LIGHT, DARK }
18
19     var mode by mutableStateOf(Mode.SYSTEM)
20     private set
21
22     private var prefs: SharedPreferences? = null
23
24     fun init(context: Context) {
25         val p = context.getSharedPreferences("brainrot_prefs", Context.MODE_PRIVATE)
26         prefs = p
27         mode = runCatching { Mode.valueOf(p.getString("theme_mode", Mode.SYSTEM.name)!!) }
28             .getOrElse(Mode.SYSTEM)
29     }
30
31     fun selectMode(newMode: Mode) {
32         mode = newMode
33         prefs?.edit()?.putString("theme_mode", newMode.name)?.apply()
34     }
35 }

```

A singleton holding the user's mode (SYSTEM/LIGHT/DARK) as **mutableStateOf** — Compose state, so every composable that reads it recomposes the instant `selectMode()` runs: the whole app recolors live with no restart. The same write also lands in the `brainrot_prefs` `SharedPreferences` file, which is exactly where the (non-Compose) overlay service and widget read `theme_mode` from — one pref key, three consumers. `init()` is called from `MainActivity` before `setContent` so the first frame is already correct; `runCatching` guards against a corrupted stored value.

```

36
37 /** Resolves the active dark/light state based on the user's chosen mode. */
38 @Composable
39 fun rememberIsDark(): Boolean = when (ThemeController.mode) {
40     ThemeController.Mode.SYSTEM -> isSystemInDarkTheme()
41     ThemeController.Mode.LIGHT -> false
42     ThemeController.Mode.DARK -> true
43 }

```

`rememberIsDark()` — the helper every screen calls: resolves SYSTEM via `isSystemInDarkTheme()` (itself reactive to OS dark-mode changes), otherwise returns the explicit choice.

theme/Type.kt — the typography scale

.../theme/Type.kt · 109 lines · Typography · 109 lines

```

1 package com.example.brainrottracker.theme
2
3 import androidx.compose.material3.Typography
4 import androidx.compose.ui.text.TextStyle
5 import androidx.compose.ui.text.font.FontFamily
6 import androidx.compose.ui.text.font.FontWeight
7 import androidx.compose.ui.unit.sp
8
9 val Typography = Typography(
10     displayLarge = TextStyle(
11         fontFamily = FontFamily.SansSerif,
12         fontWeight = FontWeight.Bold,
13         fontSize = 57.sp,
14         lineHeight = 64.sp,
15         letterSpacing = (-0.25).sp,
16     ),
17     displayMedium = TextStyle(
18         fontFamily = FontFamily.SansSerif,
19         fontWeight = FontWeight.Bold,
20         fontSize = 45.sp,

```

```
21         lineHeight = 52.sp,
22     ),
23     displaySmall = TextStyle(
24         fontFamily = FontFamily.SansSerif,
25         fontWeight = FontWeight.Bold,
26         fontSize = 36.sp,
27         lineHeight = 44.sp,
28     ),
29     headlineLarge = TextStyle(
30         fontFamily = FontFamily.SansSerif,
31         fontWeight = FontWeight.SemiBold,
32         fontSize = 32.sp,
33         lineHeight = 40.sp,
34     ),
35     headlineMedium = TextStyle(
36         fontFamily = FontFamily.SansSerif,
37         fontWeight = FontWeight.SemiBold,
38         fontSize = 28.sp,
39         lineHeight = 36.sp,
40     ),
41     headlineSmall = TextStyle(
42         fontFamily = FontFamily.SansSerif,
43         fontWeight = FontWeight.SemiBold,
44         fontSize = 24.sp,
45         lineHeight = 32.sp,
46     ).
47     titleLarge = TextStyle(
48         fontFamily = FontFamily.SansSerif,
49         fontWeight = FontWeight.SemiBold,
50         fontSize = 22.sp,
51         lineHeight = 28.sp,
52     ),
53     titleMedium = TextStyle(
54         fontFamily = FontFamily.SansSerif,
55         fontWeight = FontWeight.Medium,
56         fontSize = 16.sp,
57         lineHeight = 24.sp,
58         letterSpacing = 0.15.sp,
59     ),
60     titleSmall = TextStyle(
61         fontFamily = FontFamily.SansSerif,
62         fontWeight = FontWeight.Medium,
63         fontSize = 14.sp,
64         lineHeight = 20.sp,
65         letterSpacing = 0.1.sp,
66     ),
67     bodyLarge = TextStyle(
68         fontFamily = FontFamily.SansSerif,
69         fontWeight = FontWeight.Normal,
70         fontSize = 16.sp,
71         lineHeight = 24.sp,
72         letterSpacing = 0.5.sp,
73     ),
74     bodyMedium = TextStyle(
75         fontFamily = FontFamily.SansSerif,
76         fontWeight = FontWeight.Normal,
77         fontSize = 14.sp,
78         lineHeight = 20.sp,
79         letterSpacing = 0.25.sp,
80     ),
81     bodySmall = TextStyle(
82         fontFamily = FontFamily.SansSerif,
83         fontWeight = FontWeight.Normal,
84         fontSize = 12.sp,
85         lineHeight = 16.sp,
86         letterSpacing = 0.4.sp,
87     ),
88     labelLarge = TextStyle(
89         fontFamily = FontFamily.SansSerif,
90         fontWeight = FontWeight.Medium,
91         fontSize = 14.sp,
92         lineHeight = 20.sp,
93         letterSpacing = 0.1.sp,
94     ),
95     labelMedium = TextStyle(
96         fontFamily = FontFamily.SansSerif,
97         fontWeight = FontWeight.Medium,
98         fontSize = 12.sp,
99         lineHeight = 16.sp,
100        letterSpacing = 0.5.sp,
101    ),
102    labelSmall = TextStyle(
103        fontFamily = FontFamily.SansSerif,
104        fontWeight = FontWeight.Medium,
105        fontSize = 11.sp,
```

```
106         lineHeight = 16.sp,  
107         letterSpacing = 0.5.sp,  
108     ),  
109 )
```

The full Material 3 type scale — 13 named styles from displayLarge (57 sp, bold, tight letter-spacing) down to labelSmall (11 sp). All use the system sans-serif; weights step Bold → SemiBold → Medium → Normal down the hierarchy. Screens that need the serif logo or bespoke sizes override inline, but Material components (dialogs, buttons) pick their styles from this table.

PART 8

UI Components

Reusable composables. The first three are live (mascot, hand-drawn platform logos); the rest are legacy pieces from the earlier navy/cyan design, kept for reference — flagged as such.

BrainMascot.kt — the star of the show

.../ui/components/BrainMascot.kt · 169 lines · Composable · 169 lines

The mascot is painted artwork (five WebP images in res/drawable-nodpi, one per health tier) with *live animation layered on top in code*: breathing, blinking, and stress-jitter. It appears on the Dashboard hero, in miniature under the Stats chart, in the floating pill, and on the widget.

```
1 package com.example.brainrottracker.ui.components
2
3 import androidx.compose.animation.Crossfade
4 import androidx.compose.animation.core.Animatable
5 import androidx.compose.animation.core.EaseInOutSine
6 import androidx.compose.animation.core.RepeatMode
7 import androidx.compose.animation.core.animateFloat
8 import androidx.compose.animation.core.infiniteRepeatable
9 import androidx.compose.animation.core.rememberInfiniteTransition
10 import androidx.compose.animation.core.tween
11 import androidx.compose.foundation.Image
12 import androidx.compose.foundation.layout.Box
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.runtime.Composable
15 import androidx.compose.runtime.LaunchedEffect
16 import androidx.compose.runtime.getValue
17 import androidx.compose.runtime.mutableFloatStateOf
18 import androidx.compose.runtime.remember
19 import androidx.compose.runtime.setValue
20 import androidx.compose.ui.Alignment
21 import androidx.compose.ui.Modifier
22 import androidx.compose.ui.graphics.TransformOrigin
23 import androidx.compose.ui.graphics.graphicsLayer
24 import androidx.compose.ui.layout.ContentScale
25 import androidx.compose.ui.res.painterResource
26 import com.example.brainrottracker.R
27 import kotlinx.coroutines.delay
28 import kotlin.random.Random
29
30 /**
31  * The BrainPal-style mascot, rendered from detailed painted artwork – one image per
32  * brain-health state – with live animation layered on top:
33  *
34  * - smooth crossfade when the state changes
35  * - idle bob + squash-and-stretch breathing (lively when healthy, sluggish when not)
36  * - a periodic quick "blink" squint
37  * - anxious jitter when overstimulated, trembling when critical, glitchy shake when rotten
38  *
39  * States (drawables in res/drawable-nodpi):
40  * 75-100 HEALTHY brain_healthy
41  * 50-74 TIRED brain_tired
42  * 25-49 OVERSTIM brain_overstimulated
43  * 10-24 CRITICAL brain_critical
44  * 0-9 ROT brain_rot
45  *
46  * @param health 0..100. where 100 is healthiest.
47  */
```

Imports + the behaviour spec in the doc comment: crossfade between states, idle bob with squash-and-stretch (lively when healthy, sluggish when not), periodic blink, and escalating jitter for the three stressed tiers.

```
48 @Composable
49 fun BrainMascot(
50     health: Int,
51     modifier: Modifier = Modifier,
52 ) {
53     val h = health.coerceIn(0, 100)
```

```
54     val t = h / 100f
55
56     val state = when {
57         h >= 75 -> BrainMood.HEALTHY
58         h >= 50 -> BrainMood.TIRED
59         h >= 25 -> BrainMood.OVERSTIMULATED
60         h >= 10 -> BrainMood.CRITICAL
61         else -> BrainMood.ROT
62     }
63
64     val drawableRes = brainMoodDrawable(h)
```

Health (0–100) maps to one of five moods at the same thresholds used everywhere else (75/50/25/10), and to the matching drawable via the shared `brainMoodDrawable()`.

```
65  
66 // ■■ Idle bob + breathing (squash & stretch) ████████████████████████  
67 val infinite = rememberInfiniteTransition(label = "idle")  
68 val bobDuration = when (state) {  
69     BrainMood.HEALTHY -> 1300  
70     BrainMood.TIRED   -> 2400  
71     BrainMood.OVERSTIMULATED -> 900  
72     BrainMood.CRITICAL -> 2000  
73     BrainMood.ROT      -> 3000  
74 }  
75 val phase by infinite.animateFloat(  
76     initialValue = 0f, targetValue = 1f,  
77     animationSpec = infiniteRepeatable(  
78         tween(bobDuration, easing = EaseInOutSine), RepeatMode.Reverse  
79     ),  
80     label = "phase"  
81 )  
82 val bobAmp = 0.035f * (0.3f + 0.7f * t)    // fraction of height  
83 val squashAmp = 0.030f * (0.3f + 0.7f * t)  
84
```

The breathing loop: an infinite $0 \rightarrow 1 \rightarrow 0$ 'phase' animation whose duration depends on mood (healthy breathes fast and lively at 1.3 s; rot is a sluggish 3 s). Amplitudes scale with health ($0.3 + 0.7t$) so a rotten brain barely moves.

```
85 // ■■ Blink-style squint: quick vertical dip at random intervals ■■■■■■
86 val blink = remember { Animatable(0f) } // 0 = open, 1 = squint
87 LaunchedEffect(state) {
88     while (true) {
89         delay(Random.nextLong(2400, 5200))
90         blink.animateTo(1f, tween(70))
91         blink.animateTo(0f, tween(110))
92         if (Random.nextFloat() < 0.25f) {
93             delay(150)
94             blink.animateTo(1f, tween(60))
95             blink.animateTo(0f, tween(100))
96         }
97     }
98 }
```

The blink: an Animatable driven by a coroutine loop — wait a random 2.4–5.2 s, snap to 1 (squint) in 70 ms, release in 110 ms, with a 25% chance of an immediate double-blink. LaunchedEffect(state) restarts the loop when the mood changes.

[illegible]

```

122         delay(70)
123     }
124     else -> { shakeX = 0f; shakeY = 0f; shakeRot = 0f }
125 }
126 }

```

Stress jitter: a per-mood coroutine writes random pixel offsets on a timer — ROT shakes hard and slow (glitchy), CRITICAL trembles fast, OVERSTIMULATED gets a small horizontal-only vibration, calm moods reset to zero.

```

127
128     val bob = (phase - 0.5f) * 2f // -1..1
129     val squashY = 1f - squashAmp * bob - 0.06f * blink.value
130     val squashX = 1f + squashAmp * bob * 0.7f + 0.03f * blink.value
131
132     Box(modifier = modifier, contentAlignment = Alignment.Center) {
133         Crossfade(
134             targetState = drawableRes,
135             animationSpec = tween(600),
136             label = "brainState"
137         ) { res ->
138             Image(
139                 painter = painterResource(res),
140                 contentDescription = "Brain mascot",
141                 contentScale = ContentScale.Fit,
142                 modifier = Modifier
143                     .fillMaxSize()
144                     .graphicsLayer {
145                         translationX = shakeX
146                         translationY = shakeY + bob * bobAmp * size.height
147                         rotationZ = shakeRot
148                         scaleX = squashX
149                         scaleY = squashY
150                         transformOrigin = TransformOrigin(0.5f, 1f)
151                     }
152             )
153         }
154     }
155 }

```

Composition of all signals: bob is mapped to $-1..1$; squashY compresses on the down-bob and during blinks while squashX bulges (volume preservation — classic squash-and-stretch). Crossfade swaps artwork over 600 ms on tier change, and a single **graphicsLayer** applies translation/rotation/scale on the GPU without re-layout. TransformOrigin(0.5, 1) anchors scaling at the bottom-center so the brain squashes 'into the ground' like a real bouncing object.

```

156
157 private enum class BrainMood { HEALTHY, TIRED, OVERSTIMULATED, CRITICAL, ROT }
158
159 /**
160  * The mascot artwork for a brain-health score — shared by the in-app mascot,
161  * the floating HUD, and the home-screen widget so they always agree.
162  */
163 fun brainMoodDrawable(health: Int): Int = when {
164     health >= 75 -> R.drawable.brain_healthy
165     health >= 50 -> R.drawable.brain_tired
166     health >= 25 -> R.drawable.brain_overstimulated
167     health >= 10 -> R.drawable.brain_critical
168     else -> R.drawable.brain_rot
169 }

```

The private mood enum and the shared **brainMoodDrawable()** — the one health→artwork mapping used by the mascot, the HUD pill, and the widget, so all three always agree.

BrainAvatar.kt — the legacy procedural brain

.../ui/components/BrainAvatar.kt · 335 lines · Composable (legacy) · 335 lines

The previous-generation mascot: instead of artwork, it draws an abstract brain entirely with Canvas primitives — glowing lobes, synapse lines, a rotating orbit ring and a circular score meter. Superseded by BrainMascot on every screen but kept in the codebase; it's a nice study in procedural Compose drawing.

```

1 package com.example.brainrottracker.ui.components
2
3 import androidx.compose.animation.animateColorAsState
4 import androidx.compose.animation.core.Animatable
5 import androidx.compose.animation.core.EaseInOutSine

```



```

6 import androidx.compose.animation.core.FastOutSlowInEasing
7 import androidx.compose.animation.core.InfiniteTransition
8 import androidx.compose.animation.core.LinearEasing
9 import androidx.compose.animation.core.RepeatMode
10 import androidx.compose.animation.core.animateFloat
11 import androidx.compose.animation.core.infiniteRepeatable
12 import androidx.compose.animation.core.rememberInfiniteTransition
13 import androidx.compose.animation.core.tween
14 import androidx.compose.foundation.Canvas
15 import androidx.compose.foundation.layout.Box
16 import androidx.compose.foundation.layout.size
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.Text
19 import androidx.compose.runtime.Composable
20 import androidx.compose.runtime.LaunchedEffect
21 import androidx.compose.runtime.getValue
22 import androidx.compose.runtime.mutableStateOf
23 import androidx.compose.runtime.remember
24 import androidx.compose.runtime.setValue
25 import androidx.compose.ui.Alignment
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.geometry.Offset
28 import androidx.compose.ui.graphics.Brush
29 import androidx.compose.ui.graphics.Color
30 import androidx.compose.ui.graphics.StrokeCap
31 import androidx.compose.ui.graphics.drawscope.Stroke
32 import androidx.compose.ui.graphics.drawscope.rotate
33 import androidx.compose.ui.graphics.graphicsLayer
34 import androidx.compose.ui.text.font.FontWeight
35 import androidx.compose.ui.unit.Dp
36 import androidx.compose.ui.unit.dp
37 import androidx.compose.ui.unit.sp
38 import com.example.brainrottracker.theme.BrainCritical
39 import com.example.brainrottracker.theme.BrainHealthy
40 import com.example.brainrottracker.theme.BrainOverstimulated
41 import com.example.brainrottracker.theme.BrainRot
42 import com.example.brainrottracker.theme.BrainTired
43 import kotlinx.coroutines.delay
44 import kotlin.math.cos
45 import kotlin.math.sin
46
47 @Composable
48 fun BrainAvatar(
49     healthScore: Int,
50     modifier: Modifier = Modifier,
51     size: Dp = 200.dp
52 ) {
53     val state = remember(healthScore) {
54         when {
55             healthScore >= 75 -> "HEALTHY"
56             healthScore >= 50 -> "TIRED"
57             healthScore >= 25 -> "OVERSTIMULATED"
58             healthScore >= 10 -> "CRITICAL"
59             else -> "ROT"
60         }
61     }

```

Imports and the five-state mapping (this one splits the bottom tiers at 10 rather than using the artwork tiers).

```

62
63 // Dynamic state-specific color mapping
64 val baseColor by animateColorAsState(
65     targetValue = when (state) {
66         "HEALTHY" -> BrainHealthy
67         "TIRED" -> BrainTired
68         "OVERSTIMULATED" -> BrainOverstimulated
69         "CRITICAL" -> BrainCritical
70         else -> BrainRot
71     },
72     animationSpec = tween(800),
73     label = "brainColor"
74 )
75
76 val secondColor by animateColorAsState(
77     targetValue = when (state) {
78         "HEALTHY" -> Color(0xFF00B4D8) // Teal/Cyan helper
79         "TIRED" -> Color(0xFFD97706) // Saturated Amber
80         "OVERSTIMULATED" -> Color(0xFFEA580C) // Saturated Orange
81         "CRITICAL" -> Color(0xFFDC2626) // Saturated Red
82         else -> Color(0xFF4C0519) // Deep Rose Black
83     },
84     animationSpec = tween(800),
85     label = "brainColor2"
86 )
87

```

```

88 // Dynamic pulse duration based on stimulation state
89 val pulseDuration = when (state) {
90     "HEALTHY" -> 3000
91     "TIRED" -> 2000
92     "OVERSTIMULATED" -> 1200
93     "CRITICAL" -> 700
94     else -> 400 // Fast erratic vibration pulse
95 }
96
97 val infiniteTransition: InfiniteTransition = rememberInfiniteTransition(label = "brainAnim")
98
99 val pulseScale by infiniteTransition.animateFloat(
100     initialValue = if (state == "ROT") 0.93f else 0.88f,
101     targetValue = 1.0f,
102     animationSpec = infiniteRepeatable(
103         animation = tween(pulseDuration, easing = EaseInOutSine),
104         repeatMode = RepeatMode.Reverse
105     ),
106     label = "pulseScale"
107 )
108
109 val glowAlpha by infiniteTransition.animateFloat(
110     initialValue = if (state == "ROT") 0.05f else 0.1f,
111     targetValue = if (state == "ROT") 0.15f else 0.3f,
112     animationSpec = infiniteRepeatable(
113         animation = tween(pulseDuration, easing = EaseInOutSine),
114         repeatMode = RepeatMode.Reverse
115     ),
116     label = "glowAlpha"
117 )
118
119 // Rotating ring
120 val ringRotation by infiniteTransition.animateFloat(
121     initialValue = 0f,
122     targetValue = 360f,
123     animationSpec = infiniteRepeatable(
124         animation = tween(if (state == "ROT") 4000 else 8000, easing = LinearEasing),
125         repeatMode = RepeatMode.Restart
126     ),
127     label = "ringRotation"
128 )

```

Animated visual parameters: two `animateColorAsState` calls tween the base/secondary colors over 800 ms when the state changes; pulse speed encodes stress (3 s calm heartbeat → 400 ms erratic vibration); an infinite transition drives pulse scale, glow alpha and a ring rotation that spins twice as fast when rotten.

```

129
130 // Jitter/Glitch Offset for high stimulation states
131 var shakeX by remember { mutableStateOf(0f) }
132 var shakeY by remember { mutableStateOf(0f) }
133
134 LaunchedEffect(state) {
135     if (state == "ROT") {
136         while (true) {
137             shakeX = ((-6..6).random()).toFloat()
138             shakeY = ((-6..6).random()).toFloat()
139             delay(40)
140         }
141     } else if (state == "CRITICAL") {
142         while (true) {
143             shakeX = ((-2..2).random()).toFloat()
144             shakeY = ((-2..2).random()).toFloat()
145             delay(60)
146         }
147     } else {
148         shakeX = 0f
149         shakeY = 0f
150     }
151 }
152
153 // Arc fill for score
154 val arcSweep = remember { Animatable(0f) }
155 LaunchedEffect(healthScore) {
156     arcSweep.animateTo(
157         targetValue = 360f * (healthScore / 100f),
158         animationSpec = tween(1200, easing = FastOutSlowInEasing)
159     )
160 }

```

Jitter via `LaunchedEffect` coroutines (same trick `BrainMascot` inherited) and an `Animatable` arc sweep that eases to $360^\circ \times (\text{score}/100)$ whenever the score changes — the score meter literally fills/unfills.

```

161
162     Box(
163         contentAlignment = Alignment.Center,
164         modifier = modifier
165             .size(size)
166             .graphicsLayer {
167                 translationX = shakeX
168                 translationY = shakeY
169             }
170     ) {
171         Canvas(modifier = Modifier.size(size)) {
172             val center = Offset(this.size.width / 2, this.size.height / 2)
173             val maxRadius = this.size.minDimension / 2
174
175             // ■■ Outermost subtle ambient glow (toned down by 60%) ■■■■■■■■■■■■
176             drawCircle(
177                 brush = Brush.radialGradient(
178                     colors = listOf(
179                         baseColor.copy(alpha = glowAlpha * 0.25f),
180                         Color.Transparent
181                     ),
182                     center = center,
183                     radius = maxRadius * 1.35f * pulseScale
184                 ),
185                 radius = maxRadius * 1.35f * pulseScale,
186                 center = center
187             )
188
189             // ■■ Rotating orbit rings (only show when healthy/tired to prevent visual noise) ■■
190             if (state == "HEALTHY" || state == "TIRED") {
191                 rotate(ringRotation, pivot = center) {
192                     val dashCount = 8
193                     val orbitRadius = maxRadius * 0.88f
194                     for (d in 0 until dashCount) {
195                         val angle = Math.toRadians((d * 360.0 / dashCount))
196                         val x = center.x + orbitRadius * cos(angle).toFloat()
197                         val y = center.y + orbitRadius * sin(angle).toFloat()
198                         drawCircle(
199                             color = baseColor.copy(alpha = 0.25f),
200                             radius = 3f,
201                             center = Offset(x, y)
202                         )
203                     }
204                 }
205             }

```

Drawing begins: the whole Box shakes via graphicsLayer translation. Inside the Canvas: a radial-gradient ambient glow that breathes with the pulse, then — only when calm — 8 orbit dots placed by cos/sin around a rotating ring.

```

206
207         // ■■ Main brain lobes silhouette ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
208         val r = maxRadius * 0.45f * pulseScale
209         val lobeRadius = r * 0.52f
210
211         // Left hemisphere lobes centers
212         val lFrontal = Offset(center.x - r * 0.35f, center.y - r * 0.3f)
213         val lParietal = Offset(center.x - r * 0.45f, center.y + r * 0.05f)
214         val lOccipital = Offset(center.x - r * 0.3f, center.y + r * 0.35f)
215         val lTemporal = Offset(center.x - r * 0.15f, center.y + r * 0.05f)
216
217         // Right hemisphere lobes centers
218         val rFrontal = Offset(center.x + r * 0.35f, center.y - r * 0.3f)
219         val rParietal = Offset(center.x + r * 0.45f, center.y + r * 0.05f)
220         val rOccipital = Offset(center.x + r * 0.3f, center.y + r * 0.35f)
221         val rTemporal = Offset(center.x + r * 0.15f, center.y + r * 0.05f)
222
223         // Dynamic opacity for synapses
224         val synapseAlpha = when (state) {
225             "ROT" -> 0.1f
226             "CRITICAL" -> 0.25f
227             else -> 0.4f
228         }
229         val synapseColor = secondColor.copy(alpha = synapseAlpha)
230
231         fun drawSynapse(from: Offset, to: Offset) {
232             drawLine(
233                 color = synapseColor,
234                 start = from,
235                 end = to,
236                 strokeWidth = 2f
237             )
238         }
239
240         // Draw synapses (neural network paths)

```

```

241 drawSynapse(lParietal, lParietal)
242 drawSynapse(lParietal, lOccipital)
243 drawSynapse(lOccipital, lTemporal)
244 drawSynapse(lTemporal, lFrontal)
245 drawSynapse(rFrontal, rParietal)
246 drawSynapse(rParietal, rOccipital)
247 drawSynapse(rOccipital, rTemporal)
248 drawSynapse(rTemporal, rFrontal)
249 drawSynapse(lFrontal, rFrontal)
250 drawSynapse(lOccipital, rOccipital)
251
252 // Draw the lobes (allowing overlapping spheres representing brain halves)
253 val lobesList = listOf(
254     lFrontal, lParietal, lOccipital, lTemporal,
255     rFrontal, rParietal, rOccipital, rTemporal
256 )
257
258 lobesList.forEach { lobeCenter ->
259     drawCircle(
260         brush = Brush.radialGradient(
261             colors = listOf(
262                 baseColor.copy(alpha = if (state == "ROT") 0.4f else 0.55f),
263                 baseColor.copy(alpha = 0.05f)
264             ),
265             center = lobeCenter,
266             radius = lobeRadius * 1.3f
267         ),
268         radius = lobeRadius,
269         center = lobeCenter
270     )
271     // Specular highlights
272     drawCircle(
273         color = Color.White.copy(alpha = if (state == "ROT") 0.05f else 0.15f),
274         radius = lobeRadius * 0.25f,
275         center = Offset(lobeCenter.x - lobeRadius * 0.2f, lobeCenter.y - lobeRadius * 0.2f)
276     )
277 }

```

The brain itself: eight lobe centers laid out symmetrically (frontal/parietal/occipital/temporal per hemisphere), connected by 'synapse' lines whose opacity fades as health drops, then each lobe drawn as a radial-gradient sphere with a small white specular highlight — cheap 3D.

[illegible]

```

319         size = androidx.compose.ui.geometry.Size(maxRadius * 1.7f, maxRadius * 1.7f)
320     )
321 }
322
323 // Central health score typography display
324 Box(contentAlignment = Alignment.Center) {
325     Text(
326         text = "$healthScore",
327         style = MaterialTheme.typography.displaySmall.copy(
328             fontWeight = FontWeight.ExtraBold,
329             fontSize = (size.value * 0.22f).sp,
330             color = Color.White
331         )
332     )
333 }
334 }
335 }

```

The core glow, then the score meter: a faint full-circle track and a sweep-gradient arc with rounded caps drawn from -90° (12 o'clock). The numeric score sits centered on top, sized relative to the avatar (22% of its diameter).

PlatformLogo.kt — hand-drawn brand logos

../ui/components/PlatformLogo.kt · 153 lines · Composable · 153 lines

The Compose twin of PlatformLogoView in the overlay service — same four logos, same geometry, drawn as vectors so no trademarked bitmap ships in the APK (identifying an app by its logo is 'nominative use').

```

1  package com.example.brainrottracker.ui.components
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.runtime.Composable
5  import androidx.compose.ui.Modifier
6  import androidx.compose.ui.geometry.CornerRadius
7  import androidx.compose.ui.geometry.Offset
8  import androidx.compose.ui.geometry.Size
9  import androidx.compose.ui.graphics.Brush
10 import androidx.compose.ui.graphics.Color
11 import androidx.compose.ui.graphics.Path
12 import androidx.compose.ui.graphics.StrokeCap
13 import androidx.compose.ui.graphics.drawscope.DrawScope
14 import androidx.compose.ui.graphics.drawscope.Stroke
15 import com.example.brainrottracker.data.model.Platform
16
17 /**
18  * Original Compose-drawn renditions of each platform's logo, used to identify which app a
19  * count belongs to (nominative use). Drawn as vectors – no bitmap assets required.
20  */
21 @Composable
22 fun PlatformLogo(
23     platform: Platform,
24     modifier: Modifier = Modifier,
25 ) {
26     Canvas(modifier = modifier) {
27         when (platform) {
28             Platform.INSTAGRAM -> drawInstagram()
29             Platform.YOUTUBE -> drawYouTube()
30             Platform.TIKTOK -> drawTikTok()
31             Platform.SNAPCHAT -> drawSnapchat()
32         }
33     }
34 }

```

The dispatcher: a Canvas composable that routes to one private DrawScope extension per platform. Everything below is resolution-independent — coordinates are fractions of the canvas size s .

```

35
36 private fun DrawScope.drawInstagram() {
37     val s = size.minDimension
38     val corner = s * 0.28f
39     // Warm gradient squircle.
40     drawRoundRect(
41         brush = Brush.linearGradient(
42             colors = listOf(Color(0xFFFEA77), Color(0xFFFF58529), Color(0xFFDD2A7B), Color(0xFF8134AF)),
43             start = Offset(0f, s),
44             end = Offset(s, 0f)
45         ),
46         topLeft = Offset(0f, 0f),
47         size = Size(s, s),

```

```

48     cornerRadius = CornerRadius(corner, corner)
49 )
50 val stroke = Stroke(width = s * 0.085f)
51 // Rounded-square camera body.
52 val inset = s * 0.24f
53 drawRoundRect(
54     color = Color.White,
55     topLeft = Offset(inset, inset),
56     size = Size(s - inset * 2, s - inset * 2),
57     cornerRadius = CornerRadius(s * 0.14f, s * 0.14f),
58     style = stroke
59 )
60 // Lens.
61 drawCircle(Color.White, radius = s * 0.155f, center = Offset(s / 2f, s / 2f), style = stroke)
62 // Flash dot.
63 drawCircle(Color.White, radius = s * 0.045f, center = Offset(s * 0.685f, s * 0.315f))
64 }

```

Instagram: the brand gradient (yellow→orange→pink→purple) diagonal across a 28%-radius squircle, then the camera drawn in stroked white — body, lens, flash dot. Stroke width 8.5% of size keeps proportions at any scale.

```

65
66 private fun DrawScope.drawYouTube() {
67     val w = size.width
68     val h = size.height
69     val rectH = h * 0.72f
70     val top = (h - rectH) / 2f
71     drawRoundRect(
72         color = Color(0xFFE62C2C),
73         topLeft = Offset(0f, top),
74         size = Size(w, rectH),
75         cornerRadius = CornerRadius(rectH * 0.32f, rectH * 0.32f)
76     )
77     // White play triangle.
78     val cx = w / 2f
79     val cy = h / 2f
80     val tri = rectH * 0.26f
81     val play = Path().apply {
82         moveTo(cx - tri * 0.7f, cy - tri)
83         lineTo(cx - tri * 0.7f, cy + tri)
84         lineTo(cx + tri * 0.95f, cy)
85         close()
86     }
87     drawPath(play, Color.White)
88 }

```

YouTube: the red rounded rectangle (72% of height) and the white play triangle as a closed three-point Path.

```

89
90 private fun DrawScope.drawTikTok() {
91     val s = size.minDimension
92     val corner = s * 0.26f
93     // Black squircle backdrop so the offset-color note reads on any theme.
94     drawRoundRect(
95         color = Color(0xFF010101),
96         topLeft = Offset(0f, 0f),
97         size = Size(s, s),
98         cornerRadius = CornerRadius(corner, corner)
99     )
100     // Musical note, drawn three times with a cyan/pink offset for the signature glitch look.
101     fun note(color: Color, dx: Float, dy: Float) {
102         val stroke = Stroke(width = s * 0.085f, cap = StrokeCap.Round)
103         val stem = Path().apply {
104             moveTo(s * 0.56f + dx, s * 0.26f + dy)
105             lineTo(s * 0.56f + dx, s * 0.62f + dy)
106         }
107         drawPath(stem, color, style = stroke)
108         // Flag at the top of the stem.
109         val flag = Path().apply {
110             moveTo(s * 0.56f + dx, s * 0.26f + dy)
111             quadraticBezierTo(s * 0.74f + dx, s * 0.28f + dy, s * 0.72f + dx, s * 0.40f + dy)
112         }
113         drawPath(flag, color, style = stroke)
114         // Note head.
115         drawCircle(color, radius = s * 0.085f, center = Offset(s * 0.45f + dx, s * 0.64f + dy))
116     }
117     note(Color(0xFF25F4EE), -s * 0.025f, s * 0.02f) // cyan
118     note(Color(0xFFFE2C55), s * 0.025f, -s * 0.02f) // red/pink
119     note(Color.White, 0f, 0f)
120 }

```

TikTok: black squircle, then a local `note()` helper draws the musical note three times — cyan shifted one way, pink the other, white centered — recreating the brand's chromatic-aberration glitch. The note = a stroked stem, a quadratic-Bézier flag, and a filled head circle.

```

121
122 private fun DrawScope.drawSnapchat() {
123     val s = size.minDimension
124     val corner = s * 0.26f
125     drawRoundRect(
126         color = Color(0xFFFFFC00),
127         topLeft = Offset(0f, 0f),
128         size = Size(s, s),
129         cornerRadius = CornerRadius(corner, corner)
130     )
131     // Simple ghost silhouette.
132     val ghost = Path().apply {
133         val left = s * 0.28f
134         val right = s * 0.72f
135         val topY = s * 0.22f
136         val bottomY = s * 0.74f
137         moveTo(left, bottomY)
138         // body sides up to a rounded dome
139         lineTo(left, s * 0.42f)
140         quadraticBezierTo(left, topY, s / 2f, topY)
141         quadraticBezierTo(right, topY, right, s * 0.42f)
142         lineTo(right, bottomY)
143         // scalloped bottom (three little bumps)
144         quadraticBezierTo(s * 0.64f, bottomY + s * 0.07f, s * 0.575f, bottomY)
145         quadraticBezierTo(s * 0.5f, bottomY + s * 0.07f, s * 0.425f, bottomY)
146         quadraticBezierTo(s * 0.36f, bottomY + s * 0.07f, left, bottomY)
147         close()
148     }
149     drawPath(ghost, Color.White)
150     // Eyes + cheeks dots in the brand yellow.
151     drawCircle(Color(0xFFFFFC00), radius = s * 0.03f, center = Offset(s * 0.43f, s * 0.42f))
152     drawCircle(Color(0xFFFFFC00), radius = s * 0.03f, center = Offset(s * 0.57f, s * 0.42f))
153 }

```

Snapchat: yellow squircle, ghost as one closed path (straight sides, Bézier dome, three-bump scalloped bottom), finished with two yellow eye dots punched into the white body.

AnimatedCounter.kt — rolling digits

.../ui/components/AnimatedCounter.kt · 38 lines · Composable · 38 lines

```

1 package com.example.brainrottracker.ui.components
2
3 import androidx.compose.animation.AnimatedContent
4 import androidx.compose.animation.slideInVertically
5 import androidx.compose.animation.slideOutVertically
6 import androidx.compose.animation.togetherWith
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.material3.Text
9 import androidx.compose.runtime.Composable
10 import androidx.compose.ui.Modifier
11 import androidx.compose.ui.text.TextStyle
12
13 @Composable
14 fun AnimatedCounter(
15     count: Int,
16     modifier: Modifier = Modifier,
17     style: TextStyle = TextStyle.Default,
18 ) {
19     // Break into individual digits for per-digit rolling animation
20     val countStr = count.toString()
21     Row(modifier = modifier) {
22         countStr.forEach { digit ->
23             AnimatedContent(
24                 targetState = digit,
25                 transitionSpec = {
26                     // Slide up when increasing
27                     slideInVertically { -it } togetherWith slideOutVertically { it }
28                 },
29                 label = "digit_$digit"
30             ) { targetDigit ->
31                 Text(
32                     text = targetDigit.toString(),
33                     style = style
34                 )
35             }
36         }
37     }
38 }

```

```

36     }
37   }
38 }

```

Splits a number into characters and wraps *each digit* in its own `AnimatedContent`, so when 47→48 only the '7' slides up and out while '8' slides in — an odometer effect. **togetherWith** pairs the enter and exit transitions. Currently unused by the live screens (the Dashboard shows a plain `Text`) but kept as a ready-made component.

GlassCard.kt — legacy glassmorphism card

.../ui/components/GlassCard.kt · 135 lines · Composable (legacy) · 135 lines

From the navy/cyan design era; today only `PlatformCard` uses it. Worth reading for the 3D press-tilt gesture math.

```

1  package com.example.brainrottracker.ui.components
2
3  import androidx.compose.animation.core.animateFloatAsState
4  import androidx.compose.animation.core.spring
5  import androidx.compose.foundation.BorderStroke
6  import androidx.compose.foundation.gestures.detectTapGestures
7  import androidx.compose.foundation.layout.Box
8  import androidx.compose.foundation.layout.padding
9  import androidx.compose.foundation.shape.RoundedCornerShape
10 import androidx.compose.material3.Surface
11 import androidx.compose.runtime.Composable
12 import androidx.compose.runtime.getValue
13 import androidx.compose.runtime.mutableStateOf
14 import androidx.compose.runtime.remember
15 import androidx.compose.runtime.setValue
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.draw.drawBehind
18 import androidx.compose.ui.geometry.Offset
19 import androidx.compose.ui.graphics.Brush
20 import androidx.compose.ui.graphics.Color
21 import androidx.compose.ui.graphics.graphicsLayer
22 import androidx.compose.ui.input.pointer.pointerInput
23 import androidx.compose.ui.unit.dp
24 import com.example.brainrottracker.theme.CardBackground
25 import com.example.brainrottracker.theme.GlassBorder
26 import com.example.brainrottracker.theme.PrimaryCyan
27
28 @Composable
29 fun GlassCard(
30     modifier: Modifier = Modifier,
31     accentColor: Color = PrimaryCyan,
32     enableTilt: Boolean = true,
33     backgroundColor: Color = CardBackground.copy(alpha = 0.7f),
34     content: @Composable () -> Unit
35 ) {
36     var isPressed by remember { mutableStateOf(false) }
37     var tiltX by remember { mutableStateOf(0f) }
38     var tiltY by remember { mutableStateOf(0f) }
39
40     val animatedTiltX by animateFloatAsState(
41         targetValue = if (isPressed) tiltX else 0f,
42         animationSpec = spring(dampingRatio = 0.6f, stiffness = 300f),
43         label = "tiltX"
44     )
45     val animatedTiltY by animateFloatAsState(
46         targetValue = if (isPressed) tiltY else 0f,
47         animationSpec = spring(dampingRatio = 0.6f, stiffness = 300f),
48         label = "tiltY"
49     )
50     val scale by animateFloatAsState(
51         targetValue = if (isPressed) 0.97f else 1f,
52         animationSpec = spring(dampingRatio = 0.7f, stiffness = 400f),
53         label = "scale"
54     )

```

State + springs: where you press determines tiltX/tiltY (−1...1 from center); three `animateFloatAsState` springs ease tilt and a 0.97 press-scale back to rest on release.

```

55
56     Surface(
57         modifier = modifier
58             .graphicsLayer {
59                 scaleX = scale
60                 scaleY = scale
61                 if (enableTilt) {

```



```

62         rotationX = animatedTiltY * 8f
63         rotationY = animatedTiltX * 8f
64         cameraDistance = 12f * density
65     }
66     shadowElevation = if (isPressed) 2.dp.toPx() else 12.dp.toPx()
67 }
68 .pointerInput(Unit) {
69     detectTapGestures(
70         onPress = { offset ->
71             isPressed = true
72             if (enableTilt) {
73                 val centerX = size.width / 2f
74                 val centerY = size.height / 2f
75                 tiltX = ((offset.x - centerX) / centerX).coerceIn(-1f, 1f)
76                 tiltY = -((offset.y - centerY) / centerY).coerceIn(-1f, 1f)
77             }
78             tryAwaitRelease()
79             isPressed = false
80             tiltX = 0f
81             tiltY = 0f
82         }
83     )
84 }
85 .drawBehind {
86     // Subtle premium depth shadow glow
87     drawCircle(
88         brush = Brush.radialGradient(
89             colors = listOf(
90                 accentColor.copy(alpha = 0.02f),
91                 Color.Transparent
92             ),
93         center = Offset(size.width / 2f, size.height / 2f),
94         radius = size.maxDimension * 0.7f
95     ),
96     radius = size.maxDimension * 0.7f,
97     center = Offset(size.width / 2f, size.height * 0.9f)
98 )
99 },

```

The graphicsLayer applies the tilt as rotationX/rotationY ($\pm 8^\circ$) with a camera distance to control perspective depth; pointerInput's detectTapGestures computes the press offset, awaits release (tryAwaitRelease), then resets. drawBehind paints a faint accent-tinted radial glow under the card.

```

100     shape = RoundedCornerShape(24.dp), // More premium rounded corners (24.dp)
101     color = backgroundColor,
102     border = BorderStroke(
103         width = 1.dp,
104         brush = Brush.linearGradient(
105             colors = listOf(
106                 accentColor.copy(alpha = 0.15f),
107                 GlassBorder.copy(alpha = 0.3f),
108                 accentColor.copy(alpha = 0.05f)
109             )
110         )
111     ),
112     tonalElevation = 0.dp,
113     shadowElevation = 8.dp,
114 ) {
115     Box(
116         modifier = Modifier
117             .drawBehind {
118                 // Inner glass shimmer highlight (top-left to bottom-right)
119                 drawRect(
120                     brush = Brush.linearGradient(
121                         colors = listOf(
122                             Color.White.copy(alpha = 0.06f),
123                             Color.Transparent
124                         ),
125                     start = Offset(0f, 0f),
126                     end = Offset(size.width * 0.6f, size.height * 0.6f)
127                 )
128             }
129     )
130     .padding(16.dp)
131 ) {
132     content()
133 }
134 }
135 }

```

The Surface: 24 dp corners, translucent background, and a 1 dp gradient border (accent → glass white → accent) that fakes edge lighting. Inside, another drawBehind lays a diagonal white shimmer (6% alpha) before the content padding — the 'glass' look.

PlatformCard.kt — legacy per-platform stat card

.../ui/components/PlatformCard.kt · 139 lines · Composable (legacy) · 139 lines

```
1 package com.example.brainrottracker.ui.components
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.layout.Arrangement
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.Column
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.foundation.layout.size
9 import androidx.compose.foundation.layout.fillMaxWidth
10 import androidx.compose.foundation.layout.padding
11 import androidx.compose.foundation.shape.RoundedCornerShape
12 import androidx.compose.material.icons.Icons
13 import androidx.compose.material.icons.outlined.AccessTime
14 import androidx.compose.material.icons.outlined.Movie
15 import androidx.compose.material3.Icon
16 import androidx.compose.material3.MaterialTheme
17 import androidx.compose.material3.Text
18 import androidx.compose.runtime.Composable
19 import androidx.compose.ui.Alignment
20 import androidx.compose.ui.Modifier
21 import androidx.compose.ui.draw.clip
22 import androidx.compose.ui.graphics.Color
23 import androidx.compose.ui.text.font.FontWeight
24 import androidx.compose.ui.unit.dp
25 import androidx.compose.ui.unit.sp
26
27 @Composable
28 fun PlatformCard(
29     platformName: String,
30     reelCount: Int,
31     minutes: Int,
32     reelLimit: Int,
33     minuteLimit: Int,
34     platformColor: Color,
35     modifier: Modifier = Modifier,
36     reelLabel: String = "reels"
37 ) {
38     GlassCard(
39         modifier = modifier,
40         accentColor = platformColor,
41         enableTilt = false,
42         backgroundColor = Color(0xFF14152A)
43     ) {
44         Column(
45             modifier = Modifier
46                 .fillMaxWidth()
47                 .padding(vertical = 4.dp),
48             verticalArrangement = Arrangement.spacedBy(10.dp)
49         ) {
50             // Header Row: Title + Platform Logo
51             Row(
52                 verticalAlignment = Alignment.CenterVertically,
53                 horizontalArrangement = Arrangement.SpaceBetween,
54                 modifier = Modifier.fillMaxWidth()
55             ) {
56                 Text(
57                     text = platformName,
58                     style = MaterialTheme.typography.titleMedium.copy(
59                         fontWeight = FontWeight.SemiBold,
60                         fontSize = 14.sp
61                     ),
62                     color = Color.White
63                 )
64
65                 // Logo/Emoji container (rounded-lg 28.dp with 20% opacity brand color)
66                 Box(
67                     modifier = Modifier
68                         .size(28.dp)
69                         .clip(RoundedCornerShape(8.dp))
70                         .background(platformColor.copy(alpha = 0.2f)),
71                     contentAlignment = Alignment.Center
72                 ) {
73                     Text(
```

```

74             text = platformEmoji(platformName),
75             fontSize = 14.sp
76         )
77     }
78 }

```

A GlassCard wrapper showing one platform's day: header row with the platform name and an emoji chip on a 20%-alpha brand-color background.

```

79
80 // Metrics block: Stacked rows matching Screen 2 layout
81 Column(
82     verticalArrangement = Arrangement.spacedBy(8.dp),
83     modifier = Modifier.fillMaxWidth()
84 ) {
85     // Line 1: Reels/Shorts Count with Movie Icon
86     Row(
87         verticalAlignment = Alignment.CenterVertically,
88         horizontalArrangement = Arrangement.spacedBy(6.dp),
89         modifier = Modifier.fillMaxWidth()
90     ) {
91         Icon(
92             imageVector = Icons.Outlined.Movie,
93             contentDescription = "Reel count icon",
94             tint = Color.White.copy(alpha = 0.7f),
95             modifier = Modifier.size(14.dp)
96         )
97         Text(
98             text = "$reelCount / $reelLimit",
99             style = MaterialTheme.typography.bodyMedium.copy(
100                 fontSize = 12.sp,
101                 fontWeight = FontWeight.Normal
102             ),
103             color = Color.White.copy(alpha = 0.7f)
104         )
105     }
106
107     // Line 2: Time Watched with Clock Icon
108     Row(
109         verticalAlignment = Alignment.CenterVertically,
110         horizontalArrangement = Arrangement.spacedBy(6.dp),
111         modifier = Modifier.fillMaxWidth()
112     ) {
113         Icon(
114             imageVector = Icons.Outlined.AccessTime,
115             contentDescription = "Time icon",
116             tint = Color.White.copy(alpha = 0.7f),
117             modifier = Modifier.size(14.dp)
118         )
119         Text(
120             text = "${minutes}m / ${minuteLimit}m",
121             style = MaterialTheme.typography.bodyMedium.copy(
122                 fontSize = 12.sp,
123                 fontWeight = FontWeight.Normal
124             ),
125             color = Color.White.copy(alpha = 0.7f)
126         )
127     }
128 }
129 }
130 }
131 }
132
133 private fun platformEmoji(name: String): String = when {
134     name.contains("Instagram", ignoreCase = true) -> "📺"
135     name.contains("YouTube", ignoreCase = true) -> "📺"
136     name.contains("TikTok", ignoreCase = true) -> "📺"
137     name.contains("Snapchat", ignoreCase = true) -> "📺"
138     else -> "📺"
139 }

```

Two metric rows — 'reels / limit' with a movie icon and 'minutes / limit' with a clock icon — plus the emoji lookup by name. Not used by the current warm-design screens; the Dashboard's platform pill replaced it.

StreakCalendar.kt — legacy GitHub-style heatmap

.../ui/components/StreakCalendar.kt · 106 lines · Composable (legacy) · 106 lines

```

1 package com.example.brainrottracker.ui.components
2
3 import androidx.compose.foundation.Canvas

```

```

4 import androidx.compose.foundation.layout.Column
5 import androidx.compose.foundation.layout.Row
6 import androidx.compose.foundation.layout.Spacer
7 import androidx.compose.foundation.layout.height
8 import androidx.compose.foundation.layout.padding
9 import androidx.compose.foundation.layout.width
10 import androidx.compose.material3.MaterialTheme
11 import androidx.compose.material3.Text
12 import androidx.compose.runtime.Composable
13 import androidx.compose.ui.Alignment
14 import androidx.compose.ui.Modifier
15 import androidx.compose.ui.geometry.CornerRadius
16 import androidx.compose.ui.geometry.Offset
17 import androidx.compose.ui.geometry.Size
18 import androidx.compose.ui.graphics.Color
19 import androidx.compose.ui.unit.dp
20 import com.example.brainrottracker.theme.DangerRed
21 import com.example.brainrottracker.theme.SuccessGreen
22 import com.example.brainrottracker.theme.TextTertiary
23 import java.time.LocalDate
24
25 @Composable
26 fun StreakCalendar(
27     streakData: Map<String, Boolean>,
28     modifier: Modifier = Modifier,
29     weeks: Int = 12
30 ) {
31     val today = LocalDate.now()
32     val totalDays = weeks * 7
33     val startDate = today.minusDays(totalDays.toLong() - 1)
34     val dayLabels = listOf("M", "T", "W", "T", "F", "S", "S")
35
36     Column(modifier = modifier) {
37         Row {
38             // Day labels
39             Column(
40                 modifier = Modifier.padding(end = 4.dp)
41             ) {
42                 dayLabels.forEach { label ->
43                     Text(
44                         text = label,
45                         style = MaterialTheme.typography.labelSmall,
46                         color = TextTertiary,
47                         modifier = Modifier.height(14.dp)
48                     )
49                 }
50             }
51
52             // Grid
53             Canvas(
54                 modifier = Modifier
55                     .weight(1f)
56                     .height((7 * 14).dp)
57             ) {
58                 val cellSize = size.width / weeks
59                 val cellPadding = 2f
60                 val actualCellSize = cellSize - cellPadding * 2

```

A 12-week × 7-day grid drawn on a single Canvas (one drawRoundRect per day — far cheaper than 84 composables). Rows are weekdays (Mon first), columns are weeks ending today.

```

61
62         for (dayOffset in 0 until totalDays) {
63             val date = startDate.plusDays(dayOffset.toLong())
64             val dayOfWeek = (date.dayOfWeek.value - 1) // 0=Mon, 6=Sun
65             val weekIndex = dayOffset / 7
66
67             val dateStr = date.toString()
68             val underLimit = streakData[dateStr]
69
70             val color = when (underLimit) {
71                 true -> SuccessGreen.copy(alpha = 0.8f)
72                 false -> DangerRed.copy(alpha = 0.7f)
73                 null -> Color.White.copy(alpha = 0.05f)
74             }
75
76             drawRoundRect(
77                 color = color,
78                 topLeft = Offset(
79                     x = weekIndex * cellSize + cellPadding,
80                     y = dayOfWeek * (size.height / 7) + cellPadding
81                 ),
82                 size = Size(actualCellSize, (size.height / 7) - cellPadding * 2),
83                 cornerRadius = CornerRadius(3f, 3f)

```

```
84         )
85     }
86 }
87
88
89 Spacer(Modifier.height(8.dp))
90
91 // Legend
92 Row(verticalAlignment = Alignment.CenterVertically) {
93     Text("Under limit", style = MaterialTheme.typography.labelSmall, color = TextTertiary)
94     Spacer(Modifier.width(4.dp))
95     Canvas(Modifier.width(12.dp).height(12.dp)) {
96         drawRoundRect(SuccessGreen.copy(alpha = 0.8f), cornerRadius = CornerRadius(2f))
97     }
98     Spacer(Modifier.width(16.dp))
99     Text("Over limit", style = MaterialTheme.typography.labelSmall, color = TextTertiary)
100    Spacer(Modifier.width(4.dp))
101    Canvas(Modifier.width(12.dp).height(12.dp)) {
102        drawRoundRect(DangerRed.copy(alpha = 0.7f), cornerRadius = CornerRadius(2f))
103    }
104 }
105 }
106 }
```

Color by verdict: green = under limit, red = over, near-transparent white = no data; plus a small legend underneath. The live Streaks screen replaced this with a month-calendar of circles, but the component still compiles against the legacy palette.

PART 9

The Screens & Their ViewModels

Each screen with its ViewModel first (the data) then the composable (the pixels). By now every repository call and component these files use has been covered — this part is where it all comes together.

OnboardingScreen.kt — the permission walkthrough

.../ui/screens/onboarding/OnboardingScreen.kt · 441 lines · Screen · 441 lines

First-launch flow: four cards, one per special permission, each with a live GRANTED/PENDING badge and a deep link into the right system settings page. Notably it has no ViewModel — all state is checked directly from system APIs, because that's where the truth lives.

```
1 package com.example.brainrottracker.ui.screens.onboarding
2
3 import android.Manifest
4 import android.content.Context
5 import android.content.Intent
6 import android.os.Build
7 import android.provider.Settings
8 import androidx.activity.compose.rememberLauncherForActivityResult
9 import androidx.activity.result.contract.ActivityResultContracts
10 import androidx.compose.foundation.background
11 import androidx.compose.foundation.border
12 import androidx.compose.foundation.layout.Arrangement
13 import androidx.compose.foundation.layout.Box
14 import androidx.compose.foundation.layout.Column
15 import androidx.compose.foundation.layout.Row
16 import androidx.compose.foundation.layout.Spacer
17 import androidx.compose.foundation.layout.fillMaxSize
18 import androidx.compose.foundation.layout.fillMaxWidth
19 import androidx.compose.foundation.layout.height
20 import androidx.compose.foundation.layout.padding
21 import androidx.compose.foundation.layout.size
22 import androidx.compose.foundation.layout.width
23 import androidx.compose.foundation.rememberScrollState
24 import androidx.compose.foundation.shape.RoundedCornerShape
25 import androidx.compose.foundation.verticalScroll
26 import androidx.compose.material3.AlertDialog
27 import androidx.compose.material3.Button
28 import androidx.compose.material3.ButtonDefaults
29 import androidx.compose.material3.OutlinedButton
30 import androidx.compose.material3.Text
31 import androidx.compose.material3.TextButton
32 import androidx.compose.runtime.Composable
33 import androidx.compose.runtime.DisposableEffect
34 import androidx.compose.runtime.getValue
35 import androidx.compose.runtime.mutableStateOf
36 import androidx.compose.runtime.remember
37 import androidx.compose.runtime.saveable.rememberSaveable
38 import androidx.compose.runtime.setValue
39 import androidx.compose.ui.Alignment
40 import androidx.compose.ui.Modifier
41 import androidx.compose.ui.draw.clip
42 import androidx.compose.ui.graphics.Color
43 import androidx.compose.ui.platform.LocalContext
44 import androidx.compose.ui.platform.LocalLifecycleOwner
45 import androidx.compose.ui.text.font.FontFamily
46 import androidx.compose.ui.text.font.FontWeight
47 import androidx.compose.ui.unit.dp
48 import androidx.compose.ui.unit.sp
49 import androidx.core.app.NotificationManagerCompat
50 import androidx.lifecycle.Lifecycle
51 import androidx.lifecycle.LifecycleEventObserver
52 import com.example.brainrottracker.data.util.ScreenTimeHelper
53 import com.example.brainrottracker.theme.rememberIsDark
54 import com.example.brainrottracker.theme.WarmAccent
55 import com.example.brainrottracker.theme.WarmBackground
56 import com.example.brainrottracker.theme.WarmBorder
57 import com.example.brainrottracker.theme.WarmGrantedGreen
58 import com.example.brainrottracker.theme.WarmLightBackground
```

```

59 import com.example.brainrottracker.theme.WarmLightBorder
60 import com.example.brainrottracker.theme.WarmLightInner
61 import com.example.brainrottracker.theme.WarmLightSurface
62 import com.example.brainrottracker.theme.WarmLightText
63 import com.example.brainrottracker.theme.WarmLightTextSecondary
64 import com.example.brainrottracker.theme.WarmStepDim
65 import com.example.brainrottracker.theme.WarmSurface
66 import com.example.brainrottracker.theme.WarmText
67 import com.example.brainrottracker.theme.WarmTextSecondary
68

```

Imports. `rememberLauncherForActivityResult + ActivityResultContracts.RequestPermission` is the modern runtime-permission API; the `Warm*` palette imports drive the theming.

```

69 @Composable
70 fun OnboardingScreen(
71     onComplete: () -> Unit,
72     modifier: Modifier = Modifier
73 ) {
74     val context = LocalContext.current
75     val lifecycleOwner = LocalLifecycleOwner.current
76
77     val dark = rememberIsDark()
78     val bg = if (dark) WarmBackground else WarmLightBackground
79     val surface = if (dark) WarmSurface else WarmLightSurface
80     val cardBorder = if (dark) WarmBorder else WarmLightBorder
81     val stepDim = if (dark) WarmStepDim else WarmLightInner
82     val textPrimary = if (dark) WarmText else WarmLightText
83     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
84
85     var hasAccessibility by remember { mutableStateOf(false) }
86     var hasOverlay by remember { mutableStateOf(false) }
87     var hasNotifications by remember { mutableStateOf(false) }
88     var hasUsageStats by remember { mutableStateOf(false) }
89
90     fun checkPermissions() {
91         hasAccessibility = try {
92             val services = Settings.Secure.getString(
93                 context.contentResolver,
94                 Settings.Secure.ENABLED_ACCESSIBILITY_SERVICES
95             ) ?: ""
96             services.contains(context.packageName)
97         } catch (_: Exception) { false }
98
99         hasOverlay = Settings.canDrawOverlays(context)
100         hasNotifications = NotificationManagerCompat.from(context).areNotificationsEnabled()
101         hasUsageStats = ScreenTimeHelper.hasPermission(context)
102     }

```

Theme colors resolved once, then four booleans of permission state. `checkPermissions()` queries each authority directly: the accessibility service list from `Settings.Secure`, `Settings.canDrawOverlays()` for overlay, `NotificationManagerCompat` for notifications, and the `AppOps` check inside `ScreenTimeHelper` for usage access.

```

103
104     DisposableEffect(lifecycleOwner) {
105         val observer = LifecycleEventObserver { _, event ->
106             if (event == Lifecycle.Event.ON_RESUME) checkPermissions()
107         }
108         lifecycleOwner.lifecycle.addObserver(observer)
109         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
110     }
111
112     remember { checkPermissions(); true }

```

The refresh trick that makes badges flip 'by magic': a `DisposableEffect` registers a lifecycle observer that re-runs `checkPermissions()` on every `ON_RESUME` — i.e. exactly when the user comes back from the Settings app. The `remember {}` on line 112 runs the first check during initial composition.

```

113
114     // On API 33+ notifications need an actual runtime permission grant, not just a settings page.
115     val notificationLauncher = rememberLauncherForActivityResult(
116         ActivityResultContracts.RequestPermission()
117     ) { checkPermissions() }
118     var notificationRequestAttempted by rememberSaveable { mutableStateOf(false) }

```

Notification specifics: API 33+ requires a real runtime grant. The launcher fires the system dialog and re-checks on the result; `notificationRequestAttempted` is `rememberSaveable` so a rotation doesn't forget that the dialog was already tried.

```

119 // Play policy: prominent disclosure + consent BEFORE sending the user to enable
120 // the accessibility service.
121
122 val sharedPrefs = remember { context.getSharedPreferences("brainrot_prefs", Context.MODE_PRIVATE) }
123 var showAccessibilityDisclosure by remember { mutableStateOf(false) }
124
125 if (showAccessibilityDisclosure) {
126     AlertDialog(
127         onDismissRequest = { showAccessibilityDisclosure = false },
128         title = { Text("How Accessibility is used") },
129         text = {
130             Text(
131                 "BrainRot Tracker uses Android's Accessibility API to read screen content " +
132                 "from Instagram, YouTube, TikTok and Snapchat only, in order to count the " +
133                 "short videos you watch.\n\n" +
134                 "This data is processed and stored on your device. If you later sign in and " +
135                 "enable cloud backup, only your daily totals are uploaded to your private " +
136                 "account. Nothing is ever shared with third parties."
137             )
138         },
139         confirmButton = {
140             TextButton(onClick = {
141                 sharedPrefs.edit().putBoolean("accessibility_disclosure_accepted", true).apply()
142                 showAccessibilityDisclosure = false
143                 context.startActivity(Intent(Settings.ACTION_ACCESSIBILITY_SETTINGS))
144             }) { Text("Agree & Continue") }
145         },
146         dismissButton = {
147             TextButton(onClick = { showAccessibilityDisclosure = false }) { Text("Not now") }
148         }
149     )
150 }

```

The **prominent disclosure** dialog — a Google Play policy requirement for accessibility-API apps: before the user is sent to enable the service, the dialog states exactly what is read (the four apps' screen content), where it stays (on-device unless backup is enabled), and that nothing goes to third parties. 'Agree & Continue' persists the consent (shown once, ever) and only then opens accessibility settings.

[illegible]


```

196         .background(if (i < grantedCount) WarmGrantedGreen else stepDim)
197     )
198 }
199 }
200 Spacer(Modifier.height(8.dp))
201 Text(
202     text = "$grantedCount OF 4 GRANTED",
203     fontSize = 11.sp,
204     fontWeight = FontWeight.Medium,
205     letterSpacing = 1.5.sp,
206     color = textSecondary
207 )
208 }
209
210 Spacer(Modifier.height(32.dp))
211
212 // ■■■ Page title ████████████████████████████████████████████
213 Text(
214     text = "Grant Permissions",
215     fontFamily = FontFamily.Serif,
216     fontWeight = FontWeight.Normal,
217     fontSize = 36.sp,
218     letterSpacing = (-0.5).sp,
219     lineHeight = 41.sp,
220     color = textPrimary,
221     modifier = Modifier.fillMaxWidth()
222 )
223
224 Spacer(Modifier.height(24.dp))

```

Layout top half: the serif '* BrainRot Tracker' logo, a 4-segment progress strip that fills green as permissions are granted with an 'N OF 4 GRANTED' counter, and the big serif title.

[illegible]

```

276 notificationLauncher.launch(Manifest.permission.POST_NOTIFICATIONS)
277     } else {
278         // Repeat denials don't re-show the dialog – fall back to settings
279         val intent = Intent(Settings.ACTION_APP_NOTIFICATION_SETTINGS).apply {
280             putExtra(Settings.EXTRA_APP_PACKAGE, context.packageName)
281         }
282         context.startActivity(intent)
283     }
284     } else {
285         context.startActivity(Intent(Settings.ACTION_SETTINGS))
286     }
287 }
288 )
289 PermissionCard(
290     title = "Usage Access",
291     description = "Fetches accurate screen time directly from the system – the same data shown in Di
292     ■ gital Wellbeing.",
293     isGranted = hasUsageStats,
294     surface = surface,
295     cardBorder = cardBorder,
296     textPrimary = textPrimary,
297     textSecondary = textSecondary,
298     btnBg = bg,
299     onOpenSettings = {
300         context.startActivity(Intent(Settings.ACTION_USAGE_ACCESS_SETTINGS))
301     }
302 )
303 }

```

The four PermissionCards. Each onOpenSettings is a different lesson in Android deep-linking: accessibility → the disclosure-gated ACTION_ACCESSIBILITY_SETTINGS; overlay → ACTION_MANAGE_OVERLAY_PERMISSION with a package: URI (lands on this app's toggle directly); notifications → first tap fires the real permission dialog, later taps deep-link to the app's notification settings (Android stops showing the dialog after repeated denials); usage access → ACTION_USAGE_ACCESS_SETTINGS.

```

303
304     Spacer(Modifier.height(32.dp))
305
306     // ■■ Footer
307     // Primary CTA reflects state: a clear "Continue" once everything is
308     // granted, otherwise a muted "Continue Anyway" with Skip still offered.
309     Button(
310         onClick = onComplete,
311         modifier = Modifier
312             .fillMaxWidth()
313             .height(40.dp),
314         shape = RoundedCornerShape(8.dp),
315         colors = ButtonDefaults.buttonColors(
316             containerColor = if (allGranted) WarmAccent else stepDim,
317             contentColor = if (allGranted) Color.White else textPrimary
318         )
319     ) {
320         Text(
321             text = if (allGranted) "Continue" else "Continue Anyway",
322             fontWeight = FontWeight.Medium,
323             fontSize = 14.sp
324         )
325     }
326
327     if (!allGranted) {
328         Spacer(Modifier.height(16.dp))
329         TextButton(onClick = onComplete) {
330             Text(
331                 text = "Skip for now",
332                 color = textSecondary,
333                 fontWeight = FontWeight.Medium,
334                 fontSize = 14.sp
335             )
336         }
337     }
338 }
339 }

```

Footer logic: when everything is granted the CTA becomes a confident terracotta 'Continue'; otherwise a muted 'Continue Anyway' plus a 'Skip for now' text button — every permission is genuinely optional, the app degrades feature-by-feature.

```
340
341 @Composable
342 private fun PermissionCard(
343     title: String,
```

```

344     description: String,
345     isGranted: Boolean,
346     surface: Color,
347     cardBorder: Color,
348     textPrimary: Color,
349     textSecondary: Color,
350     btnBg: Color,
351     onOpenSettings: () -> Unit
352 ) {
353     Column(
354         modifier = Modifier
355             .fillMaxWidth()
356             .clip(RoundedCornerShape(12.dp))
357             .background(surface)
358             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
359             .padding(32.dp),
360         verticalArrangement = Arrangement.spacedBy(8.dp)
361     ) {
362         // Title row with status badge
363         Row(
364             modifier = Modifier.fillMaxWidth(),
365             horizontalArrangement = Arrangement.SpaceBetween,
366             verticalAlignment = Alignment.CenterVertically
367         ) {
368             Text(
369                 text = title,
370                 fontWeight = FontWeight.Medium,
371                 fontSize = 16.sp,
372                 lineHeight = 22.sp,
373                 color = textPrimary
374             )
375             StatusBadge(isGranted = isGranted, cardBorder = cardBorder, textSecondary = textSecondary)
376         }
377
378         // Description
379         Text(
380             text = description,
381             fontSize = 14.sp,
382             lineHeight = 22.sp,
383             color = textSecondary
384         )
385
386         // Open Settings button
387         OutlinedButton(
388             onClick = onOpenSettings,
389             modifier = Modifier
390                 .fillMaxWidth()
391                 .height(40.dp),
392             shape = RoundedCornerShape(8.dp),
393             border = androidx.compose.foundation.BorderStroke(1.dp, cardBorder),
394             colors = ButtonDefaults.outlinedButtonColors(
395                 containerColor = btnBg,
396                 contentColor = textPrimary
397             )
398         ) {
399             Text(
400                 text = "■ Open Settings",
401                 fontWeight = FontWeight.Medium,
402                 fontSize = 14.sp
403             )
404         }
405     }
406 }
407
408 @Composable
409 private fun StatusBadge(isGranted: Boolean, cardBorder: Color, textSecondary: Color) {
410     if (isGranted) {
411         Box(
412             modifier = Modifier
413                 .clip(RoundedCornerShape(50.dp))
414                 .background(WarmGrantedGreen)
415                 .padding(horizontal = 12.dp, vertical = 4.dp)
416         ) {
417             Text(
418                 text = "GRANTED",
419                 fontSize = 11.sp,
420                 fontWeight = FontWeight.Medium,
421                 letterSpacing = 1.5.sp,
422                 color = Color.White
423             )
424         }
425     } else {
426         Box(
427             modifier = Modifier
428                 .clip(RoundedCornerShape(50.dp))

```

```

429         .border(1.dp, cardBorder, RoundedCornerShape(50.dp))
430         .padding(horizontal = 12.dp, vertical = 4.dp)
431     ) {
432         Text(
433             text = "PENDING",
434             fontSize = 11.sp,
435             fontWeight = FontWeight.Medium,
436             letterSpacing = 1.5.sp,
437             color = textSecondary
438         )
439     }
440 }
441 }

```

The two private components: PermissionCard (title + badge, description, outlined '■ Open Settings' button) and StatusBadge (solid green GRANTED pill vs. outlined PENDING pill). Pure presentation — all behaviour was passed in as lambdas.

GoogleSignInViewModel.kt — Credential Manager → Firebase

.../ui/screens/signin/GoogleSignInViewModel.kt · 98 lines · ViewModel · 98 lines

Sign-in is a three-step exchange: Android's Credential Manager produces a Google ID token → Firebase Auth swaps it for a stable user id (uid) → the result is persisted to DataStore. The UI only ever sees the sealed SignInState.

```

1  package com.example.brainrottracker.ui.screens.signin
2
3  import android.app.Application
4  import android.content.Context
5  import androidx.credentials.CredentialManager
6  import androidx.credentials.CustomCredential
7  import androidx.credentials.GetCredentialRequest
8  import androidx.credentials.exceptions.GetCredentialCancellationException
9  import androidx.credentials.exceptions.GetCredentialException
10 import androidx.lifecycle.AndroidViewModel
11 import androidx.lifecycle.ViewModelScope
12 import com.example.brainrottracker.data.local.prefs.AppPreferences
13 import com.google.android.identity.googleid.GetGoogleIdOption
14 import com.google.android.identity.googleid.GoogleIdTokenCredential
15 import com.google.firebase.FirebaseApp
16 import com.google.firebase.auth.FirebaseAuth
17 import com.google.firebase.auth.GoogleAuthProvider
18 import kotlinx.coroutines.flow.MutableStateFlow
19 import kotlinx.coroutines.flow.StateFlow
20 import kotlinx.coroutines.launch
21 import kotlinx.coroutines.tasks.await
22
23 sealed class SignInState {
24     object Idle : SignInState()
25     object Loading : SignInState()
26     data class Success(val name: String, val email: String) : SignInState()
27     data class Error(val message: String) : SignInState()
28     object Cancelled : SignInState()
29 }

```

Imports + the sealed state machine: Idle, Loading, Success(name,email), Error(message), Cancelled. A **sealed class** means the UI's when-expression provably covers every case.

```

30
31 class GoogleSignInViewModel(application: Application) : AndroidViewModel(application) {
32
33     private val _state = MutableStateFlow<SignInState>(SignInState.Idle)
34     val state: StateFlow<SignInState> = _state
35
36     fun signIn(activityContext: Context, webClientId: String) {
37         _state.value = SignInState.Loading
38         viewModelScope.launch {
39             try {
40                 val credentialManager = CredentialManager.create(activityContext)
41
42                 val googleIdOption = GetGoogleIdOption.Builder()
43                     .setFilterByAuthorizedAccounts(false)
44                     .setServerClientId(webClientId)
45                     .setAutoSelectEnabled(false)
46                     .build()
47
48                 val request = GetCredentialRequest.Builder()

```

```

49         .addCredentialOption(googleIdOption)
50         .build()
51
52     val result = credentialManager.getCredential(
53         context = activityContext,
54         request = request
55     )

```

signIn() flips to Loading and launches a coroutine. GetGoogleIdOption configures the sheet: setFilterByAuthorizedAccounts(false) shows *all* Google accounts on the device (not just previously-used ones), and the serverClientId is the Web client ID from strings.xml — the OAuth audience the token is minted for. credentialManager.getCredential() suspends while the system bottom-sheet is up.

```

56
57     val credential = result.credential
58     if (credential is CustomCredential &&
59         credential.type == GoogleIdTokenCredential.TYPE_GOOGLE_ID_TOKEN_CREDENTIAL
60     ) {
61         val googleCred = GoogleIdTokenCredential.createFrom(credential.data)
62         // Exchange the Google ID token for a Firebase user so Firestore rules
63         // can scope data to the account. Skipped when Firebase isn't configured
64         // (no google-services.json) – sign-in then remains local-only.
65         if (FirebaseApp.getApps(activityContext).isNotEmpty()) {
66             val firebaseCred = GoogleAuthProvider.getCredential(googleCred.idToken, null)
67             FirebaseAuth.getInstance().signInWithCredential(firebaseCred).await()
68         }
69         AppPreferences.setSignedIn(
70             context = activityContext,
71             email = googleCred.id,
72             name = googleCred.displayName ?: "",
73             photoUrl = googleCred.profilePictureUri?.toString()
74         )
75         _state.value = SignInState.Success(
76             name = googleCred.displayName ?: "",
77             email = googleCred.id
78         )
79     } else {
80         _state.value = SignInState.Error("Unexpected credential type")
81     }

```

Result handling: verify it's actually a Google ID credential, then the Firebase exchange — guarded by FirebaseApp.getApps(), so without google-services.json sign-in still works *locally* (name/email stored, no cloud). .await() bridges the Play Services Task API into coroutines. On success, persist to AppPreferences and emit Success.

```

82     } catch (e: GetCredentialCancellationException) {
83         _state.value = SignInState.Cancelled
84     } catch (e: GetCredentialException) {
85         _state.value = SignInState.Error(e.message ?: "Sign-in failed. Try again.")
86     } catch (e: Exception) {
87         // e.g. the Firebase credential exchange failed
88         _state.value = SignInState.Error(e.message ?: "Sign-in failed. Try again.")
89     }
90 }
91
92
93 fun resetError() {
94     if (_state.value is SignInState.Error || _state.value is SignInState.Cancelled) {
95         _state.value = SignInState.Idle
96     }
97 }
98 }

```

Three-tier error mapping: user closed the sheet → Cancelled (not an error!), credential API failures → Error with the message, anything else (e.g. the Firebase exchange) → generic Error. resetError() lets the screen return the machine to Idle.

GoogleSignInScreen.kt

.../ui/screens/signin/GoogleSignInScreen.kt · 225 lines · Screen · 225 lines

```

1 package com.example.brainrottracker.ui.screens.signin
2
3 import androidx.compose.foundation.background
4 import androidx.compose.foundation.border
5 import androidx.compose.foundation.layout.Box
6 import androidx.compose.foundation.layout.Column
7 import androidx.compose.foundation.layout.Row
8 import androidx.compose.foundation.layout.Spacer

```

```

9  import androidx.compose.foundation.layout.fillMaxSize
10 import androidx.compose.foundation.layout.fillMaxWidth
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.foundation.layout.size
14 import androidx.compose.foundation.layout.width
15 import androidx.compose.foundation.shape.RoundedCornerShape
16 import androidx.compose.material3.AlertDialog
17 import androidx.compose.material3.Button
18 import androidx.compose.material3.ButtonDefaults
19 import androidx.compose.material3.CircularProgressIndicator
20 import androidx.compose.material3.Text
21 import androidx.compose.material3.TextButton
22 import androidx.compose.runtime.Composable
23 import androidx.compose.runtime.LaunchedEffect
24 import androidx.compose.runtime.collectAsState
25 import androidx.compose.runtime.getValue
26 import androidx.compose.runtime.mutableStateOf
27 import androidx.compose.runtime.remember
28 import androidx.compose.runtime.setValue
29 import androidx.compose.ui.Alignment
30 import androidx.compose.ui.Modifier
31 import androidx.compose.ui.draw.clip
32 import androidx.compose.ui.graphics.Color
33 import androidx.compose.ui.platform.LocalContext
34 import androidx.compose.ui.res.stringResource
35 import androidx.compose.ui.text.font.FontFamily
36 import androidx.compose.ui.text.font.FontWeight
37 import androidx.compose.ui.unit.dp
38 import androidx.compose.ui.unit.sp
39 import androidx.lifecycle.viewmodel.compose.viewModel
40 import com.example.brainrottracker.R
41 import com.example.brainrottracker.theme.WarmAccent
42 import com.example.brainrottracker.theme.WarmBackground
43 import com.example.brainrottracker.theme.WarmBorder
44 import com.example.brainrottracker.theme.WarmError
45 import com.example.brainrottracker.theme.WarmLightBackground
46 import com.example.brainrottracker.theme.WarmLightBorder
47 import com.example.brainrottracker.theme.WarmLightText
48 import com.example.brainrottracker.theme.WarmLightTextSecondary
49 import com.example.brainrottracker.theme.WarmSurface
50 import com.example.brainrottracker.theme.WarmText
51 import com.example.brainrottracker.theme.WarmTextSecondary
52 import com.example.brainrottracker.theme.rememberIsDark
53
54 @Composable
55 fun GoogleSignInScreen(
56     onSignInSuccess: () -> Unit,
57     onSkip: (() -> Unit)? = null,
58     viewModel: GoogleSignInViewModel = viewModel()
59 ) {
60     val context = LocalContext.current
61     val webClientId = stringResource(R.string.google_web_client_id)
62     val dark = rememberIsDark()
63
64     val bg = if (dark) WarmBackground else WarmLightBackground
65     val textPrimary = if (dark) WarmText else WarmLightText
66     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
67     val surface = if (dark) WarmSurface else Color(0xFFEFE9DE)
68     val border = if (dark) WarmBorder else WarmLightBorder
69
70     val state by viewModel.state.collectAsState()
71     var showSyncConsent by remember { mutableStateOf(false) }
72
73     LaunchedEffect(state) {
74         when (state) {
75             // Cloud upload of usage data needs explicit consent (Play accessibility policy)
76             is SignInState.Success -> showSyncConsent = true
77             is SignInState.Cancelled -> viewModel.resetError()
78             else -> Unit
79         }
80     }

```

Setup: theme colors, the web client ID from resources, the ViewModel state, and a LaunchedEffect that reacts to state changes — Success opens the consent dialog (not straight back to the app!), Cancelled quietly resets to Idle.

```

81
82     if (showSyncConsent) {
83         val prefs = remember {
84             context.getSharedPreferences("brainrot_prefs", android.content.Context.MODE_PRIVATE)
85         }
86         AlertDialog(
87             onDismissRequest = { },
88             title = { Text("Back up your stats?") },

```

```

89         text = {
90             Text(
91                 "Upload your daily usage totals (video counts, screen time, streaks) to " +
92                 "your private account for backup and cross-device access?\n\n" +
93                 "Only daily totals are uploaded – never screen content. You can turn " +
94                 "this off anytime in Settings."
95             )
96         },
97         confirmButton = {
98             TextButton(onClick = {
99                 prefs.edit().putBoolean("cloud_sync_enabled", true).apply()
100                 showSyncConsent = false
101                 onSignInSuccess()
102             }) { Text("Enable backup") }
103         },
104         dismissButton = {
105             TextButton(onClick = {
106                 prefs.edit().putBoolean("cloud_sync_enabled", false).apply()
107                 showSyncConsent = false
108                 onSignInSuccess()
109             }) { Text("Not now") }
110         }
111     )
112 }

```

The **cloud-backup consent dialog**, the second half of the Play-policy story: sign-in alone does NOT enable uploads. The dialog spells out what would be uploaded (daily totals, never screen content); both buttons write `cloud_sync_enabled` explicitly (true/false) and then leave the screen — declining backup still completes sign-in. `onDismissRequest` is empty: the user must make an explicit choice.

```

113
114     Column(
115         modifier = Modifier
116             .fillMaxSize()
117             .background(bg)
118             .padding(horizontal = 32.dp)
119             .padding(top = 96.dp, bottom = 48.dp),
120         horizontalAlignment = Alignment.Start
121     ) {
122         // Logo
123         Row(verticalAlignment = Alignment.CenterVertically) {
124             Text(text = "*", fontSize = 22.sp, color = textPrimary)
125             Spacer(Modifier.width(8.dp))
126             Text(
127                 text = "BrainRot Tracker",
128                 fontFamily = FontFamily.Serif,
129                 fontWeight = FontWeight.Normal,
130                 fontSize = 28.sp,
131                 letterSpacing = (-0.3).sp,
132                 color = textPrimary
133             )
134         }
135
136         Spacer(Modifier.height(48.dp))
137
138         Text(
139             text = "Sign In",
140             fontFamily = FontFamily.Serif,
141             fontWeight = FontWeight.Normal,
142             fontSize = 36.sp,
143             letterSpacing = (-0.5).sp,
144             color = textPrimary
145         )
146
147         Spacer(Modifier.height(12.dp))
148
149         Text(
150             text = "Track your scroll habits and protect your focus.",
151             fontSize = 15.sp,
152             color = textSecondary,
153             lineHeight = 22.sp
154         )
155
156         Spacer(Modifier.height(48.dp))

```

Static layout: serif logo, 'Sign In' title, one-line value proposition.

```

157
158     when (state) {
159         is SignInState.Loading -> {
160             Box(
161                 modifier = Modifier

```

```

162         .fillMaxWidth()
163         .height(52.dp),
164         contentAlignment = Alignment.Center
165     ) {
166         CircularProgressIndicator(
167             color = WarmAccent,
168             modifier = Modifier.size(28.dp),
169             strokeWidth = 2.5.dp
170         )
171     }
172 }
173 else -> {
174     Button(
175         onClick = { viewModel.signIn(context, webClientId) },
176         modifier = Modifier
177             .fillMaxWidth()
178             .height(52.dp)
179             .clip(RoundedCornerShape(12.dp))
180             .border(1.dp, border, RoundedCornerShape(12.dp)),
181         colors = ButtonDefaults.buttonColors(
182             containerColor = surface,
183             contentColor = textPrimary
184         ),
185         shape = RoundedCornerShape(12.dp)
186     ) {
187         Text(
188             text = "G",
189             fontSize = 18.sp,
190             fontWeight = FontWeight.Bold,
191             color = Color(0xFF4285F4)
192         )
193         Spacer(Modifier.width(12.dp))
194         Text(
195             text = "Continue with Google",
196             fontSize = 15.sp,
197             fontWeight = FontWeight.Medium,
198             color = textPrimary
199         )
200     }
201 }
202 }
203
204 if (state is SignInState.Error) {
205     Spacer(Modifier.height(16.dp))
206     Text(
207         text = (state as SignInState.Error).message,
208         fontSize = 13.sp,
209         color = WarmError
210     )
211 }
212
213 if (onSkip != null) {
214     Spacer(Modifier.height(16.dp))
215     TextButton(onClick = onSkip, modifier = Modifier.fillMaxWidth()) {
216         Text(
217             text = "Skip for now",
218             color = textSecondary,
219             fontWeight = FontWeight.Medium,
220             fontSize = 14.sp
221         )
222     }
223 }
224 }
225 }

```

The state-driven bottom: Loading swaps the button for a spinner (the button can't be double-tapped because it no longer exists); otherwise the 'Continue with Google' button — a blue 'G' + label on a surface-colored rounded button. Errors render in the warm error red below; the optional 'Skip for now' appears only when the caller supplied an `onSkip` (it's offered post-onboarding, where skipping must be easy).

DashboardViewModel.kt — combining live flows

[../ui/screens/dashboard/DashboardViewModel.kt](#) · 148 lines · [ViewModel](#) · 148 lines

The best file for learning the app's reactive pattern: every number on the Dashboard is a `StateFlow` built by transforming and combining repository flows, so the screen is a pure function of this `ViewModel`'s outputs.

```

1 package com.example.brainrottracker.ui.screens.dashboard
2

```



```

3 import android.app.Application
4 import androidx.lifecycle.AndroidViewModel
5 import androidx.lifecycle.ViewModelScope
6 import com.example.brainrottracker.data.local.db.AppDatabase
7 import com.example.brainrottracker.data.local.db.entity.DailyLog
8 import com.example.brainrottracker.data.local.db.entity.UserLimits
9 import com.example.brainrottracker.data.model.Platform
10 import com.example.brainrottracker.data.repository.UsageRepository
11 import com.example.brainrottracker.data.util.ScreenTimeHelper
12 import com.example.brainrottracker.service.ReelCounterService
13 import kotlinx.coroutines.Dispatchers
14 import kotlinx.coroutines.delay
15 import kotlinx.coroutines.flow.Flow
16 import kotlinx.coroutines.flow.SharingStarted
17 import kotlinx.coroutines.flow.StateFlow
18 import kotlinx.coroutines.flow.combine
19 import kotlinx.coroutines.flow.flow
20 import kotlinx.coroutines.flow.flowOn
21 import kotlinx.coroutines.flow.map
22 import kotlinx.coroutines.flow.stateIn
23 import java.time.LocalDate
24 import kotlin.math.abs
25
26 class DashboardViewModel(application: Application) : AndroidViewModel(application) {
27
28     private val repository: UsageRepository
29     private val app = application
30
31     init {
32         val db = AppDatabase.getInstance(application)
33         repository = UsageRepository(db)
34     }

```

AndroidViewModel (a ViewModel that gets the Application context — needed for UsageStatsManager); the repository is constructed directly in init, no DI framework.

```

35
36     val todayLog: StateFlow<DailyLog?> = repository.getTodayLog()
37         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), null)
38
39     val limits: StateFlow<List<UserLimits>> = repository.getLimits()
40         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
41
42     // Live UsageStats for today – refreshes every 30 seconds
43     private val todayScreenTime: Flow<Map<Platform, Int>> = flow {
44         while (true) {
45             emit(ScreenTimeHelper.getTodayMinutesByPlatform(app))
46             delay(30_000)
47         }
48     }.flowOn(Dispatchers.IO)
49
50     // Live UsageStats for yesterday (one-shot per subscription)
51     private val yesterdayScreenTime: Flow<Map<Platform, Int>> = flow {
52         emit(ScreenTimeHelper.getYesterdayMinutesByPlatform(app))
53     }.flowOn(Dispatchers.IO)
54
55     /** Total foreground minutes across all platforms today (from system UsageStats). */
56     val screenTimeToday: StateFlow<Int> = todayScreenTime
57         .map { it.values.sum() }
58         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)

```

The recurring pattern: repository Flow → **stateIn(viewModelScope, WhileSubscribed(5000), initial)**. stateIn turns a cold flow into a hot StateFlow the UI can collect; WhileSubscribed(5000) keeps it warm for 5 s after the last subscriber leaves — surviving rotations without re-querying, stopping fully when the screen is really gone. Screen time has no change-notification API, so todayScreenTime is a polling flow: emit, sleep 30 s, repeat — on the IO dispatcher.

```

59
60     val brainHealth: StateFlow<Int> = combine(
61         repository.getTodayLog(),
62         limits,
63         todayScreenTime
64     ) { log, currentLimits, minuteMap ->
65         if (log != null) repository.calculateBrainHealth(log, currentLimits, minuteMap)
66         else 100
67     }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 100)
68
69     val isTrackingActive: Boolean
70         get() = ReelCounterService.isRunning
71
72     val yesterdayLog: StateFlow<DailyLog?> =
73         repository.getLogForDate(LocalDate.now().minusDays(1).toString())
74         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), null)

```

```

75
76     val currentStreak: StateFlow<Int> = repository.getStreakData()
77       .map { records -> repository.calculateCurrentStreak(records) }
78       .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), 0)

```

brainHealth **combines** three sources (today's log, limits, live minutes) and recomputes the score whenever *any* of them changes — count a reel, move a slider, or just let minutes tick: the mascot reacts. isTrackingActive peeks at the service's static flag; currentStreak maps the streak records through the repository's pure helper.

```

79
80     val insights: StateFlow<List<String>> = combine(
81       todayLog,
82       limits,
83       currentStreak,
84       todayScreenTime,
85       yesterdayScreenTime
86     ) { args ->
87       @Suppress("UNCHECKED_CAST")
88       val today = args[0] as DailyLog?
89       @Suppress("UNCHECKED_CAST")
90       val currentLimits = args[1] as List<UserLimits>
91       val streak = args[2] as Int
92       @Suppress("UNCHECKED_CAST")
93       val todayMins = (args[3] as Map<Platform, Int>).values.sum()
94       @Suppress("UNCHECKED_CAST")
95       val yesterdayMins = (args[4] as Map<Platform, Int>).values.sum()
96
97       val list = mutableListOf<String>()
98
99       if (streak > 0) {
100         list.add("■ You are on a $streak-day streak! Keep protecting your focus.")
101       } else {
102         list.add("■ Start a new focus streak today by staying under your limits.")
103       }
104
105       val totalLimit = if (currentLimits.isNotEmpty()) currentLimits.sumOf { it.dailyReelLimit } else 120
106       val todayReels = today?.getTotalReels() ?: 0
107       if (todayReels < totalLimit) {
108         list.add("■ You avoided ${totalLimit - todayReels} reels/videos today.")
109       } else {
110         list.add("■ Daily limits reached. Time to close the feeds and rest your eyes.")
111       }
112
113       if (yesterdayMins > 0) {
114         val diff = yesterdayMins - todayMins
115         if (diff > 0) {
116           val percent = (diff * 100) / yesterdayMins
117           list.add("■ Focus improved: $percent% less screen time than yesterday!")
118         } else if (diff < 0) {
119           val percent = (abs(diff) * 100) / yesterdayMins
120           list.add("■ Screen time is $percent% higher than yesterday. Step away when ready.")
121         } else {
122           list.add("■ Screen time matches yesterday. Try reducing it by 5 minutes!")
123         }
124       } else if (todayMins > 0) {
125         list.add("■ Building healthy habits today. You've logged $todayMins minutes.")
126       } else {
127         list.add("■ Pure focus today. Zero minutes of short-form content watched.")
128       }
129
130       val health = if (today != null) repository.calculateBrainHealth(today, currentLimits) else 100
131       if (health >= 90) list.add("■ Your brain health is elite! Your concentration is razor sharp.")
132       else if (health >= 75) list.add("■ You're keeping your mind clean and mindful today.")
133
134       list
135     }.stateIn(
136       viewModelScope,
137       SharingStarted.WhileSubscribed(5000),
138       listOf("■ Start your wellness journey today.")
139     )

```

The insights generator: one combine over five flows (the 5-flow overload passes an Array, hence the casts). It builds 3–4 sentences: streak status, reels avoided vs. total limit (default 120 when no limits saved), a screen-time comparison against yesterday in percent, and a health compliment ≥ 75 . The screen shows one at a time and cycles on tap.

```

140
141     fun getMotivationalMessage(score: Int): String = when {
142       score >= 90 -> "■ Elite Focus. Your brain is thriving!"
143       score >= 75 -> "■ Healthy Mind. Stay mindful."
144       score >= 50 -> "■ Moderate Usage. Take a brief break."
145       score >= 25 -> "■ High Brainrot. Your mind needs rest."
146       else -> "■ Critical Damage. Close those apps now!"

```

```

147     }
148 }

```

getMotivationalMessage() — the tier copy for the score circle, same thresholds as everywhere (90/75/50/25).

DashboardScreen.kt — the home screen

.../ui/screens/dashboard/DashboardScreen.kt · 599 lines · Screen · 599 lines

Top to bottom: header with tracking badge → (conditional) usage-access banner → mascot hero → health score ring → tappable insight card → two stat cards. Everything scrolls in one LazyColumn.

```

1  package com.example.brainrottracker.ui.screens.dashboard
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.background
5  import androidx.compose.foundation.border
6  import androidx.compose.foundation.clickable
7  import androidx.compose.foundation.layout.Arrangement
8  import androidx.compose.foundation.layout.Box
9  import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.fillMaxHeight
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.fillMaxWidth
15 import androidx.compose.foundation.layout.height
16 import androidx.compose.foundation.layout.padding
17 import androidx.compose.foundation.layout.size
18 import androidx.compose.foundation.layout.width
19 import androidx.compose.foundation.layout.widthIn
20 import androidx.compose.foundation.lazy.LazyColumn
21 import androidx.compose.foundation.shape.CircleShape
22 import androidx.compose.foundation.shape.RoundedCornerShape
23 import androidx.compose.material3.Text
24 import androidx.compose.runtime.Composable
25 import android.content.Intent
26 import android.provider.Settings
27 import androidx.compose.runtime.DisposableEffect
28 import androidx.compose.runtime.LaunchedEffect
29 import androidx.compose.runtime.collectAsState
30 import androidx.compose.runtime.getValue
31 import androidx.compose.runtime.mutableStateOf
32 import androidx.compose.runtime.remember
33 import androidx.compose.runtime.setValue
34 import androidx.compose.ui.Alignment
35 import androidx.compose.ui.Modifier
36 import androidx.compose.ui.draw.clip
37 import androidx.compose.ui.geometry.Offset
38 import androidx.compose.ui.geometry.Size
39 import androidx.compose.ui.graphics.Color
40 import androidx.compose.ui.graphics.StrokeCap
41 import androidx.compose.ui.graphics.drawscope.Stroke
42 import androidx.compose.ui.platform.LocalContext
43 import androidx.compose.ui.platform.LocalLifecycleOwner
44 import androidx.compose.ui.text.font.FontStyle
45 import androidx.compose.ui.text.font.FontWeight
46 import androidx.compose.ui.unit.dp
47 import androidx.compose.ui.unit.sp
48 import androidx.lifecycle.Lifecycle
49 import androidx.lifecycle.LifecycleEventObserver
50 import androidx.lifecycle.viewmodel.compose.viewModel
51 import androidx.compose.foundation.Image
52 import androidx.compose.ui.res.painterResource
53 import com.example.brainrottracker.R
54 import com.example.brainrottracker.data.local.db.entity.DailyLog
55 import com.example.brainrottracker.data.model.Platform
56 import com.example.brainrottracker.service.ReelCounterService
57 import com.example.brainrottracker.ui.components.BrainMascot
58 import com.example.brainrottracker.ui.components.PlatformLogo
59 import com.example.brainrottracker.theme.WarmAccent
60 import com.example.brainrottracker.theme.rememberIsDark
61 import com.example.brainrottracker.theme.WarmBackground
62 import com.example.brainrottracker.theme.WarmBorder
63 import com.example.brainrottracker.theme.WarmError
64 import com.example.brainrottracker.theme.WarmGrantedGreen
65 import com.example.brainrottracker.theme.WarmLightBackground
66 import com.example.brainrottracker.theme.WarmLightBorder
67 import com.example.brainrottracker.theme.WarmLightInner
68 import com.example.brainrottracker.theme.WarmLightSurface
69 import com.example.brainrottracker.theme.WarmLightText

```

```

70 import com.example.brainrottracker.theme.WarmLightTextSecondary
71 import com.example.brainrottracker.theme.WarmStepDim
72 import com.example.brainrottracker.theme.WarmSurface
73 import com.example.brainrottracker.theme.WarmText
74 import com.example.brainrottracker.theme.WarmTextSecondary
75

```

Imports — note `LocalLifecycleOwner` and `LifecycleEventObserver`, used for the resume-refresh trick below.

```

76 @Composable
77 fun DashboardScreen(
78     modifier: Modifier = Modifier,
79     viewModel: DashboardViewModel = viewModel()
80 ) {
81     val dark = rememberIsDark()
82     val bg = if (dark) WarmBackground else WarmLightBackground
83     val surface = if (dark) WarmSurface else WarmLightSurface
84     val cardBorder = if (dark) WarmBorder else WarmLightBorder
85     val trackBg = if (dark) WarmStepDim else WarmLightInner
86     val textPrimary = if (dark) WarmText else WarmLightText
87     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
88
89     val context = LocalContext.current
90     val todayLog by viewModel.todayLog.collectAsState()
91     val brainHealth by viewModel.brainHealth.collectAsState()
92     val insightsList by viewModel.insights.collectAsState()
93     val screenTimeToday by viewModel.screenTimeToday.collectAsState()
94
95     val lifecycleOwner = LocalLifecycleOwner.current
96     var isTracking by remember { mutableStateOf(ReelCounterService.isRunning) }
97     var hasUsagePermission by remember { mutableStateOf(com.example.brainrottracker.data.util.ScreenTimeHelper.h
98         asPermission(context)) }
99     DisposableEffect(lifecycleOwner) {
100         val observer = LifecycleEventObserver { _, event ->
101             if (event == Lifecycle.Event.ON_RESUME) {
102                 isTracking = ReelCounterService.isRunning
103                 hasUsagePermission = com.example.brainrottracker.data.util.ScreenTimeHelper.hasPermission(context)
104             }
105         }
106         lifecycleOwner.lifecycle.addObserver(observer)
107         onDispose { lifecycleOwner.lifecycle.removeObserver(observer) }
108     }
109

```

Setup: warm colors via `rememberIsDark()`, four `collectAsState()` subscriptions (each value recomposes only what reads it), and a `DisposableEffect` lifecycle observer that refreshes the two non-reactive facts (is the service running, is usage access granted) on every `ON_RESUME` — the same pattern Onboarding uses, here catching the user returning from Settings.

```

108
109     val totalMinutes = screenTimeToday
110     val hours = totalMinutes / 60
111     val mins = totalMinutes % 60
112     val timeLabel = if (hours > 0) "${hours}h ${mins}m" else "${mins}m"
113     val totalReels = todayLog?.getTotalReels() ?: 0
114
115     val healthLabel = when {
116         brainHealth >= 90 -> "Elite"
117         brainHealth >= 75 -> "Healthy"
118         brainHealth >= 50 -> "Tired"
119         brainHealth >= 25 -> "Overstimulated"
120         else -> "Brainrot"
121     }
122
123     val currentInsightIndex = remember { mutableStateOf(0) }
124     LaunchedEffect(insightsList) { currentInsightIndex.value = 0 }
125     val insightText = insightsList.getOrNull(currentInsightIndex.value)
126         ?: "You've been mindful today — keep the momentum going."
127

```

Derived display values: minutes → '1h 12m', the five-tier health label, and the insight cycling state (reset to the first insight whenever the list regenerates).

```

128     LazyColumn(
129         modifier = modifier
130             .fillMaxSize()
131             .background(bg),
132     ) {
133         // Header
134         item {
135             Row(
136                 modifier = Modifier

```

```

137         .fillMaxWidth()
138         .padding(horizontal = 20.dp, vertical = 24.dp),
139         horizontalArrangement = Arrangement.SpaceBetween,
140         verticalAlignment = Alignment.CenterVertically
141     ) {
142         Row(
143             verticalAlignment = Alignment.CenterVertically,
144             horizontalArrangement = Arrangement.spacedBy(8.dp)
145         ) {
146             Text("✱", color = textPrimary, fontSize = 18.sp)
147             Text(
148                 "FocusCenter",
149                 fontWeight = FontWeight.Medium,
150                 color = textPrimary,
151                 fontSize = 16.sp,
152                 letterSpacing = (-0.3).sp
153             )
154         }
155         val badgeBg = if (isTracking) WarmAccent else trackBg
156         val badgeContent = if (isTracking) Color.White else textSecondary
157         Box(
158             modifier = Modifier
159                 .clip(CircleShape)
160                 .background(badgeBg)
161                 .padding(horizontal = 12.dp, vertical = 4.dp)
162         ) {
163             Row(
164                 verticalAlignment = Alignment.CenterVertically,
165                 horizontalArrangement = Arrangement.spacedBy(6.dp)
166             ) {
167                 Box(
168                     modifier = Modifier
169                         .size(6.dp)
170                         .clip(CircleShape)
171                         .background(badgeContent)
172                 )
173                 Text(
174                     if (isTracking) "TRACKING ON" else "TRACKING OFF",
175                     color = badgeContent,
176                     fontSize = 11.sp,
177                     fontWeight = FontWeight.Medium,
178                     letterSpacing = 1.5.sp
179                 )
180             }
181         }
182     }
183 }

```

Header item: '✱ FocusCenter' wordmark and the tracking badge — terracotta with a white dot when the accessibility service runs, muted otherwise. The badge is the user's at-a-glance health check that counting is actually on.

```

184
185 // Usage Access banner — shown only when permission is missing
186 if (!hasUsagePermission) {
187     item {
188         androidx.compose.foundation.layout.Row(
189             modifier = Modifier
190                 .fillMaxWidth()
191                 .padding(horizontal = 20.dp)
192                 .padding(bottom = 12.dp)
193                 .clip(RoundedCornerShape(10.dp))
194                 .background(trackBg)
195                 .clickable {
196                     context.startActivity(Intent(Settings.ACTION_USAGE_ACCESS_SETTINGS))
197                 }
198                 .padding(horizontal = 16.dp, vertical = 12.dp),
199             horizontalArrangement = Arrangement.SpaceBetween,
200             verticalAlignment = Alignment.CenterVertically
201         ) {
202             androidx.compose.foundation.layout.Column(modifier = Modifier.weight(1f)) {
203                 Text(
204                     "Grant Usage Access",
205                     fontWeight = FontWeight.Medium,
206                     color = textPrimary,
207                     fontSize = 14.sp
208                 )
209                 Text(
210                     "Required to show screen time",
211                     color = textSecondary,
212                     fontSize = 12.sp
213                 )
214             }
215             Text("→", color = WarmAccent, fontSize = 18.sp, fontWeight = FontWeight.Bold)
216         }
217     }
218 }

```

```

217     }
218 }

```

The Usage Access banner, rendered *only* while the permission is missing: explains what's missing and deep-links to ACTION_USAGE_ACCESS_SETTINGS. Combined with the ON_RESUME refresh, it deletes itself the moment the user grants access.

```

219
220 // Mascot hero – the brain degrades visually as today's usage climbs
221 item {
222     MascotHero(
223         health = brainHealth,
224         totalReels = totalReels,
225         todayLog = todayLog,
226         surface = surface,
227         trackBg = trackBg,
228         textPrimary = textPrimary,
229         textSecondary = textSecondary,
230         dark = dark
231     )
232 }

```

Mascot hero item — delegated to MascotHero below.

```

233
234 // Score circle with a progress ring colored by brain-health tier
235 item {
236     val healthColor = when {
237         brainHealth >= 75 -> WarmGrantedGreen
238         brainHealth >= 40 -> WarmAccent
239         else -> WarmError
240     }
241     Box(
242         modifier = Modifier
243             .fillMaxWidth()
244             .padding(bottom = 24.dp),
245         contentAlignment = Alignment.Center
246     ) {
247         Box(
248             modifier = Modifier
249                 .size(180.dp)
250                 .clip(CircleShape)
251                 .background(surface)
252                 .then(
253                     if (!dark) Modifier.border(1.dp, cardBorder, CircleShape)
254                     else Modifier
255                 ),
256             contentAlignment = Alignment.Center
257         ) {
258             Canvas(modifier = Modifier.size(180.dp)) {
259                 val stroke = 10.dp.toPx()
260                 val inset = stroke / 2f + 6.dp.toPx()
261                 val topLeft = Offset(inset, inset)
262                 val arcSize = Size(size.width - inset * 2f, size.height - inset * 2f)
263                 // Track
264                 drawArc(
265                     color = trackBg,
266                     startAngle = 0f,
267                     sweepAngle = 360f,
268                     useCenter = false,
269                     style = Stroke(width = stroke),
270                     topLeft = topLeft,
271                     size = arcSize
272                 )
273                 // Progress
274                 if (brainHealth > 0) {
275                     drawArc(
276                         color = healthColor,
277                         startAngle = -90f,
278                         sweepAngle = 360f * brainHealth / 100f,
279                         useCenter = false,
280                         style = Stroke(width = stroke, cap = StrokeCap.Round),
281                         topLeft = topLeft,
282                         size = arcSize
283                     )
284                 }
285             }
286             Box(
287                 modifier = Modifier
288                     .size(140.dp)
289                     .clip(CircleShape)
290                     .background(if (dark) WarmBackground else WarmLightBorder),
291                 contentAlignment = Alignment.Center

```

```

292         ) {
293             Column(horizontalAlignment = Alignment.CenterHorizontally) {
294                 Text(
295                     "$brainHealth",
296                     fontWeight = FontWeight.Medium,
297                     color = healthColor,
298                     fontSize = 36.sp,
299                     lineHeight = 40.sp,
300                     letterSpacing = (-0.5).sp
301                 )
302                 Text(
303                     healthLabel.uppercase(),
304                     fontWeight = FontWeight.Medium,
305                     color = textSecondary,
306                     fontSize = 10.sp,
307                     letterSpacing = 1.5.sp
308                 )
309             }
310         }
311     }
312 }
313

```

The score ring: a Canvas drawing a full-circle track plus a progress arc sweeping $360^\circ \times \text{health}/100$ from 12 o'clock, colored by tier (green ≥ 75 , terracotta ≥ 40 , red below). Inside sits a 140 dp inner disc with the big number and tier label — layered Boxes with CircleShape clips, no custom view needed.

```

314
315 // Insight card – tap to cycle through all of today's insights
316 item {
317     val hasMultiple = insightsList.size > 1
318     Box(
319         modifier = Modifier
320             .fillMaxWidth()
321             .padding(horizontal = 20.dp)
322             .padding(bottom = 16.dp)
323             .clip(RoundedCornerShape(12.dp))
324             .background(surface)
325             .then(
326                 if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(12.dp))
327                 else Modifier
328             )
329         .clickable(enabled = hasMultiple) {
330             currentInsightIndex.value =
331                 (currentInsightIndex.value + 1) % insightsList.size
332         }
333         .padding(24.dp)
334     ) {
335         Column {
336             Row(
337                 verticalAlignment = Alignment.CenterVertically,
338                 horizontalArrangement = Arrangement.spacedBy(12.dp)
339             ) {
340                 Box(
341                     modifier = Modifier
342                         .padding(top = 6.dp)
343                         .size(8.dp)
344                         .clip(CircleShape)
345                         .background(WarmAccent)
346                         .align(Alignment.Top)
347                 )
348                 Text(
349                     insightText,
350                     fontStyle = FontStyle.Italic,
351                     color = textSecondary,
352                     fontSize = 15.sp,
353                     lineHeight = 24.sp,
354                     modifier = Modifier.weight(1f)
355                 )
356                 Image(
357                     painterResource(R.drawable.char_proud),
358                     contentDescription = null,
359                     modifier = Modifier.size(56.dp)
360                 )
361             }
362             if (hasMultiple) {
363                 Spacer(Modifier.height(16.dp))
364                 Row(
365                     modifier = Modifier.fillMaxWidth(),
366                     horizontalArrangement = Arrangement.spacedBy(6.dp)
367                 ) {
368                     insightsList.indices.forEach { i ->
369                         Box(

```

```

370             modifier = Modifier
371                 .size(6.dp)
372                 .clip(CircleShape)
373                 .background(
374                     if (i == currentInsightIndex.value) WarmAccent else trackBg
375                 )
376             )
377         }
378     }
379 }
380 }
381 }
382 }

```

The insight card: italic insight text with a terracotta bullet, the proud character art, and — when several insights exist — tap-to-cycle behaviour with a dot-pager indicator (modulo arithmetic on the index).

```

383
384 // Time + Reels stats
385 item {
386     Row(
387         modifier = Modifier
388             .fillMaxWidth()
389             .padding(horizontal = 20.dp)
390             .padding(bottom = 24.dp),
391         horizontalArrangement = Arrangement.spacedBy(16.dp)
392     ) {
393         DashStatCard(
394             label = "TIME TODAY",
395             value = timeLabel,
396             modifier = Modifier.weight(1f),
397             surface = surface,
398             cardBorder = cardBorder,
399             textPrimary = textPrimary,
400             textSecondary = textSecondary,
401             dark = dark
402         )
403         DashStatCard(
404             label = "REELS WATCHED",
405             value = "$totalReels videos",
406             modifier = Modifier.weight(1f),
407             surface = surface,
408             cardBorder = cardBorder,
409             textPrimary = textPrimary,
410             textSecondary = textSecondary,
411             dark = dark
412         )
413     }
414 }
415
416 item { Spacer(Modifier.height(8.dp)) }
417 }
418 }

```

The stats row: two equal-width DashStatCards (time today / reels watched) via Modifier.weight(1f).

```

419
420 @Composable
421 private fun DashStatCard(
422     label: String,
423     value: String,
424     modifier: Modifier,
425     surface: Color,
426     cardBorder: Color,
427     textPrimary: Color,
428     textSecondary: Color,
429     dark: Boolean
430 ) {
431     Box(
432         modifier = modifier
433             .clip(RoundedCornerShape(8.dp))
434             .background(surface)
435             .then(
436                 if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(8.dp))
437                 else Modifier
438             )
439         .padding(16.dp)
440     ) {
441         Column {
442             Text(
443                 value,
444                 fontWeight = FontWeight.Medium,
445                 color = textPrimary,
446                 fontSize = 22.sp,

```



```

447         letterSpacing = (-0.3).sp,
448         lineHeight = 26.sp
449     )
450     Spacer(Modifier.height(8.dp))
451     Text(
452         label,
453         fontWeight = FontWeight.Medium,
454         color = textSecondary,
455         fontSize = 10.sp,
456         letterSpacing = 1.5.sp
457     )
458 }
459 }
460 }

```

DashStatCard — the minimal stat tile: value large on top, SMALL-CAPS-style label under it (10 sp, wide letter-spacing), bordered only in light mode where there's no tonal contrast.

```

461
462 /**
463  * The hero block: a speech bubble, the degrading [BrainMascot], the big "reels scrolled
464  * today" count, and a compact pill listing only the platforms actually used today.
465  */
466 @Composable
467 private fun MascotHero(
468     health: Int,
469     totalReels: Int,
470     todayLog: DailyLog?,
471     surface: Color,
472     trackBg: Color,
473     textPrimary: Color,
474     textSecondary: Color,
475     dark: Boolean
476 ) {
477     val message = when {
478         health >= 90 -> "Feeling razor sharp! ■"
479         health >= 75 -> "All good up here."
480         health >= 50 -> "Getting a little foggy..."
481         health >= 25 -> "Ease off the scroll."
482         else -> "The damage is real."
483     }
484
485     val usedPlatforms = Platform.entries.mapNotNull { p ->
486         val count = todayLog?.getReelsForPlatform(p) ?: 0
487         if (count > 0) p to count else null
488     }
489
490     Column(
491         modifier = Modifier
492             .fillMaxWidth()
493             .padding(horizontal = 20.dp)
494             .padding(bottom = 8.dp),
495         horizontalAlignment = Alignment.CenterHorizontally
496     ) {
497         SpeechBubble(text = message, surface = surface, textPrimary = textPrimary, dark = dark)
498
499         Spacer(Modifier.height(4.dp))
500
501         BrainMascot(
502             health = health,
503             modifier = Modifier.size(180.dp)
504         )
505
506         Spacer(Modifier.height(12.dp))
507
508         Text(
509             "$totalReels",
510             fontWeight = FontWeight.Bold,
511             color = textPrimary,
512             fontSize = 56.sp,
513             lineHeight = 60.sp,
514             letterSpacing = (-1).sp
515         )
516         Text(
517             "Reels Scrolled Today",
518             color = textSecondary,
519             fontSize = 15.sp,
520             modifier = Modifier.padding(top = 2.dp)
521         )
522
523         Spacer(Modifier.height(16.dp))
524
525         if (usedPlatforms.isEmpty()) {
526             Text(

```

```

527         "No reels yet today – nice. ■",
528         color = textSecondary,
529         fontSize = 13.sp
530     )
531 } else {
532     Row(
533         modifier = Modifier
534             .clip(CircleShape)
535             .background(trackBg)
536             .padding(horizontal = 18.dp, vertical = 10.dp),
537         verticalAlignment = Alignment.CenterVertically
538     ) {
539         usedPlatforms.forEachIndexed { index, (platform, count) ->
540             if (index > 0) {
541                 Box(
542                     modifier = Modifier
543                         .padding(horizontal = 14.dp)
544                         .size(width = 1.dp, height = 16.dp)
545                         .background(textSecondary.copy(alpha = 0.3f))
546                 )
547             }
548             Row(verticalAlignment = Alignment.CenterVertically) {
549                 PlatformLogo(platform = platform, modifier = Modifier.size(20.dp))
550                 Spacer(Modifier.width(8.dp))
551                 Text(
552                     "$count",
553                     fontWeight = FontWeight.SemiBold,
554                     color = textPrimary,
555                     fontSize = 16.sp
556                 )
557             }
558         }
559     }
560 }
561 }
562 }

```

MascotHero: speech-bubble message by health tier ('Feeling razor sharp' → 'The damage is real.'), the 180 dp BrainMascot, the 56 sp reel count, and the platform pill — only platforms used today, each as hand-drawn logo + count, separated by hairline dividers; a zen message when the day is clean.

```

563
564 /** A rounded message bubble with a downward tail, like a comic speech balloon. */
565 @Composable
566 private fun SpeechBubble(
567     text: String,
568     surface: Color,
569     textPrimary: Color,
570     dark: Boolean
571 ) {
572     val bubbleColor = if (dark) surface else Color.White
573     Column(horizontalAlignment = Alignment.CenterHorizontally) {
574         Box(
575             modifier = Modifier
576                 .widthIn(max = 280.dp)
577                 .clip(RoundedCornerShape(20.dp))
578                 .background(bubbleColor)
579                 .padding(horizontal = 20.dp, vertical = 12.dp)
580         ) {
581             Text(
582                 text,
583                 color = textPrimary,
584                 fontSize = 15.sp,
585                 fontWeight = FontWeight.Medium
586             )
587         }
588         // Tail
589         Canvas(modifier = Modifier.size(width = 18.dp, height = 9.dp)) {
590             val path = androidx.compose.ui.graphics.Path().apply {
591                 moveTo(0f, 0f)
592                 lineTo(size.width, 0f)
593                 lineTo(size.width / 2f, size.height)
594                 close()
595             }
596             drawPath(path, bubbleColor)
597         }
598     }
599 }

```

SpeechBubble: a rounded Box plus a Canvas triangle tail — the bubble color matches the surface in dark mode and pure white in light mode for contrast against the cream background.

StatsViewModel.kt — the weekly aggregate

.../ui/screens/stats/StatsViewModel.kt · 75 lines · ViewModel · 75 lines

```

1 package com.example.brainrottracker.ui.screens.stats
2
3 import android.app.Application
4 import androidx.lifecycle.AndroidViewModel
5 import androidx.lifecycle.ViewModelScope
6 import com.example.brainrottracker.data.local.db.AppDatabase
7 import com.example.brainrottracker.data.local.db.entity.DailyLog
8 import com.example.brainrottracker.data.model.Platform
9 import com.example.brainrottracker.data.repository.UsageRepository
10 import com.example.brainrottracker.data.util.ScreenTimeHelper
11 import kotlinx.coroutines.Dispatchers
12 import kotlinx.coroutines.flow.SharingStarted
13 import kotlinx.coroutines.flow.StateFlow
14 import kotlinx.coroutines.flow.combine
15 import kotlinx.coroutines.flow.flow
16 import kotlinx.coroutines.flow.flowOn
17 import kotlinx.coroutines.flow.stateIn
18 import kotlin.math.roundToInt
19
20 data class WeeklyStats(
21     val dailyLogs: List<DailyLog> = emptyList(),
22     val totalReels: Int = 0,
23     val totalMinutes: Int = 0,
24     val productivityScore: Int = 100,
25     val bestDay: DailyLog? = null,
26     val worstDay: DailyLog? = null,
27     val healthByDate: Map<String, Int> = emptyMap(),
28     val topApps: List<Pair<Platform, Int>> = emptyList(),
29 )

```

WeeklyStats is a single immutable UI-state object — the screen reads exactly one StateFlow. Sensible defaults mean the screen renders instantly before data arrives.

```

30
31 class StatsViewModel(application: Application) : AndroidViewModel(application) {
32
33     private val repository: UsageRepository
34     private val app = application
35
36     init {
37         val db = AppDatabase.getInstance(application)
38         repository = UsageRepository(db)
39     }
40
41     // Fetch week screen time from UsageStats once per subscription (no need to poll here)
42     private val weekScreenTime = flow {
43         emit(ScreenTimeHelper.getWeekMinutesByPlatform(app))
44     }.flowOn(Dispatchers.IO)
45
46     val weeklyStats: StateFlow<WeeklyStats> =
47         combine(repository.getWeeklyLogs(), repository.getLimits(), weekScreenTime) { logs, limits, weekMinutes
48             ->
49             val totalReels = logs.sumOf { it.getTotalReels() }
50             val totalMinutes = weekMinutes.values.sum()
51
52             val healthByDate = logs.associate { log ->
53                 log.date to repository.calculateBrainHealth(log, limits)
54             }
55
56             val productivityScore = if (healthByDate.isEmpty()) 100
57             else healthByDate.values.average().roundToInt().coerceIn(0, 100)
58
59             val topApps = Platform.entries
60                 .map { p -> p to logs.sumOf { it.getReelsForPlatform(p) } }
61                 .filter { it.second > 0 }
62                 .sortedByDescending { it.second }
63
64             WeeklyStats(
65                 dailyLogs = logs,
66                 totalReels = totalReels,
67                 totalMinutes = totalMinutes,
68                 productivityScore = productivityScore,
69                 bestDay = logs.minByOrNull { it.getTotalReels() },
70                 worstDay = logs.maxByOrNull { it.getTotalReels() },
71                 healthByDate = healthByDate,
72                 topApps = topApps
73             )
74         }
75 }

```

```

74         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), WeeklyStats())
75     }

```

One combine builds everything from three sources (7-day logs, limits, week screen-time — fetched once per subscription, no polling): total reels; total minutes; per-day health scores (recomputed via calculateBrainHealth — minutes aren't stored per historical day, so these scores reflect reels vs. limits); productivity = the average of those daily scores; best/worst day by reel count (min/max); and the ranked top-apps list filtered to platforms actually used. Room re-emits when any day in the window changes, so the whole screen updates live while you scroll reels.

StatsScreen.kt — gauge, stacked chart, rankings

.../ui/screens/stats/StatsScreen.kt · 606 lines · Screen · 606 lines

```

1  package com.example.brainrottracker.ui.screens.stats
2
3  import androidx.compose.foundation.Canvas
4  import androidx.compose.foundation.Image
5  import androidx.compose.foundation.background
6  import androidx.compose.foundation.border
7  import androidx.compose.foundation.layout.Arrangement
8  import androidx.compose.foundation.layout.Box
9  import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.Spacer
12 import androidx.compose.foundation.layout.fillMaxSize
13 import androidx.compose.foundation.layout.fillMaxWidth
14 import androidx.compose.foundation.layout.height
15 import androidx.compose.foundation.layout.padding
16 import androidx.compose.foundation.layout.size
17 import androidx.compose.foundation.layout.width
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.shape.CircleShape
20 import androidx.compose.foundation.shape.RoundedCornerShape
21 import androidx.compose.material3.Text
22 import androidx.compose.runtime.Composable
23 import androidx.compose.runtime.collectAsState
24 import androidx.compose.runtime.getValue
25 import androidx.compose.ui.Alignment
26 import androidx.compose.ui.Modifier
27 import androidx.compose.ui.draw.clip
28 import androidx.compose.ui.geometry.CornerRadius
29 import androidx.compose.ui.geometry.Offset
30 import androidx.compose.ui.geometry.Size
31 import androidx.compose.ui.graphics.Color
32 import androidx.compose.ui.graphics.StrokeCap
33 import androidx.compose.ui.graphics.drawscope.Stroke
34 import androidx.compose.ui.text.font.FontWeight
35 import androidx.compose.ui.text.style.TextAlign
36 import androidx.compose.ui.unit.dp
37 import androidx.compose.ui.unit.sp
38 import androidx.compose.ui.res.painterResource
39 import androidx.lifecycle.viewmodel.compose.viewModel
40 import com.example.brainrottracker.R
41 import com.example.brainrottracker.data.local.db.entity.DailyLog
42 import com.example.brainrottracker.data.model.Platform
43 import com.example.brainrottracker.ui.components.BrainMascot
44 import com.example.brainrottracker.ui.components.PlatformLogo
45 import com.example.brainrottracker.theme.StatsChartInstagram
46 import com.example.brainrottracker.theme.StatsChartSnachat
47 import com.example.brainrottracker.theme.StatsChartTikTok
48 import com.example.brainrottracker.theme.StatsChartYouTube
49 import com.example.brainrottracker.theme.WarmAccent
50 import com.example.brainrottracker.theme.rememberIsDark
51 import com.example.brainrottracker.theme.WarmBackground
52 import com.example.brainrottracker.theme.WarmBorder
53 import com.example.brainrottracker.theme.WarmError
54 import com.example.brainrottracker.theme.WarmGrantedGreen
55 import com.example.brainrottracker.theme.WarmLightBackground
56 import com.example.brainrottracker.theme.WarmLightBorder
57 import com.example.brainrottracker.theme.WarmLightSurface
58 import com.example.brainrottracker.theme.WarmLightText
59 import com.example.brainrottracker.theme.WarmLightTextSecondary
60 import com.example.brainrottracker.theme.WarmSurface
61 import com.example.brainrottracker.theme.WarmText
62 import com.example.brainrottracker.theme.WarmTextSecondary
63 import java.time.LocalDate
64 import java.time.format.TextStyle
65 import java.util.Locale
66
67 @Composable
68 fun StatsScreen(

```

```

69     modifier: Modifier = Modifier,
70     viewModel: StatsViewModel = viewModel(),
71     onClick: () -> Unit = {}
72 ) {
73     val dark = rememberIsDark()
74     val bg = if (dark) WarmBackground else WarmLightBackground
75     val surface = if (dark) WarmSurface else WarmLightSurface
76     val cardBorder = if (dark) WarmBorder else WarmLightBorder
77     val gaugeBg = if (dark) WarmBackground else WarmLightBorder
78     val textPrimary = if (dark) WarmText else WarmLightText
79     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
80
81     val stats by viewModel.weeklyStats.collectAsState()
82
83     LazyColumn(
84         modifier = modifier
85             .fillMaxSize()
86             .background(bg),
87     ) {

```

Imports + theme setup + the single weeklyStats subscription.

```

88         // Title
89         item {
90             Text(
91                 "Stats & Reports",
92                 fontWeight = FontWeight.Normal,
93                 color = textPrimary,
94                 fontSize = 36.sp,
95                 lineHeight = 41.sp,
96                 letterSpacing = (-0.5).sp,
97                 modifier = Modifier
98                     .fillMaxWidth()
99                     .padding(horizontal = 20.dp)
100                     .padding(top = 32.dp, bottom = 24.dp)
101             )
102         }
103
104         // Productivity gauge card
105         item {
106             Box(
107                 modifier = Modifier
108                     .fillMaxWidth()
109                     .padding(horizontal = 20.dp)
110                     .padding(bottom = 16.dp)
111                     .clip(RoundedCornerShape(16.dp))
112                     .background(surface)
113                     .then(
114                         if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(16.dp))
115                         else Modifier
116                     )
117                 .padding(horizontal = 24.dp, vertical = 32.dp),
118                 contentAlignment = Alignment.Center
119             ) {
120                 Row(
121                     modifier = Modifier.fillMaxWidth(),
122                     verticalAlignment = Alignment.CenterVertically
123                 ) {
124                     Column(
125                         horizontalAlignment = Alignment.CenterHorizontally,
126                         modifier = Modifier.weight(1f)
127                     ) {
128                         Box(
129                             modifier = Modifier
130                                 .size(width = 200.dp, height = 110.dp),
131                             contentAlignment = Alignment.BottomCenter
132                         ) {
133                             val score = stats.productivityScore
134                             Canvas(modifier = Modifier.fillMaxSize()) {
135                                 val strokeWidth = 28.dp.toPx()
136                                 val cx = size.width / 2f
137                                 val cy = size.height
138                                 val radius = cx - strokeWidth / 2f
139                                 val topLeft = Offset(cx - radius, cy - radius)
140                                 val arcSize = Size(radius * 2f, radius * 2f)
141
142                                 // Full semicircle track
143                                 drawArc(
144                                     color = gaugeBg,
145                                     startAngle = 180f,
146                                     sweepAngle = 180f,
147                                     useCenter = false,
148                                     style = Stroke(width = strokeWidth, cap = StrokeCap.Butt),
149                                     topLeft = topLeft,
150                                     size = arcSize

```

```

151         )
152         // Progress arc
153         if (score > 0) {
154             drawArc(
155                 color = WarmAccent,
156                 startAngle = 180f,
157                 sweepAngle = 180f * score / 100f,
158                 useCenter = false,
159                 style = Stroke(width = strokeWidth, cap = StrokeCap.Butt),
160                 topLeft = topLeft,
161                 size = arcSize
162             )
163         }
164     }
165     Text(
166         "${stats.productivityScore}",
167         fontWeight = FontWeight.Normal,
168         color = textPrimary,
169         fontSize = 36.sp,
170         lineHeight = 40.sp,
171         modifier = Modifier.padding(bottom = 4.dp)
172     )
173 }
174 Spacer(Modifier.height(12.dp))
175 Text(
176     "PRODUCTIVITY SCORE",
177     color = textSecondary,
178     fontSize = 11.sp,
179     fontWeight = FontWeight.Medium,
180     letterSpacing = 1.5.sp
181 )
182 }
183 Image(
184     painterResource(
185         if (stats.productivityScore >= 50) R.drawable.char_excited
186         else R.drawable.char_overwhelmed
187     ),
188     contentDescription = null,
189     modifier = Modifier.size(96.dp)
190 )
191 }
192 }
193 }

```

The productivity gauge: a Canvas semicircle — geometry trick: the Box is exactly half as tall as it is wide and the circle's center sits on the bottom edge, so arcs from 180° sweep along the visible top half. Track in the muted gauge color, progress sweeping 180°×score/100 in terracotta, the score number overlapping from below. The side character flips between excited and overwhelmed at score 50.

```

194
195     // Weekly bar chart card
196     item {
197         Box(
198             modifier = Modifier
199                 .fillMaxWidth()
200                 .padding(horizontal = 20.dp)
201                 .padding(bottom = 16.dp)
202                 .clip(RoundedCornerShape(12.dp))
203                 .background(surface)
204                 .then(
205                     if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(12.dp))
206                     else Modifier
207                 )
208             .padding(24.dp)
209         ) {
210             Column {
211                 Text(
212                     "This Week",
213                     fontWeight = FontWeight.Medium,
214                     color = textPrimary,
215                     fontSize = 16.sp,
216                     lineHeight = 22.sp
217                 )
218                 // Legend
219                 Row(
220                     modifier = Modifier.padding(top = 12.dp, bottom = 20.dp),
221                     horizontalArrangement = Arrangement.spacedBy(16.dp)
222                 ) {
223                     LegendDot(color = StatsChartTikTok, label = "TikTok", textColor = textSecondary)
224                     LegendDot(color = StatsChartYouTube, label = "YouTube", textColor = textSecondary)
225                     LegendDot(color = StatsChartInstagram, label = "Instagram", textColor = textSecondary)
226                     LegendDot(color = StatsChartSnapchat, label = "Snapchat", textColor = textSecondary)
227                 }

```

```

228                // Bar chart
229                WarmWeeklyBarChart(
230                    dailyLogs = stats.dailyLogs,
231                    healthByDate = stats.healthByDate,
232                    textSecondary = textSecondary,
233                    modifier = Modifier
234                        .fillMaxWidth()
235                        .height(186.dp)
236                )
237            }
238        }
239    }

```

The This Week card: title, a four-dot legend, then the custom stacked bar chart.

```

240
241    // Total videos + total time
242    item {
243        Row(
244            modifier = Modifier
245                .fillMaxWidth()
246                .padding(horizontal = 20.dp)
247                .padding(bottom = 16.dp),
248            horizontalArrangement = Arrangement.spacedBy(16.dp)
249        ) {
250            StatSummaryCard(
251                label = "TOTAL VIDEOS",
252                value = "${stats.totalReels}",
253                modifier = Modifier.weight(1f),
254                surface = surface,
255                cardBorder = cardBorder,
256                textPrimary = textPrimary,
257                textSecondary = textSecondary,
258                dark = dark
259            )
260            val h = stats.totalMinutes / 60
261            val m = stats.totalMinutes % 60
262            StatSummaryCard(
263                label = "TOTAL TIME",
264                value = if (h > 0) "${h}h ${m}m" else "${m}m",
265                modifier = Modifier.weight(1f),
266                surface = surface,
267                cardBorder = cardBorder,
268                textPrimary = textPrimary,
269                textSecondary = textSecondary,
270                dark = dark
271            )
272        }
273    }
274
275    // Best + Worst day
276    item {
277        Row(
278            modifier = Modifier
279                .fillMaxWidth()
280                .padding(horizontal = 20.dp)
281                .padding(bottom = 24.dp),
282            horizontalArrangement = Arrangement.spacedBy(16.dp)
283        ) {
284            BestWorstCard(
285                label = "Best",
286                labelColor = WarmGrantedGreen,
287                dayName = getDayName(stats.bestDay?.date),
288                count = "${stats.bestDay?.getTotalReels() ?: 0} videos",
289                charRes = R.drawable.char_proud,
290                modifier = Modifier.weight(1f),
291                surface = surface,
292                cardBorder = cardBorder,
293                textPrimary = textPrimary,
294                textSecondary = textSecondary,
295                dark = dark
296            )
297            BestWorstCard(
298                label = "Worst",
299                labelColor = WarmError,
300                dayName = getDayName(stats.worstDay?.date),
301                count = "${stats.worstDay?.getTotalReels() ?: 0} videos",
302                charRes = R.drawable.char_overwhelmed,
303                modifier = Modifier.weight(1f),
304                surface = surface,
305                cardBorder = cardBorder,
306                textPrimary = textPrimary,
307                textSecondary = textSecondary,
308                dark = dark
309            )

```

```

310     }
311 }

```

Two summary cards (total videos / total time h-m) and the Best/Worst day pair — computed by reel count, each with its character art.

```

312
313 // Top apps — only platforms actually used this week, ranked
314 if (stats.topApps.isNotEmpty()) {
315     item {
316         Text(
317             "Top apps",
318             fontWeight = FontWeight.Medium,
319             color = textPrimary,
320             fontSize = 18.sp,
321             modifier = Modifier
322                 .fillMaxWidth()
323                 .padding(horizontal = 20.dp)
324                 .padding(bottom = 12.dp)
325         )
326     }
327     item {
328         Column(
329             modifier = Modifier
330                 .fillMaxWidth()
331                 .padding(horizontal = 20.dp)
332                 .padding(bottom = 24.dp)
333                 .clip(RoundedCornerShape(12.dp))
334                 .background(surface)
335                 .then(
336                     if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(12.dp))
337                     else Modifier
338                 )
339         ) {
340             stats.topApps.forEachIndexed { index, (platform, count) ->
341                 if (index > 0) {
342                     HorizontalDividerLine(cardBorder)
343                 }
344                 Row(
345                     modifier = Modifier
346                         .fillMaxWidth()
347                         .padding(horizontal = 16.dp, vertical = 16.dp),
348                     verticalAlignment = Alignment.CenterVertically
349                 ) {
350                     PlatformLogo(platform = platform, modifier = Modifier.size(36.dp))
351                     Spacer(Modifier.width(16.dp))
352                     Text(
353                         platform.displayName,
354                         fontWeight = FontWeight.Medium,
355                         color = textPrimary,
356                         fontSize = 16.sp,
357                         modifier = Modifier.weight(1f)
358                     )
359                     Text(
360                         "$count",
361                         fontWeight = FontWeight.SemiBold,
362                         color = textPrimary,
363                         fontSize = 16.sp
364                     )
365                 }
366             }
367         }
368     }
369 }
370
371 item { Spacer(Modifier.height(8.dp)) }
372 }

```

Top apps: a ranked list card, divider-separated rows of logo + name + weekly count. Only rendered if at least one platform was used.

```

373 }
374
375 @Composable
376 private fun HorizontalDividerLine(color: Color) {
377     Box(
378         modifier = Modifier
379             .fillMaxWidth()
380             .padding(horizontal = 16.dp)
381             .height(1.dp)
382             .background(color)
383     )
384 }

```



```

385
386 @Composable
387 private fun LegendDot(color: Color, label: String, textColor: Color) {
388     Row(
389         verticalAlignment = Alignment.CenterVertically,
390         horizontalArrangement = Arrangement.spacedBy(6.dp)
391     ) {
392         Box(
393             modifier = Modifier
394                 .size(10.dp)
395                 .clip(CircleShape)
396                 .background(color)
397         )
398         Text(label, color = textColor, fontSize = 13.sp, fontWeight = FontWeight.Medium)
399     }
400 }

```

Small helpers: a manual 1 dp divider Box and the LegendDot row.

```

401
402 @Composable
403 private fun WarmWeeklyBarChart(
404     dailyLogs: List<DailyLog>,
405     healthByDate: Map<String, Int>,
406     textSecondary: Color,
407     modifier: Modifier = Modifier
408 ) {
409     val today = LocalDate.now()
410     val days = (6 downTo 0).map { today.minusDays(it.toLong()) }
411     val logMap = dailyLogs.associateBy { it.date }
412
413     Column(modifier = modifier) {
414         Canvas(
415             modifier = Modifier
416                 .fillMaxWidth()
417                 .weight(1f)
418         ) {
419             val barWidth = size.width / 7f * 0.55f
420             val slotWidth = size.width / 7f
421             val maxTotal = days.maxOrNull { day ->
422                 logMap[day.toString()]?.getTotalReels() ?: 0
423             }?.coerceAtLeast(1) ?: 1
424
425             days.forEachIndexed { index, day ->
426                 val log = logMap[day.toString()]
427                 val tiktok = log?.tiktokVideos ?: 0
428                 val youtube = log?.youtubeShorts ?: 0
429                 val instagram = log?.instagramReels ?: 0
430                 val snapchat = log?.snapchatSpotlights ?: 0
431                 val total = tiktok + youtube + instagram + snapchat
432                 if (total == 0) return@forEachIndexed
433
434                 val xCenter = index * slotWidth + slotWidth / 2f
435                 val xLeft = xCenter - barWidth / 2f
436                 val totalBarH = (total.toFloat() / maxTotal) * size.height
437                 var currentY = size.height
438
439                 // Draw segments bottom -> top: Snapchat, Instagram, YouTube, TikTok
440                 if (snapchat > 0) {
441                     val h = (snapchat.toFloat() / total) * totalBarH
442                     currentY -= h
443                     drawRect(StatsChartSnapchat, Offset(xLeft, currentY), Size(barWidth, h))
444                 }
445                 if (instagram > 0) {
446                     val h = (instagram.toFloat() / total) * totalBarH
447                     currentY -= h
448                     drawRect(StatsChartInstagram, Offset(xLeft, currentY), Size(barWidth, h))
449                 }
450                 if (youtube > 0) {
451                     val h = (youtube.toFloat() / total) * totalBarH
452                     currentY -= h
453                     drawRect(StatsChartYouTube, Offset(xLeft, currentY), Size(barWidth, h))
454                 }
455                 if (tiktok > 0) {
456                     val h = (tiktok.toFloat() / total) * totalBarH
457                     currentY -= h
458                     drawRoundRect(
459                         StatsChartTikTok,
460                         Offset(xLeft, currentY),
461                         Size(barWidth, h),
462                         CornerRadius(4f, 4f)
463                     )
464                 }
465             }
466         }
467     }
468 }

```

```

467         Spacer(Modifier.height(8.dp))
468
469         Row(modifier = Modifier.fillMaxWidth()) {
470             days.forEach { day ->
471                 Text(
472                     day.dayOfWeek.getDisplayName(TextStyle.SHORT, Locale.getDefault()),
473                     fontSize = 12.sp,
474                     fontWeight = FontWeight.Medium,
475                     color = textSecondary,
476                     textAlign = TextAlign.Center,
477                     modifier = Modifier.weight(1f)
478                 )
479             }
480         }
481     }
482
483     Spacer(Modifier.height(6.dp))
484
485     // Per-day brain mascot reflecting that day's brain-health.
486     Row(modifier = Modifier.fillMaxWidth()) {
487         days.forEach { day ->
488             val log = logMap[day.toString()]
489             Box(
490                 modifier = Modifier.weight(1f),
491                 contentAlignment = Alignment.Center
492             ) {
493                 if (log != null && log.getTotalReels() > 0) {
494                     BrainMascot(
495                         health = healthByDate[day.toString()] ?: 100,
496                         modifier = Modifier.size(30.dp)
497                     )
498                 }
499             }
500         }
501     }
502 }
503 }

```

WarmWeeklyBarChart — the most interesting custom drawing in the screens. Builds the last 7 days oldest-first, normalizes bar heights against the busiest day (`coerceAtLeast(1)` avoids $\div 0$), then for each day stacks platform segments bottom-up by walking a `currentY` cursor upward: Snapchat, Instagram, YouTube, then TikTok with rounded top corners (it's the top segment). Under the bars: weekday initials, then a row of 30 dp `BrainMascots` — each day's mascot rendered at *that day's* health, a charming way to show the week's trajectory.

```

504
505 @Composable
506 private fun StatSummaryCard(
507     label: String,
508     value: String,
509     modifier: Modifier,
510     surface: Color,
511     cardBorder: Color,
512     textPrimary: Color,
513     textSecondary: Color,
514     dark: Boolean
515 ) {
516     Box(
517         modifier = modifier
518             .clip(RoundedCornerShape(12.dp))
519             .background(surface)
520             .then(
521                 if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(12.dp))
522                 else Modifier
523             )
524         .padding(20.dp)
525     ) {
526         Column {
527             Text(
528                 label,
529                 color = textSecondary,
530                 fontSize = 10.sp,
531                 fontWeight = FontWeight.Medium,
532                 letterSpacing = 1.5.sp
533             )
534             Spacer(Modifier.height(8.dp))
535             Text(
536                 value,
537                 fontWeight = FontWeight.Normal,
538                 color = textPrimary,
539                 fontSize = 26.sp,
540                 lineHeight = 32.sp,
541                 letterSpacing = (-0.3).sp

```

```

542     )
543   }
544 }
545 }
546
547 @Composable
548 private fun BestWorstCard(
549     label: String,
550     labelColor: Color,
551     dayName: String,
552     count: String,
553     charRes: Int,
554     modifier: Modifier,
555     surface: Color,
556     cardBorder: Color,
557     textPrimary: Color,
558     textSecondary: Color,
559     dark: Boolean
560 ) {
561     Box(
562         modifier = modifier
563             .clip(RoundedCornerShape(12.dp))
564             .background(surface)
565             .then(
566                 if (!dark) Modifier.border(1.dp, cardBorder, RoundedCornerShape(12.dp))
567                 else Modifier
568             )
569         .padding(16.dp)
570     ) {
571         Row(verticalAlignment = Alignment.Bottom) {
572             Column(modifier = Modifier.weight(1f)) {
573                 Box(
574                     modifier = Modifier
575                         .clip(CircleShape)
576                         .background(labelColor)
577                         .padding(horizontal = 12.dp, vertical = 4.dp)
578                 ) {
579                     Text(
580                         label.uppercase(),
581                         color = Color.White,
582                         fontSize = 10.sp,
583                         fontWeight = FontWeight.Medium,
584                         letterSpacing = 1.5.sp
585                     )
586                 }
587                 Spacer(Modifier.height(12.dp))
588                 Text(dayName, fontWeight = FontWeight.Medium, color = textPrimary, fontSize = 16.sp, lineHeight
589                     = 22.sp)
590                 Text(count, color = textSecondary, fontSize = 14.sp, lineHeight = 22.sp)
591             }
592             Image(
593                 painterResource(charRes),
594                 contentDescription = null,
595                 modifier = Modifier.size(56.dp)
596             )
597         }
598     }
599
600 private fun getDayName(dateStr: String?): String {
601     if (dateStr == null) return "N/A"
602     return try {
603         val date = LocalDate.parse(dateStr)
604         date.dayOfWeek.getDisplayName(TextStyle.FULL, Locale.getDefault())
605     } catch (_: Exception) { "N/A" }
606 }

```

StatSummaryCard and BestWorstCard (label pill, day name, count, character) plus getDayName() which parses the ISO date and prints the full weekday name, defaulting to N/A.

LimitsViewModel.kt

.../ui/screens/limits/LimitsViewModel.kt · 47 lines · ViewModel · 47 lines

```

1 package com.example.brainrottracker.ui.screens.limits
2
3 import android.app.Application
4 import androidx.lifecycle.AndroidViewModel
5 import androidx.lifecycle.ViewModelScope
6 import com.example.brainrottracker.data.local.db.AppDatabase
7 import com.example.brainrottracker.data.local.db.entity.DailyLog
8 import com.example.brainrottracker.data.local.db.entity.UserLimits

```

```

 9 import com.example.brainrottracker.data.model.Platform
10 import com.example.brainrottracker.data.repository.UsageRepository
11 import kotlinx.coroutines.flow.SharingStarted
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.StateIn
14 import kotlinx.coroutines.launch
15
16 class LimitsViewModel(application: Application) : AndroidViewModel(application) {
17
18     private val repository: UsageRepository
19
20     init {
21         val db = AppDatabase.getInstance(application)
22         repository = UsageRepository(db)
23     }
24
25     val limits: StateFlow<List<UserLimits>> = repository.getLimits()
26         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), emptyList())
27
28     val todayLog: StateFlow<DailyLog?> = repository.getTodayLog()
29         .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), null)
30
31     /** Exposes the repository so the screen can trigger an immediate cloud sync. */
32     fun repositoryForSync(): UsageRepository = repository
33
34     fun updateLimit(limits: UserLimits) {
35         viewModelScope.launch {
36             repository.updateLimits(limits)
37         }
38     }
39
40     fun updateGlobalLimits(reelLimit: Int, minuteLimit: Int) {
41         viewModelScope.launch {
42             Platform.entries.forEach { platform ->
43                 repository.updateLimits(UserLimits(platform.name, reelLimit, minuteLimit))
44             }
45         }
46     }
47 }

```

Thin by design: live limits + today's log as StateFlows, repositoryForSync() so the screen can hand the repository to UsageSyncManager for the immediate sync after enabling backup, and updateGlobalLimits() which writes the *same* reel/minute values to all four platform rows — the UI's 'one global limit' model on top of a per-platform schema. Each write goes through viewModelScope.launch because the DAO upsert is a suspend function.

LimitsScreen.kt — the settings hub ('Goals' tab)

.../ui/screens/limits/LimitsScreen.kt · 702 lines · Screen · 702 lines

Five cards: daily limit sliders, app blocking (toggle + mode), appearance, account/cloud backup, and HUD size. This screen is where most of the brainrot_prefs keys get written.

```

 1 package com.example.brainrottracker.ui.screens.limits
 2
 3 import android.content.Context
 4 import androidx.compose.foundation.Image
 5 import androidx.compose.foundation.background
 6 import androidx.compose.foundation.border
 7 import androidx.compose.foundation.clickable
 8 import androidx.compose.foundation.layout.Arrangement
 9 import androidx.compose.foundation.layout.Box
10 import androidx.compose.foundation.layout.Column
11 import androidx.compose.foundation.layout.Row
12 import androidx.compose.foundation.layout.Spacer
13 import androidx.compose.foundation.layout.fillMaxSize
14 import androidx.compose.foundation.layout.fillMaxWidth
15 import androidx.compose.foundation.layout.height
16 import androidx.compose.foundation.layout.padding
17 import androidx.compose.foundation.layout.size
18 import androidx.compose.foundation.lazy.LazyColumn
19 import androidx.compose.foundation.shape.CircleShape
20 import androidx.compose.foundation.shape.RoundedCornerShape
21 import androidx.compose.material3.HorizontalDivider
22 import androidx.compose.material3.Slider
23 import androidx.compose.material3.SliderDefaults
24 import androidx.compose.material3.Text
25 import androidx.compose.runtime.Composable
26 import androidx.compose.runtime.collectAsState
27 import androidx.compose.runtime.getValue

```

```

28 import androidx.compose.runtime.mutableFloatStateOf
29 import androidx.compose.runtime.mutableStateOf
30 import androidx.compose.runtime.remember
31 import androidx.compose.runtime.setValue
32 import androidx.compose.ui.Alignment
33 import androidx.compose.ui.Modifier
34 import androidx.compose.ui.draw.clip
35 import androidx.compose.ui.graphics.Color
36 import androidx.compose.ui.platform.LocalContext
37 import androidx.compose.ui.text.font.FontWeight
38 import androidx.compose.ui.unit.dp
39 import androidx.compose.ui.unit.sp
40 import androidx.compose.runtime.rememberCoroutineScope
41 import androidx.compose.ui.res.painterResource
42 import androidx.lifecycle.viewmodel.compose.viewModel
43 import com.example.brainrottracker.R
44 import com.example.brainrottracker.data.local.prefs.AppPreferences
45 import com.example.brainrottracker.data.sync.UsageSyncManager
46 import com.example.brainrottracker.service.BlockingMode
47 import com.example.brainrottracker.theme.ThemeController
48 import com.google.firebase.FirebaseApp
49 import com.google.firebase.auth.FirebaseAuth
50 import kotlinx.coroutines.Dispatchers
51 import kotlinx.coroutines.launch
52 import com.example.brainrottracker.theme.rememberIsDark
53 import com.example.brainrottracker.theme.WarmAccent
54 import com.example.brainrottracker.theme.WarmBackground
55 import com.example.brainrottracker.theme.WarmBorder
56 import com.example.brainrottracker.theme.WarmLightBackground
57 import com.example.brainrottracker.theme.WarmLightBorder
58 import com.example.brainrottracker.theme.WarmLightInner
59 import com.example.brainrottracker.theme.WarmLightSurface
60 import com.example.brainrottracker.theme.WarmLightText
61 import com.example.brainrottracker.theme.WarmLightTextSecondary
62 import com.example.brainrottracker.theme.WarmSurface
63 import com.example.brainrottracker.theme.WarmText
64 import com.example.brainrottracker.theme.WarmTextSecondary
65

```

Imports — unusually for a screen, Firebase classes appear: sign-out is handled right here.

```

66 @Composable
67 fun LimitsScreen(
68     modifier: Modifier = Modifier,
69     onNavigateToSignIn: () -> Unit = {},
70     viewModel: LimitsViewModel = viewModel()
71 ) {
72     val dark = rememberIsDark()
73     val bg = if (dark) WarmBackground else WarmLightBackground
74     val surface = if (dark) WarmSurface else WarmLightSurface
75     val cardBorder = if (dark) WarmBorder else WarmLightBorder
76     val trackBg = if (dark) com.example.brainrottracker.theme.WarmStepDim else WarmLightInner
77     val textPrimary = if (dark) WarmText else WarmLightText
78     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
79
80     val limits by viewModel.limits.collectAsState()
81     val context = LocalContext.current
82     val sharedPrefs = remember { context.getSharedPreferences("brainrot_prefs", Context.MODE_PRIVATE) }
83
84     val initialReelLimit = limits.firstOrNull()?.dailyReelLimit?.toFloat() ?: 50f
85     val initialMinuteLimit = limits.firstOrNull()?.dailyMinuteLimit?.toFloat() ?: 60f
86
87     var reelLimit by remember(initialReelLimit) { mutableFloatStateOf(initialReelLimit) }
88     var minuteLimit by remember(initialMinuteLimit) { mutableFloatStateOf(initialMinuteLimit) }
89     var blockingEnabled by remember { mutableStateOf(sharedPrefs.getBoolean("blocking_enabled", false)) }
90     var blockingMode by remember {
91         mutableStateOf(BlockingMode.fromPref(sharedPrefs.getString(BlockingMode.PREF_KEY, null)))
92     }
93     var hudScale by remember { mutableFloatStateOf(sharedPrefs.getFloat("hud_scale", 1.2f)) }
94
95     val signedInUser by remember { AppPreferences.userFlow(context) }
96         .collectAsState(initial = null)
97     var cloudSyncEnabled by remember {
98         mutableStateOf(sharedPrefs.getBoolean(UsageSyncManager.KEY_ENABLED, false))
99     }
100     val scope = rememberCoroutineScope()

```

State plumbing. The sliders' local state uses **remember(initialReelLimit)** — keyed remember, so when the DB value first loads (or changes elsewhere) the slider snaps to it, but while the user drags, the local float is the source of truth. Blocking, mode, and HUD scale mirror their pref values; signedInUser collects the DataStore flow.

```

102     Box(
103         modifier = modifier
104         .fillMaxSize()
105         .background(bg)
106     ) {
107         LazyColumn(modifier = Modifier.fillMaxSize()) {
108
109             // Header
110             item {
111                 Column(
112                     modifier = Modifier
113                     .fillMaxWidth()
114                     .padding(horizontal = 20.dp)
115                     .padding(top = 32.dp, bottom = 12.dp)
116                 ) {
117                     Row(
118                         verticalAlignment = Alignment.CenterVertically,
119                         horizontalArrangement = Arrangement.spacedBy(8.dp)
120                     ) {
121                         Text("✱", color = textPrimary, fontSize = 18.sp)
122                         Text(
123                             "SETTINGS",
124                             color = textSecondary,
125                             fontSize = 13.sp,
126                             fontWeight = FontWeight.Medium,
127                             letterSpacing = 1.5.sp
128                         )
129                     }
130                     Spacer(Modifier.height(12.dp))
131                     Text(
132                         "Daily Limits",
133                         fontWeight = FontWeight.Normal,
134                         color = textPrimary,
135                         fontSize = 36.sp,
136                         lineHeight = 41.sp,
137                         letterSpacing = (-0.5).sp
138                     )
139                 }
140             }

```

Header: '✱ SETTINGS' eyebrow + 'Daily Limits' serif-scale title.

```

141
142     // Global daily limits card
143     item {
144         Box(
145             modifier = Modifier
146             .fillMaxWidth()
147             .padding(horizontal = 20.dp)
148             .padding(bottom = 8.dp)
149             .clip(RoundedCornerShape(12.dp))
150             .background(surface)
151             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
152             .padding(24.dp)
153         ) {
154             Column(verticalArrangement = Arrangement.spacedBy(20.dp)) {
155                 Row(
156                     modifier = Modifier.fillMaxWidth(),
157                     verticalAlignment = Alignment.CenterVertically,
158                     horizontalArrangement = Arrangement.spacedBy(8.dp)
159                 ) {
160                     Text("■", fontSize = 18.sp)
161                     Column(modifier = Modifier.weight(1f)) {
162                         Text(
163                             "Daily Video Limit",
164                             fontWeight = FontWeight.Medium,
165                             color = textPrimary,
166                             fontSize = 16.sp,
167                             lineHeight = 22.sp
168                         )
169                         Text(
170                             "Applies to all platforms",
171                             color = textSecondary,
172                             fontSize = 14.sp,
173                             lineHeight = 20.sp
174                         )
175                     }
176                     Image(
177                         painterResource(R.drawable.char_excited),
178                         contentDescription = null,
179                         modifier = Modifier.size(64.dp)
180                     )
181                 }
182
183                 // Video limit slider

```

```

184         Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
185             Row(
186                 modifier = Modifier.fillMaxWidth(),
187                 horizontalArrangement = Arrangement.SpaceBetween,
188                 verticalAlignment = Alignment.CenterVertically
189             ) {
190                 Text("Videos per day", color = textSecondary, fontSize = 14.sp)
191                 Text(
192                     "${reelLimit.toInt()} videos",
193                     fontWeight = FontWeight.Medium,
194                     color = textPrimary,
195                     fontSize = 13.sp
196                 )
197             }
198             Slider(
199                 value = reelLimit,
200                 onValueChange = { reelLimit = it },
201                 onValueChangeFinished = {
202                     viewModel.updateGlobalLimits(reelLimit.toInt(), minuteLimit.toInt())
203                 },
204                 valueRange = 0f..200f,
205                 modifier = Modifier.fillMaxWidth(),
206                 colors = SliderDefaults.colors(
207                     thumbColor = WarmAccent,
208                     activeTrackColor = WarmAccent,
209                     inactiveTrackColor = trackBg,
210                     activeTickColor = Color.Transparent,
211                     inactiveTickColor = Color.Transparent
212                 )
213             )
214         }
215
216         HorizontalDivider(color = cardBorder, thickness = 1.dp)
217
218         // Time limit slider
219         Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
220             Row(
221                 modifier = Modifier.fillMaxWidth(),
222                 horizontalArrangement = Arrangement.SpaceBetween,
223                 verticalAlignment = Alignment.CenterVertically
224             ) {
225                 Text("Time per day", color = textSecondary, fontSize = 14.sp)
226                 Text(
227                     "${minuteLimit.toInt()} min",
228                     fontWeight = FontWeight.Medium,
229                     color = textPrimary,
230                     fontSize = 13.sp
231                 )
232             }
233             Slider(
234                 value = minuteLimit,
235                 onValueChange = { minuteLimit = it },
236                 onValueChangeFinished = {
237                     viewModel.updateGlobalLimits(reelLimit.toInt(), minuteLimit.toInt())
238                 },
239                 valueRange = 0f..240f,
240                 modifier = Modifier.fillMaxWidth(),
241                 colors = SliderDefaults.colors(
242                     thumbColor = WarmAccent,
243                     activeTrackColor = WarmAccent,
244                     inactiveTrackColor = trackBg,
245                     activeTickColor = Color.Transparent,
246                     inactiveTickColor = Color.Transparent
247                 )
248             )
249         }
250     }
251 }
252

```

The limits card: two sliders (0–200 videos, 0–240 min) with live value readouts. The key detail is **onValueChangeFinished** — the DB write (all four platforms) happens only when the thumb is released, not on every pixel of drag; **onValueChange** just updates the local float for a smooth UI.

```

253
254     // App blocking toggle
255     item {
256         Box(
257             modifier = Modifier
258                 .fillMaxWidth()
259                 .padding(horizontal = 20.dp)
260                 .padding(bottom = 8.dp)
261                 .clip(RoundedCornerShape(12.dp))

```

```

262         .background(surface)
263         .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
264         .padding(24.dp)
265     ) {
266         Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
267             Row(
268                 modifier = Modifier.fillMaxWidth(),
269                 horizontalArrangement = Arrangement.SpaceBetween,
270                 verticalAlignment = Alignment.CenterVertically
271             ) {
272                 Row(
273                     verticalAlignment = Alignment.CenterVertically,
274                     horizontalArrangement = Arrangement.spacedBy(8.dp),
275                     modifier = Modifier.weight(1f)
276                 ) {
277                     Text("■", fontSize = 18.sp)
278                     Column {
279                         Text(
280                             "App Blocking",
281                             fontWeight = FontWeight.Medium,
282                             color = textPrimary,
283                             fontSize = 16.sp,
284                             lineHeight = 22.sp
285                         )
286                         Text(
287                             "Show overlay when daily limit is reached",
288                             color = textSecondary,
289                             fontSize = 14.sp,
290                             lineHeight = 20.sp
291                         )
292                     }
293                 }
294                 WarmToggle(
295                     checked = blockingEnabled,
296                     onCheckedChange = {
297                         blockingEnabled = it
298                         sharedPrefs.edit().putBoolean("blocking_enabled", it).apply()
299                     }
300                 )
301             }
302             if (blockingEnabled) {
303                 HorizontalDivider(color = cardBorder, thickness = 1.dp)
304                 BlockingModeSelector(
305                     selected = blockingMode,
306                     onSelect = {
307                         blockingMode = it
308                         sharedPrefs.edit().putString(BlockingMode.PREF_KEY, it.name).apply()
309                     },
310                     trackBg = trackBg,
311                     textSecondary = textSecondary
312                 )
313                 Row(
314                     modifier = Modifier.fillMaxWidth(),
315                     verticalAlignment = Alignment.CenterVertically,
316                     horizontalArrangement = Arrangement.spacedBy(12.dp)
317                 ) {
318                     Text(
319                         when (blockingMode) {
320                             BlockingMode.HARD -> "Blocks the app until midnight – the only way out i
321                             BlockingMode.SNOOZE -> "Dismissing gives you 5 more minutes, then the bl
322                             BlockingMode.REMIND -> "Reminds you once each time you open the app"
323                         },
324                     color = textSecondary,
325                     fontSize = 13.sp,
326                     lineHeight = 18.sp,
327                     modifier = Modifier.weight(1f)
328                 )
329                 Image(
330                     painterResource(R.drawable.char_stop),
331                     contentDescription = null,
332                     modifier = Modifier.size(60.dp)
333                 )
334             }
335         }
336     }
337 }
338

```

The blocking card: the custom WarmToggle writes blocking_enabled (read each evaluation by ReelCounterService — changes apply immediately, no restart). When enabled, the three-segment mode selector appears, writes blocking_mode, and a mode-specific explainer sentence plus the stop-character preview what the user signed up for.


```

339
340 // Appearance / theme selector
341 item {
342     Box(
343         modifier = Modifier
344             .fillMaxWidth()
345             .padding(horizontal = 20.dp)
346             .padding(bottom = 8.dp)
347             .clip(RoundedCornerShape(12.dp))
348             .background(surface)
349             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
350             .padding(24.dp)
351     ) {
352         Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
353             Row(
354                 modifier = Modifier.fillMaxWidth(),
355                 verticalAlignment = Alignment.CenterVertically,
356                 horizontalArrangement = Arrangement.spacedBy(8.dp)
357             ) {
358                 Text("■", fontSize = 18.sp)
359                 Column(modifier = Modifier.weight(1f)) {
360                     Text(
361                         "Appearance",
362                         fontWeight = FontWeight.Medium,
363                         color = textPrimary,
364                         fontSize = 16.sp,
365                         lineHeight = 22.sp
366                     )
367                     Text(
368                         "Choose your theme",
369                         color = textSecondary,
370                         fontSize = 14.sp,
371                         lineHeight = 20.sp
372                     )
373                 }
374                 Image(
375                     painterResource(R.drawable.char_thinking),
376                     contentDescription = null,
377                     modifier = Modifier.size(56.dp)
378                 )
379             }
380             ThemeModeSelector(
381                 selected = ThemeController.mode,
382                 onSelect = { ThemeController.selectMode(it) },
383                 trackBg = trackBg,
384                 textPrimary = textPrimary,
385                 textSecondary = textSecondary
386             )
387         }
388     }
389 }

```

Appearance: the three-segment Light/Dark/System selector bound straight to ThemeController — selecting recolors the entire app in the same frame (Compose state), persists the pref (services/widget pick it up), no confirmation needed.

```

390
391 // Account / cloud backup
392 item {
393     Box(
394         modifier = Modifier
395             .fillMaxWidth()
396             .padding(horizontal = 20.dp)
397             .padding(bottom = 8.dp)
398             .clip(RoundedCornerShape(12.dp))
399             .background(surface)
400             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
401             .padding(24.dp)
402     ) {
403         Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
404             Row(
405                 modifier = Modifier.fillMaxWidth(),
406                 verticalAlignment = Alignment.CenterVertically,
407                 horizontalArrangement = Arrangement.spacedBy(8.dp)
408             ) {
409                 Text("■", fontSize = 18.sp)
410                 Column(modifier = Modifier.weight(1f)) {
411                     Text(
412                         "Account",
413                         fontWeight = FontWeight.Medium,
414                         color = textPrimary,
415                         fontSize = 16.sp,
416                         lineHeight = 22.sp

```

```

417         )
418         Text(
419             signedInUser?.let { it.email.ifEmpty { it.name } }
420                 ?: "Sign in to back up your stats",
421             color = textSecondary,
422             fontSize = 14.sp,
423             lineHeight = 20.sp
424         )
425     }
426     Image(
427         painterResource(R.drawable.char_thankyou),
428         contentDescription = null,
429         modifier = Modifier.size(64.dp)
430     )
431 }
432 val user = signedInUser
433 if (user == null) {
434     Box(
435         modifier = Modifier
436             .fillMaxWidth()
437             .clip(RoundedCornerShape(8.dp))
438             .background(WarmAccent)
439             .clickable { onNavigateToSignIn() }
440             .padding(vertical = 10.dp),
441         contentAlignment = Alignment.Center
442     ) {
443         Text(
444             "Sign in with Google",
445             color = Color.White,
446             fontSize = 14.sp,
447             fontWeight = FontWeight.Medium
448         )
449     }
450 } else {
451     Row(
452         modifier = Modifier.fillMaxWidth(),
453         horizontalArrangement = Arrangement.SpaceBetween,
454         verticalAlignment = Alignment.CenterVertically
455     ) {
456         Column(modifier = Modifier.weight(1f)) {
457             Text(
458                 "Cloud backup",
459                 color = textPrimary,
460                 fontSize = 14.sp,
461                 fontWeight = FontWeight.Medium
462             )
463             Text(
464                 "Upload daily totals to your private account",
465                 color = textSecondary,
466                 fontSize = 13.sp,
467                 lineHeight = 18.sp
468             )
469         }
470         WarmToggle(
471             checked = cloudSyncEnabled,
472             onCheckedChange = { enabled ->
473                 cloudSyncEnabled = enabled
474                 sharedPrefs.edit()
475                     .putBoolean(UsageSyncManager.KEY_ENABLED, enabled)
476                     .apply()
477                 if (enabled) {
478                     scope.launch(Dispatchers.IO) {
479                         try {
480                             UsageSyncManager(
481                                 context.applicationContext,
482                                 viewModel.renositorvForSvnc()
483                             ).syncIfEnabled()
484                         } catch (_: Exception) {
485                         }
486                     }
487                 }
488             }
489         )
490     }
491     Box(
492         modifier = Modifier
493             .fillMaxWidth()
494             .clip(RoundedCornerShape(8.dp))
495             .border(1.dp, cardBorder, RoundedCornerShape(8.dp))
496             .clickable {
497                 scope.launch {
498                     cloudSyncEnabled = false
499                     sharedPrefs.edit()
500                         .putBoolean(UsageSyncManager.KEY_ENABLED, false)
501                         .remove(UsageSyncManager.KEY_LAST_SYNCED)

```

```

502         .apply()
503         if (FirebaseApp.getApps(context).isEmpty()) {
504             FirebaseAuth.getInstance().signOut()
505         }
506         AppPreferences.signOut(context)
507     }
508 }
509 .padding(vertical = 10.dp),
510 contentAlignment = Alignment.Center
511 ) {
512     Text(
513         "Sign out",
514         color = textSecondary,
515         fontSize = 14.sp,
516         fontWeight = FontWeight.Medium
517     )
518 }
519 }
520 }
521 }
522 }

```

The account card, both states. Signed out: a terracotta 'Sign in with Google' button pushing the sign-in screen. Signed in: the email in the header, the **cloud backup toggle** (writes the consent pref; enabling kicks an immediate background sync if `isEnabled()` so the user sees the backup happen *now*), and 'Sign out' — which carefully disables sync, clears `last_synced_date` (so a future sign-in re-uploads history), signs out of Firebase (guarded for the no-config case), and wipes the `DataStore`.

```

523
524 // Floating counter (HUD) size
525 item {
526     Box(
527         modifier = Modifier
528             .fillMaxWidth()
529             .padding(horizontal = 20.dp)
530             .padding(bottom = 24.dp)
531             .clip(RoundedCornerShape(12.dp))
532             .background(surface)
533             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
534             .padding(24.dp)
535     ) {
536         Column(verticalArrangement = Arrangement.spacedBy(16.dp)) {
537             Row(
538                 verticalAlignment = Alignment.CenterVertically,
539                 horizontalArrangement = Arrangement.spacedBy(8.dp)
540             ) {
541                 Text("■", fontSize = 18.sp)
542                 Column {
543                     Text(
544                         "Floating Counter",
545                         fontWeight = FontWeight.Medium,
546                         color = textPrimary,
547                         fontSize = 16.sp,
548                         lineHeight = 22.sp
549                     )
550                     Text(
551                         "Adjust the on-screen counter size",
552                         color = textSecondary,
553                         fontSize = 14.sp,
554                         lineHeight = 20.sp
555                     )
556                 }
557             }
558             Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
559                 Row(
560                     modifier = Modifier.fillMaxWidth(),
561                     horizontalArrangement = Arrangement.SpaceBetween,
562                     verticalAlignment = Alignment.CenterVertically
563                 ) {
564                     Text("Size", color = textSecondary, fontSize = 14.sp)
565                     Text(
566                         "${(hudScale * 100).toInt()}%",
567                         fontWeight = FontWeight.Medium,
568                         color = textPrimary,
569                         fontSize = 13.sp
570                     )
571                 }
572                 Slider(
573                     value = hudScale,
574                     onValueChange = { hudScale = it },
575                     onValueChangeFinished = {
576                         sharedPrefs.edit().putFloat("hud_scale", hudScale).apply()
577                     }
578                 )
579             }
580         }
581     }
582 }

```

```

577         },
578         valueRange = 0.8f..1.8f,
579         modifier = Modifier.fillMaxWidth(),
580         colors = SliderDefaults.colors(
581             thumbColor = WarmAccent,
582             activeTrackColor = WarmAccent,
583             inactiveTrackColor = trackBg,
584             activeTickColor = Color.Transparent,
585             inactiveTickColor = Color.Transparent
586         )
587     )
588 }
589 }
590 }
591 }
592 }
593 }
594 }

```

HUD size: a 0.8–1.8× slider writing `hud_scale` on release; `FloatingCounterService` re-reads it on every `show()`, so the pill resizes the next time it appears.

```

595
596 @Composable
597 private fun BlockingModeSelector(
598     selected: BlockingMode,
599     onSelect: (BlockingMode) -> Unit,
600     trackBg: Color,
601     textSecondary: Color
602 ) {
603     val options = listOf(
604         BlockingMode.HARD to "■ Hard",
605         BlockingMode.SNOOZE to "■ Snooze",
606         BlockingMode.REMIND to "■ Remind",
607     )
608     Row(
609         modifier = Modifier
610             .fillMaxWidth()
611             .clip(RoundedCornerShape(8.dp))
612             .background(trackBg)
613             .padding(4.dp),
614         horizontalArrangement = Arrangement.spacedBy(4.dp)
615     ) {
616         options.forEach { (mode, label) ->
617             val isSelected = mode == selected
618             Box(
619                 modifier = Modifier
620                     .weight(1f)
621                     .clip(RoundedCornerShape(6.dp))
622                     .background(if (isSelected) WarmAccent else Color.Transparent)
623                     .clickable { onSelect(mode) }
624                     .padding(vertical = 8.dp),
625                 contentAlignment = Alignment.Center
626             ) {
627                 Text(
628                     label,
629                     color = if (isSelected) Color.White else textSecondary,
630                     fontSize = 13.sp,
631                     fontWeight = FontWeight.Medium
632                 )
633             }
634         }
635     }
636 }
637
638 @Composable
639 private fun ThemeModeSelector(
640     selected: ThemeController.Mode,
641     onSelect: (ThemeController.Mode) -> Unit,
642     trackBg: Color,
643     textPrimary: Color,
644     textSecondary: Color
645 ) {
646     val options = listOf(
647         ThemeController.Mode.LIGHT to "■ Light",
648         ThemeController.Mode.DARK to "■ Dark",
649         ThemeController.Mode.SYSTEM to "■ System",
650     )
651     Row(
652         modifier = Modifier
653             .fillMaxWidth()
654             .clip(RoundedCornerShape(8.dp))
655             .background(trackBg)
656             .padding(4.dp),

```

```

657     horizontalArrangement = Arrangement.spacedBy(4.dp)
658   ) {
659     options.forEach { (mode, label) ->
660       val isSelected = mode == selected
661       Box(
662         modifier = Modifier
663           .weight(1f)
664           .clip(RoundedCornerShape(6.dp))
665           .background(if (isSelected) WarmAccent else Color.Transparent)
666           .clickable { onSelect(mode) }
667           .padding(vertical = 8.dp),
668         contentAlignment = Alignment.Center
669       ) {
670         Text(
671           label,
672           color = if (isSelected) Color.White else textSecondary,
673           fontSize = 13.sp,
674           fontWeight = FontWeight.Medium
675         )
676       }
677     }
678   }
679 }

```

BlockingModeSelector and ThemeModeSelector — the same segmented-control pattern: a rounded track Row, each option an equal-weight Box, the selected one filled terracotta with white text. Stateless; selection comes in, events go out.

```

680
681 @Composable
682 private fun WarmToggle(
683     checked: Boolean,
684     onCheckedChange: (Boolean) -> Unit
685 ) {
686     Box(
687         modifier = Modifier
688           .size(width = 48.dp, height = 28.dp)
689           .clip(CircleShape)
690           .background(if (checked) WarmAccent else WarmLightBorder)
691           .clickable { onCheckedChange(!checked) }
692           .padding(4.dp),
693         contentAlignment = if (checked) Alignment.CenterEnd else Alignment.CenterStart
694     ) {
695         Box(
696             modifier = Modifier
697               .size(20.dp)
698               .clip(CircleShape)
699               .background(Color.White)
700         )
701     }
702 }

```

WarmToggle — a switch in 20 lines: a 48×28 pill whose contentAlignment flips CenterStart/CenterEnd to slide the white knob, terracotta when on. Simpler than restyling Material's Switch to match the warm palette.

StreaksViewModel.kt

.../ui/screens/streaks/StreaksViewModel.kt · 59 lines · ViewModel · 59 lines

```

1  package com.example.brainrottracker.ui.screens.streaks
2
3  import android.app.Application
4  import androidx.lifecycle.AndroidViewModel
5  import androidx.lifecycle.ViewModelScope
6  import com.example.brainrottracker.data.local.db.AppDatabase
7  import com.example.brainrottracker.data.local.db.entity.StreakRecord
8  import com.example.brainrottracker.data.repository.UsageRepository
9  import com.example.brainrottracker.data.util.ScreenTimeHelper
10 import kotlinx.coroutines.Dispatchers
11 import kotlinx.coroutines.flow.SharingStarted
12 import kotlinx.coroutines.flow.StateFlow
13 import kotlinx.coroutines.flow.combine
14 import kotlinx.coroutines.flow.flow
15 import kotlinx.coroutines.flow.flowOn
16 import kotlinx.coroutines.flow.stateIn
17
18 data class StreakUiState(
19     val records: List<StreakRecord> = emptyList(),
20     val currentStreak: Int = 0,
21     val longestStreak: Int = 0,
22     val streakMap: Map<String, Boolean> = emptyMap(),

```

```

23     val hasZeroReelDay: Boolean = false,
24     val hasHalfHourDay: Boolean = false,
25 )
26
27 class StreaksViewModel(application: Application) : AndroidViewModel(application) {
28
29     private val repository: UsageRepository
30
31     init {
32         val db = AppDatabase.getInstance(application)
33         repository = UsageRepository(db)
34     }
35
36     // Yesterday's screen-time minutes from UsageStats (minutes aren't stored in the DB)
37     private val yesterdayMinutes = flow {
38         emit(ScreenTimeHelper.getYesterdayMinutesByPlatform(getApplication()).values.sum())
39     }.flowOn(Dispatchers.IO)
40
41     val streakState: StateFlow<StreakUiState> = combine(
42         repository.getStreakData(),
43         repository.getMonthlyLogs(),
44         yesterdayMinutes
45     ) { records, logs, minutes ->
46         // A "Zero Reel Day" counts only for days that were actually tracked
47         // (a log row exists), so empty/untouched days don't unlock it for free.
48         val hasZeroReelDay = logs.any { it.getTotalReels() == 0 }
49         val hasHalfHourDay = minutes in 1..30
50         StreakUiState(
51             records = records,
52             currentStreak = repository.calculateCurrentStreak(records),
53             longestStreak = repository.calculateLongestStreak(records),
54             streakMap = records.associate { it.date to it.underLimit },
55             hasZeroReelDay = hasZeroReelDay,
56             hasHalfHourDay = hasHalfHourDay
57         )
58     }.stateIn(viewModelScope, SharingStarted.WhileSubscribed(5000), StreakUiState())
59 }

```

One `StreakUiState` `StateFlow` from three sources: all streak records, 30 days of logs, and yesterday's minutes. Computes current/longest streak via the repository helpers, a date→verdict map for the calendar, and the two special achievements: 'Zero Reel Day' requires a day that *has a log row* with zero total (the comment explains why: untracked days shouldn't unlock it for free), and 'Half Hour Hero' checks yesterday's minutes $\in 1\text{--}30$ (a 0 likely means no usage-permission data, so it doesn't count).

StreaksScreen.kt — calendar & achievements

.../ui/screens/streaks/StreaksScreen.kt · 403 lines · Screen · 403 lines

```

1 package com.example.brainrottracker.ui.screens.streaks
2
3 import androidx.compose.foundation.Image
4 import androidx.compose.foundation.background
5 import androidx.compose.foundation.border
6 import androidx.compose.foundation.layout.Arrangement
7 import androidx.compose.foundation.layout.Box
8 import androidx.compose.foundation.layout.Column
9 import androidx.compose.foundation.layout.Row
10 import androidx.compose.foundation.layout.Spacer
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.height
14 import androidx.compose.foundation.layout.padding
15 import androidx.compose.foundation.layout.size
16 import androidx.compose.foundation.layout.width
17 import androidx.compose.foundation.lazy.LazyColumn
18 import androidx.compose.foundation.shape.CircleShape
19 import androidx.compose.foundation.shape.RoundedCornerShape
20 import androidx.compose.material.icons.Icons
21 import androidx.compose.material.icons.filled.Lock
22 import androidx.compose.material3.Icon
23 import androidx.compose.material3.Text
24 import androidx.compose.runtime.Composable
25 import androidx.compose.runtime.collectAsState
26 import androidx.compose.runtime.getValue
27 import androidx.compose.ui.Alignment
28 import androidx.compose.ui.Modifier
29 import androidx.compose.ui.draw.clip
30 import androidx.compose.ui.graphics.Color
31 import androidx.compose.ui.text.font.FontWeight
32 import androidx.compose.ui.text.style.TextAlign
33 import androidx.compose.ui.unit.dp

```

```

34 import androidx.compose.ui.unit.sp
35 import androidx.lifecycle.viewmodel.compose.viewModel
36 import androidx.compose.ui.res.painterResource
37 import com.example.brainrottracker.R
38 import com.example.brainrottracker.theme.WarmAccent
39 import com.example.brainrottracker.theme.rememberIsDark
40 import com.example.brainrottracker.theme.WarmBackground
41 import com.example.brainrottracker.theme.WarmBorder
42 import com.example.brainrottracker.theme.WarmError
43 import com.example.brainrottracker.theme.WarmGrantedGreen
44 import com.example.brainrottracker.theme.WarmLightBackground
45 import com.example.brainrottracker.theme.WarmLightBorder
46 import com.example.brainrottracker.theme.WarmLightInner
47 import com.example.brainrottracker.theme.WarmLightSurface
48 import com.example.brainrottracker.theme.WarmLightText
49 import com.example.brainrottracker.theme.WarmLightTextSecondary
50 import com.example.brainrottracker.theme.WarmStepDim
51 import com.example.brainrottracker.theme.WarmSurface
52 import com.example.brainrottracker.theme.WarmText
53 import com.example.brainrottracker.theme.WarmTextSecondary
54 import java.time.LocalDate
55 import java.time.YearMonth
56
57 data class Achievement(
58     val emoji: String,
59     val title: String,
60     val description: String,
61     val requirement: Int,
62 )
63
64 @Composable
65 fun StreaksScreen(
66     modifier: Modifier = Modifier,
67     viewModel: StreaksViewModel = viewModel()
68 ) {
69     val dark = rememberIsDark()
70     val bg = if (dark) WarmBackground else WarmLightBackground
71     val surface = if (dark) WarmSurface else WarmLightSurface
72     val cardBorder = if (dark) WarmBorder else WarmLightBorder
73     val futureDay = if (dark) WarmStepDim else WarmLightInner
74     val futureDayText = if (dark) WarmTextSecondary else WarmLightTextSecondary
75     val textPrimary = if (dark) WarmText else WarmLightText
76     val textSecondary = if (dark) WarmTextSecondary else WarmLightTextSecondary
77
78     val state by viewModel.streakState.collectAsState()
79
80     val achievements = listOf(
81         Achievement("🟩", "First Step", "1 day under limit", 1),
82         Achievement("🟩🟩", "7-Day Warrior", "7 consecutive days", 7),
83         Achievement("🟩🟩", "30-Day Legend", "30 consecutive days", 30),
84         Achievement("🟩", "Zero Reel Day", "0 videos watched", 0),
85         Achievement("🟩", "Half Hour Hero", "Under 30 min total", 0),
86     )

```

Setup + the achievement definitions — a local list of five (emoji, title, requirement); the unlock logic lives at render time below.

```

87
88 LazyColumn(
89     modifier = modifier
90         .fillMaxSize()
91         .background(bg),
92 ) {
93     // Title
94     item {
95         Text(
96             "Streaks & Progress",
97             fontWeight = FontWeight.Normal,
98             color = textPrimary,
99             fontSize = 36.sp,
100             lineHeight = 41.sp,
101             letterSpacing = (-0.5).sp,
102             modifier = Modifier
103                 .fillMaxWidth()
104                 .padding(horizontal = 20.dp)
105                 .padding(top = 32.dp, bottom = 24.dp)
106         )
107     }
108
109     // Streak hero card
110     item {
111         Box(
112             modifier = Modifier
113                 .fillMaxWidth()

```

```

114         .padding(horizontal = 20.dp)
115         .padding(bottom = 24.dp)
116         .clip(RoundedCornerShape(16.dp))
117         .background(surface)
118         .border(1.dp, cardBorder, RoundedCornerShape(16.dp))
119         .padding(horizontal = 24.dp, vertical = 32.dp),
120         contentAlignment = Alignment.Center
121     ) {
122         Column(
123             horizontalAlignment = Alignment.CenterHorizontally
124         ) {
125             Row(
126                 modifier = Modifier.fillMaxWidth(),
127                 verticalAlignment = Alignment.CenterVertically
128             ) {
129                 Column(
130                     horizontalAlignment = Alignment.CenterHorizontally,
131                     modifier = Modifier.weight(1f)
132                 ) {
133                     Text("■", fontSize = 32.sp)
134                     Spacer(Modifier.height(16.dp))
135                     Text(
136                         "${state.currentStreak}",
137                         color = textPrimary,
138                         fontSize = 64.sp,
139                         fontWeight = FontWeight.Normal,
140                         lineHeight = 67.sp,
141                         letterSpacing = (-1.5).sp
142                     )
143                     Spacer(Modifier.height(8.dp))
144                     Text(
145                         "DAY STREAK",
146                         color = textSecondary,
147                         fontSize = 11.sp,
148                         fontWeight = FontWeight.Medium,
149                         letterSpacing = 1.5.sp
150                     )
151                 }
152                 // The streak warrior cheers you on
153                 Image(
154                     painterResource(R.drawable.char_warrior),
155                     contentDescription = null,
156                     modifier = Modifier.size(132.dp)
157                 )
158             }
159             Spacer(Modifier.height(32.dp))
160             Row(
161                 modifier = Modifier.fillMaxWidth(),
162                 verticalAlignment = Alignment.CenterVertically
163             ) {
164                 Column(
165                     modifier = Modifier.weight(1f),
166                     horizontalAlignment = Alignment.CenterHorizontally
167                 ) {
168                     Text("Current", color = textSecondary, fontSize = 14.sp, lineHeight = 22.sp)
169                     Text("${state.currentStreak} days", color = textPrimary, fontSize = 14.sp, lineHeigh
170 ■ t = 22.sp)
171                 }
172                 Box(
173                     modifier = Modifier
174                         .width(1.dp)
175                         .height(40.dp)
176                         .background(if (dark) WarmBorder else WarmLightBorder)
177                 )
178                 Column(
179                     modifier = Modifier.weight(1f),
180                     horizontalAlignment = Alignment.CenterHorizontally
181                 ) {
182                     Text("Record", color = textSecondary, fontSize = 14.sp, lineHeight = 22.sp)
183                     Text("${state.longestStreak} days", color = textPrimary, fontSize = 14.sp, lineHeigh
184 ■ t = 22.sp)
185                 }
186             }
187         }

```

Title and the hero card: ■, the current streak at 64 sp, the warrior character, and a Current | Record split row fed by the two repository calculations.

```

188
189     // Calendar
190     item {
191         Box(

```



```

277         fontWeight = FontWeight.Medium
278     )
279 }
280 }
281 }
282 // Pad empty cells if week has less than 7 days
283 repeat(7 - week.size) {
284     Box(modifier = Modifier.weight(1f))
285 }
286 }
287 }
288 }
289 }
290 }

```

The month calendar — calendar math worth reading: Monday-first headers, **leadingBlanks = first-of-month.dayOfWeek.value - 1** nulls pad the first week, then chunked(7) slices the cells into week rows. Each day circle is colored by the streakMap verdict (green/red), dim for future or unevaluated days, and today gets a terracotta ring. Trailing short weeks are padded with weight-matched empty boxes so columns stay aligned.

```

291
292 // Achievements header
293 item {
294     Row(
295         modifier = Modifier
296             .fillMaxWidth()
297             .padding(horizontal = 20.dp)
298             .padding(bottom = 8.dp),
299         verticalAlignment = Alignment.CenterVertically
300     ) {
301         Text(
302             "Achievements",
303             fontWeight = FontWeight.Medium,
304             color = textPrimary,
305             fontSize = 18.sp,
306             lineHeight = 25.sp,
307             modifier = Modifier.weight(1f)
308         )
309         Image(
310             painterResource(R.drawable.char_trophy),
311             contentDescription = null,
312             modifier = Modifier.size(64.dp)
313         )
314     }
315 }
316
317 // Achievement items
318 items(achievements.size) { index ->
319     val achievement = achievements[index]
320     val isUnlocked = when (achievement.title) {
321         "First Step" -> state.longestStreak >= 1
322         "7-Day Warrior" -> state.longestStreak >= 7
323         "30-Day Legend" -> state.longestStreak >= 30
324         "Zero Reel Day" -> state.hasZeroReelDay
325         "Half Hour Hero" -> state.hasHalfHourDay
326         else -> false
327     }
328
329     Box(
330         modifier = Modifier
331             .fillMaxWidth()
332             .padding(horizontal = 20.dp)
333             .padding(bottom = 8.dp)
334             .clip(RoundedCornerShape(12.dp))
335             .background(surface)
336             .border(1.dp, cardBorder, RoundedCornerShape(12.dp))
337             .padding(16.dp)
338     ) {
339         Row(
340             verticalAlignment = Alignment.CenterVertically,
341             horizontalArrangement = Arrangement.spacedBy(16.dp)
342         ) {
343             Box(
344                 modifier = Modifier
345                     .size(40.dp)
346                     .clip(CircleShape)
347                     .background(if (dark) WarmStepDim else WarmLightInner),
348                 contentAlignment = Alignment.Center
349             ) {
350                 Text(achievement.emoji, fontSize = 18.sp)
351             }
352             Column(modifier = Modifier.weight(1f)) {
353                 Text(

```

```

354         achievement.title,
355         fontWeight = FontWeight.Medium,
356         color = textPrimary,
357         fontSize = 16.sp,
358         lineHeight = 22.sp
359     )
360     Text(
361         achievement.description,
362         color = textSecondary,
363         fontSize = 14.sp,
364         lineHeight = 22.sp
365     )
366 }
367 if (isUnlocked) {
368     Box(
369         modifier = Modifier
370             .clip(CircleShape)
371             .background(WarmGrantedGreen)
372             .padding(horizontal = 12.dp, vertical = 4.dp)
373     ) {
374         Text(
375             "DONE",
376             color = Color.White,
377             fontSize = 10.sp,
378             fontWeight = FontWeight.Medium,
379             letterSpacing = 1.5.sp
380         )
381     }
382 } else {
383     Box(
384         modifier = Modifier
385             .clip(CircleShape)
386             .background(if (dark) WarmStepDim else WarmLightBorder)
387             .padding(horizontal = 12.dp, vertical = 6.dp)
388     ) {
389         Icon(
390             Icons.Filled.Lock,
391             contentDescription = "Locked",
392             tint = textSecondary,
393             modifier = Modifier.size(14.dp)
394         )
395     }
396 }
397 }
398 }
399 }
400
401 item { Spacer(Modifier.height(24.dp)) }
402 }
403 }

```

Achievements: header with trophy art, then one card per achievement using `items(count)` — proper `LazyColumn` usage rather than a loop inside one item. Unlock rules: streak milestones compare against *longest* streak (a later broken streak can't take 'First Step' away), the special two come precomputed from the `ViewModel`. Unlocked → green DONE pill; locked → padlock chip.

Appendix · Recipes & honest limitations

Adding a fifth platform (the official recipe)

1. Add the entry to **Platform.kt** (package name, display name, emoji). 2. Add a count column to **DailyLog** and bump the version in **AppDatabase**. 3. Add the increment query in **DailyLogDao** and the case in **UsageRepository.incrementReelCount()**. 4. Add the package to **accessibility_service_config.xml**'s **packageNames**. 5. If its reel UI is unusual, add a branch in **ReelCounterService**.

Known limitations (by design or accepted trade-off)

Limitation	Detail
Destructive migrations	Schema changes wipe the DB — must become real migrations before launch.
REMIND session detection	Best-effort when no reel feed is open: the heartbeat stops once the user leaves all tracked apps.
Historical streaks vs. current limits	Limits aren't versioned; changing them re-judges the past on the next evaluation of unevaluated days.
Milestones reset on service restart	shownMilestones is in-memory; worst case a duplicate notification.
UI detection heuristics	Platform app updates can change view IDs / layouts; the YouTube fallback signature and generic pager test are the hedge.
Manual release steps	Firebase project + google-services.json, signing keystore, final applicationId, hosted privacy policy (required for the accessibility declaration).

— End of document —